LYRA CAPITAL

Guida dello sviluppatore

Architettura, motore quantitativo, estensioni e rilascio della piattaforma.

A chi serve. A chi deve *modificare* Lyra: aggiungere una strategia, cambiare il motore, toccare l'interfaccia, mettere mano al marketplace.

A chi non serve. A chi deve *usare* Lyra: quello è la *User guide*, il manuale utente in inglese, documento separato nella stessa cartella.

Documento generato da sorgente LaTeX
14 novembre 2026

Come è fatto questo documento

Quattro parti, in ordine di quanto a fondo si deve andare.

Parte I — Orientarsi
: Che cos'è Lyra, come è diviso, e come si mette in moto una copia sulla propria macchina. Si legge una volta.

Parte II — Come è fatto
: L'architettura e il motore di calcolo: dove stanno le decisioni e perché sono dove sono. È la parte da leggere prima di toccare qualunque cosa.

Parte III — Come si cambia
: Aggiungere una strategia, una modalità di pesatura, una pagina, un'operazione sul database. Ogni capitolo è una procedura che si segue con il codice aperto accanto.

Parte IV — Tenerlo in piedi
: Le prove automatiche, il rilascio, e l'elenco di ciò che si rompe più spesso con il motivo accanto.

Le convenzioni

I riquadri sono quattro e hanno quattro significati distinti.

> **Nota**
>
> Un chiarimento utile ma non necessario: saltandolo non ci si perde niente di ciò che serve per lavorare.

> **Perché così**
>
> La ragione di una scelta che, vista da fuori, sembrerebbe potersi fare diversamente. Serve a non «semplificare» qualcosa che è complicato per un motivo.

> **In pratica**
>
> Cosa fare, passo per passo. Se si cerca solo l'operazione, sono questi i riquadri da seguire.

> **Attenzione**
>
> Qui si rompe qualcosa: dati persi, denaro movimentato, o un errore che si scopre solo settimane dopo. Non sono avvisi di cortesia.

Nel testo, `questo` è un simbolo del codice, `questo/` un percorso nel progetto, **Questa** una voce dell'interfaccia e **Questo** un pulsante da premere. Ogni blocco di codice porta in alto a destra il file da cui viene: la documentazione si legge con il sorgente aperto accanto, e un frammento senza il suo file costringe a cercarlo.

Indice

I Orientarsi 1

1 Che cos'è Lyra . . . 2
1.1 Il problema che risolve . . . 2
1.2 Le due metà . . . 3
1.2.1 La metà che gira sul computer dell'utente . . . 3
1.2.2 La metà che gira in rete . . . 4
1.2.3 I tre canali, e perché sono tre . . . 4
1.3 Le due applicazioni dentro la metà locale . . . 4
1.4 Dieci parole da fissare subito . . . 4
1.5 Da dove cominciare, secondo cosa si deve fare . . . 6
1.6 Che cosa questa guida non copre . . . 6

2 Mettere in moto una copia . . . 8
2.1 Che cosa serve avere installato . . . 8
2.1.1 Installazione delle dipendenze Python . . . 8
2.1.2 Con Docker, invece . . . 9
2.2 I quattro modi di avviarla . . . 9
2.2.1 La dashboard . . . 9
2.2.2 L'API del motore . . . 10
2.2.3 Il sito . . . 10
2.3 Dove stanno i dati . . . 10
2.3.1 La cartella di lavoro . . . 11
2.3.2 Che cosa c'è sotto `data/` . . . 11
2.3.3 Portare dentro i primi dati . . . 12
2.4 La mappa delle cartelle . . . 12
2.4.1 Il guscio a strati . . . 12
2.4.2 Il motore . . . 13
2.4.3 Gli artefatti . . . 13
2.4.4 L'interfaccia, e il resto . . . 13

II Come è fatto 15

3 Come è fatto . . . 16
3.1 L'architettura esagonale, in concreto . . . 16
3.1.1 La regola, e perché è una sola . . . 16
3.1.2 `domain/`: i tipi, e basta . . . 17
3.1.3 `ports/`: trentadue contratti . . . 17
3.1.4 `application/`: i casi d'uso . . . 18
3.1.5 `composition/`: centonove righe, e sono quelle decisive . . . 18
3.2 Due applicazioni web, e il motivo . . . 19
3.2.1 Che cos'è uno strangler fig . . . 19
3.2.2 Come si avvia l'API . . . 20

3.2.3 I tredici router . . . 21
3.2.4 Dove passa il confine, oggi . . . 21
3.3 Le 249 rotte, e chi le registra . . . 21
3.3.1 L'ordine di montaggio . . . 22
3.3.2 Il caricamento automatico delle estensioni v2 . . . 22
3.3.3 Rotte statiche che coprono rotte dinamiche . . . 23
3.3.4 Tre cose morte, da non seguire . . . 24
3.3.5 Due template orfani . . . 24
3.4 Chi può fare che cosa . . . 24
3.4.1 Le tre liste . . . 25
3.4.2 I quattro livelli, sopra la guardia . . . 26
3.4.3 La scelta di fondo, e il suo prezzo . . . 26
3.4.4 Quattro punti dove la protezione non è dove ci si aspetta . . . 26
3.4.5 Gli endpoint sull'account sono quattro . . . 27

4 Il motore quantitativo . . . 28
4.1 Che cosa succede a ogni barra . . . 28
4.1.1 Prima di tutto: i trigger intra-barra . . . 28
4.2 Decisione ed esecuzione sono due istanti . . . 29
4.2.1 Perché due istanti e non uno . . . 29
4.2.2 La firma che non si può cambiare . . . 30
4.2.3 La stessa regola altrove . . . 30
4.3 Dall'intenzione alla posizione . . . 31
4.3.1 Le quattro modalità . . . 31
4.3.2 I tetti di rischio . . . 32
4.3.3 La validazione è nel costruttore . . . 32
4.4 Comporre più stream . . . 32
4.4.1 Primo stadio: portare gli stream alla stessa scala . . . 32
4.4.2 Secondo stadio: la pesatura . . . 34
4.4.3 Terzo stadio: gli overlay . . . 35
4.4.4 I sette overlay installati . . . 35
4.5 Lo stesso calcolo, su un conto vero . . . 36
4.5.1 Il difetto che ha definito il progetto . . . 36
4.5.2 I due punti in cui la normalizzazione entra . . . 36
4.5.3 Il riscaldamento . . . 36
4.5.4 I broker . . . 37
4.5.5 Simulazione o ordini reali . . . 39
4.5.6 Il registro del runner . . . 39
4.5.7 Dati vecchi . . . 39
4.5.8 Il kill-switch del conto . . . 40
4.5.9 Il ciclo dal vivo . . . 41

III Come si cambia 43

5 Aggiungere un artefatto . . . 44
5.1 Non c'è un contratto solo: ce ne sono due . . . 44
5.1.1 Come si differenziano i due registri . . . 45
5.1.2 Il nome canonico è il nome della cartella . . . 45

5.2 Una strategia . . . 45
5.2.1 Che cosa si crea . . . 46
5.2.2 Il livello di esecuzione . . . 46
5.2.3 Il livello dei metadati . . . 47
5.2.4 I campi opzionali che contano davvero . . . 48
5.2.5 La procedura . . . 49
5.3 Un approccio . . . 49
5.3.1 Il contratto . . . 49
5.3.2 Il descrittore . . . 50
5.3.3 Approcci e overlay: lo stesso registro, due elenchi . . . 50
5.4 Una modalità di pesatura, o un overlay . . . 52
5.4.1 Due forme di cartella, e perché . . . 52
5.4.2 Il registro si costruisce all'import . . . 52
5.4.3 Il campo `META` . . . 53
5.4.4 Che cosa si può importare . . . 54
5.4.5 Le regole dei pesi . . . 54
5.4.6 Le regole degli overlay . . . 54
5.5 Un regime: due file, in quest'ordine . . . 55
5.5.1 La regola . . . 55
5.5.2 Il mercato . . . 56
5.5.3 `META` di soli letterali: qui è un vincolo vero . . . 57
5.5.4 Il merge dei parametri . . . 57
5.6 «L'ho scritto e non compare» . . . 58
5.6.1 Il nodo della ricarica . . . 59
5.6.2 Due package che sono solo rinvii . . . 59
5.6.3 Un modulo morto da non seguire . . . 59
5.6.4 Due incoerenze nelle mappe dell'interfaccia . . . 59
5.6.5 Un elenco obsoleto . . . 60
5.7 Una metrica: i dati del sistema e la pagina dello stream . . . 60
5.7.1 L'oggetto `data` . . . 60
5.7.2 Grafici a più serie . . . 61
5.7.3 La pagina di uno stream . . . 62
5.7.4 La guardia: numpy e pandas filtrati . . . 63

6 L'interfaccia . . . 64
6.1 Anatomia di `dashboard/` . . . 64
6.1.1 Le quattro generazioni di codice, tutte vive . . . 64
6.1.2 I template . . . 65
6.2 Come i dati arrivano alle pagine . . . 65
6.2.1 Che cosa è pigro e che cosa no . . . 66
6.2.2 Gli ensemble preconfigurati non esistono più . . . 66
6.2.3 Come uno stream entra nell'elenco . . . 66
6.3 Il lavoro lungo: job e registro in diretta . . . 67
6.3.1 Gli stati . . . 67
6.3.2 Il registro in diretta . . . 67
6.3.3 Un lavoro per volta . . . 68
6.3.4 Le notifiche . . . 68
6.4 Aggiungere una pagina o una rotta . . . 68
6.4.1 Il contenuto si mostra quando è pronto . . . 69

6.4.2 I fogli di stile stanno tutti nel `<head>` . . . 70
6.4.3 Il modello da copiare . . . 70
6.4.4 Una funzione di servizio che vale la pena copiare . . . 71
6.4.5 Le due pagine speciali . . . 71

7 Account, abbonamenti e marketplace . . . 72
7.1 Il ponte verso il sito . . . 72
7.1.1 Dove sta il token . . . 72
7.1.2 La chiamata che non solleva . . . 72
7.1.3 La regola del token giusto . . . 73
7.1.4 «Non sapere vale come no» . . . 73
7.2 I due strati del permesso di pubblicare . . . 73
7.2.1 Primo strato: il lucchetto . . . 75
7.2.2 Secondo strato: l'abbonamento . . . 75
7.2.3 Gli amministratori . . . 76
7.3 I pagamenti . . . 77
7.3.1 La regola, e perché c'è . . . 77
7.3.2 Quando il webhook non arriva . . . 77
7.3.3 Distinguere «non comprato» da «non ho potuto chiedere» . . . 78
7.3.4 Gli abbonamenti: una rotta sola per due cose . . . 78
7.3.5 Le contestazioni . . . 79
7.3.6 Le variabili d'ambiente . . . 79
7.4 Pubblicare e installare . . . 79
7.4.1 Nessuna moderazione, un filtro all'ingresso . . . 79
7.4.2 I conflitti si dicono prima . . . 79
7.4.3 La pubblicazione automatica . . . 80
7.4.4 L'identità di chi pubblica . . . 80

IV Tenerlo in piedi 81

8 Prove, rilascio e diagnosi . . . 82
8.1 Come sono scritte le prove . . . 82
8.1.1 Due suite girano contro il sito vero . . . 82
8.1.2 Le proprietà che una prova deve avere qui . . . 82
8.1.3 Il resto di `tests/` . . . 83
8.2 Quando qualcosa non va . . . 83
8.2.1 Gli strumenti di diagnosi . . . 84
8.3 Impacchettare e rilasciare . . . 85
8.3.1 Un comando solo . . . 85
8.3.2 Dove finiscono i dati dell'utente . . . 85
8.3.3 Gli aggiornamenti automatici . . . 85
8.4 Il manifesto delle query . . . 85
8.4.1 L'identificativo di una query . . . 86
8.4.2 La legatura all'utente . . . 86

9 Riferimento . . . 88
9.1 Le porte . . . 88
9.2 Le costanti e la configurazione . . . 90
9.2.1 `config/quantsys.yaml` . . . 90

9.2.2 Le costanti del motore . . . 91
9.2.3 Le variabili d'ambiente . . . 91
9.3 Le guide dentro il progetto . . . 91
9.4 Le rotte della dashboard . . . 93
9.4.1 app.py — il nucleo . . . 93
9.4.2 studio.py — blueprint `studio` . . . 96
9.4.3 community.py — blueprint `community` . . . 97
9.4.4 marketplace_routes.py — sul blueprint `community` . . . 98
9.4.5 agent.py — blueprint `agent` . . . 99
9.4.6 genetic.py — blueprint `genetic` . . . 99
9.4.7 auth.py — dentro `init_auth(app)` . . . 99
9.4.8 ui_ext_a.py — estensione v2 . . . 100
9.4.9 ui_ext_b.py — estensione v2 . . . 100
9.4.10 ui_ext_e.py — estensione v2 . . . 100
9.4.11 ui_ext_f.py — estensione v2 . . . 100

Indice analitico . . . 101

Parte I

Orientarsi

1

Che cos'è Lyra

1.1 Il problema che risolve

Lyra è una piattaforma per costruire strategie di investimento sistematiche, provarle sui dati del passato e farle girare su un conto reale. Le tre cose sono una sola: il motivo per cui il sistema è grande è che il *backtest* e l'operatività *live* devono produrre la stessa decisione a partire dagli stessi dati, e tenere allineate due strade che calcolano la stessa cosa costa più codice che scriverne una.

Tre numeri, per dare la scala prima dei dettagli:

Misura	Valore	Dove sta
File, in tutto	806	dei quali 544 Python
Righe Python	119 084	28 494 nella sola `dashboard/`
Righe di sorgente	175 225	Python, JavaScript, CSS, HTML, YAML, Markdown
Rotte HTTP	249	`dashboard/`, undici moduli

Circa il 40 % del sorgente è interfaccia: 28 494 righe di Python più 42 072 fra template e file statici. Non è uno squilibrio. Il valore del prodotto sta nel poter costruire una strategia senza scrivere Python, e quella possibilità costa pagine, editor, validatori e un sistema a plugin.

Che cosa fa, in ordine di uso

1. **Scarica i dati.** Prezzi, fondamentali, tassi, notizie. Finiscono in file `parquet` su disco, in una gerarchia fissa.
2. **Costruisce una strategia.** Scrivendo Python, disegnando un grafo di nodi, o descrivendola a parole a un modello locale.
3. **La prova.** Un *walk-forward* che riaddestra e rivaluta a finestre successive, mai sull'intero campione in una volta.
4. **Combina più strategie.** Ogni strategia produce uno *stream* di profitti e perdite; un *ensemble* li pesa, e come pesarli è esso stesso un plugin — quattordici modalità, dall'equiponderato al meta-allocatore addestrato.
5. **Riduce l'esposizione quando serve.** Gli *overlay* tagliano l'esposizione in regime di stress, e sono anch'essi plugin: sette.
6. **Esegue.** Lo stesso ensemble, lo stesso codice di decisione, collegato a un broker invece che ai dati storici.
7. **Pubblica.** Una strategia può essere messa nel marketplace: qualcun altro la compra e se la installa nel proprio sistema.

Perché il marketplace cambia l'architettura

Finché una strategia è un file sul proprio computer, può essere qualunque cosa. Dal momento in cui può essere *venduta e installata altrove*, diventa un artefatto con un contratto: deve dichiarare che cosa gli serve, deve poter essere verificata senza essere eseguita, e deve poter essere confrontata con ciò che il venditore aveva promesso. Buona parte dei vincoli che nel codice sembrano eccessivi — i descrittori obbligatori, i validatori `AST`, i `META` fatti di soli letterali — esiste per questo, e non per rigore astratto.

Che cosa non fa

Non è un broker, non custodisce denaro e non esegue ordini per conto di nessuno: si collega al conto che l'utente ha già. Non è un fornitore di dati: usa le chiavi API dell'utente verso Polygon, FRED e simili. Non dà consigli d'investimento, e nessun numero prodotto dal sistema è una previsione — un backtest dice come una strategia si *sarebbe* comportata, con i costi stimati, sui dati che esistevano.

1.2 Le due metà

La prima cosa da sapere di Lyra è che non è un'applicazione: sono due, che girano in posti diversi e si parlano attraverso tre soli canali.

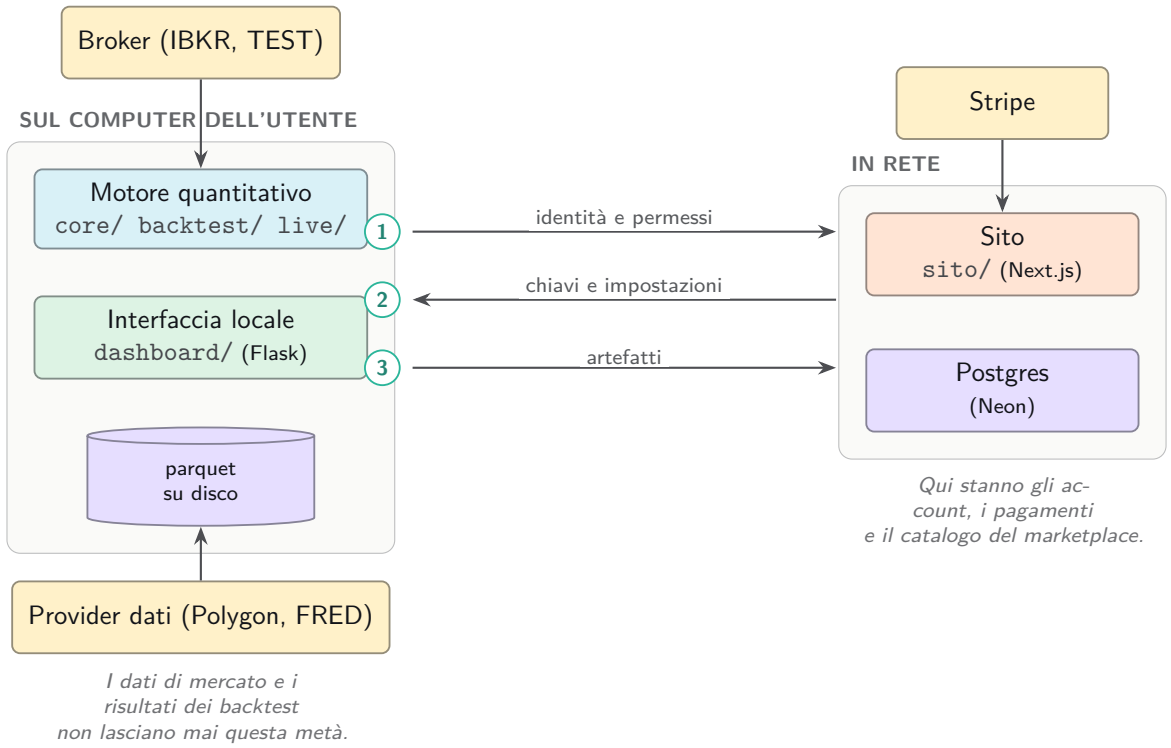


Figura 1.1. Le due metà del sistema. Tutto ciò che non attraversa uno dei tre canali numerati resta dalla parte in cui è nato.

1.2.1 La metà che gira sul computer dell'utente

È il progetto Python: `core/`, `strategies/`, `backtest/`, `live/`, `reporting/`, e l'interfaccia Flask in `dashboard/`. Gira in locale, tipicamente su http://127.0.0.1:5000, e si impacchetta come applicazione desktop (`desktop_app.py` apre la stessa dashboard in una finestra nativa invece

che in una scheda del browser).

Qui stanno i dati di mercato e i risultati dei backtest, e qui restano. Non c'è nessun percorso per cui un prezzo scaricato o un'equity line calcolata finisca in rete.

1.2.2 La metà che gira in rete

È il sito, un'applicazione Next.js con un database Postgres. Tiene gli account, le password, gli abbonamenti, i pagamenti e il catalogo del marketplace. Non ha accesso ai dati di mercato e non esegue calcoli quantitativi.

1.2.3 I tre canali, e perché sono tre

1 — identità e permessi
La dashboard chiede al sito «chi è questo utente, e che cosa può fare». La risposta è un *token* personale con dentro il livello di abbonamento e il flag di amministratore. Senza di esso l'applicazione funziona lo stesso: non si entra nel marketplace, e basta.

2 — chiavi e impostazioni
Le variabili d'ambiente — chiavi Polygon, FRED, Stripe, SMTP — sono gestite dal sito, non da un `.env` locale. È il verso opposto al canale 1, ed è l'unico che va da destra a sinistra.

3 — artefatti
Una strategia venduta sul marketplace viaggia come pacchetto di file. Va su quando si pubblica, viene giù quando si installa.

> **Dove cercare un bug che «sparisce»**
>
> Quando un comportamento si vede in un ambiente e non nell'altro, la domanda da farsi è quale dei tre canali è in mezzo. Un marketplace vuoto in locale e pieno sul sito è il canale 1 (token mancante o scaduto); un download dati che fallisce solo su una macchina è il canale 2 (chiave non arrivata); un artefatto installato che non compare è il canale 3 più una delle trappole del sistema a plugin (capitolo 5).

1.3 Le due applicazioni dentro la metà locale

Dentro la metà locale ci sono, a loro volta, due applicazioni web: la dashboard Flask storica e un'API FastAPI più recente, avviate separatamente e in ascolto su porte diverse. Non è un residuo da ripulire: è una migrazione deliberata, il cosiddetto *strangler fig*, e il capitolo 3 la spiega per esteso.

1.4 Dieci parole da fissare subito

Nel codice e nell'interfaccia queste parole hanno un significato preciso, e alcune significano qualcosa di diverso da ciò che si direbbe leggendole. Vale la pena leggerle una volta: metà delle domande su Lyra sono domande di vocabolario travestite da domande di architettura.

Strategia (`strategies/`)
Un oggetto che, a ogni barra, guarda gli indicatori e il portafoglio e restituisce una lista di *segnali*. Non decide quanto comprare: decide *che cosa* e in che direzione. Vive dentro il motore e passa per l'allocatore, il rischio e l'esecuzione.

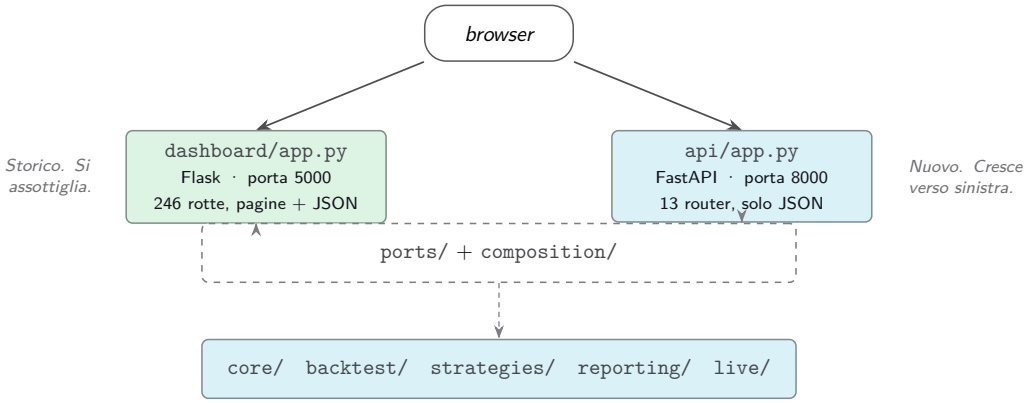


Figura 1.2. Le due applicazioni web della metà locale. Entrambe passano dalle stesse porte e usano lo stesso motore: ciò che cambia è il guscio.

Approccio (`approaches/`)
Un oggetto che *salta* tutto il motore e produce direttamente una serie di rendimenti giornalieri. Serve a chi ha già una pipeline propria — un modello ML con il suo feature store, per dire — e vuole solo che il risultato entri nel sistema come uno stream qualsiasi. Una strategia e un approccio non sono due nomi per la stessa cosa.

Stream
Una serie storica di profitti e perdite. È l'unità di misura comune: una strategia, un approccio e un overlay producono tutti uno stream, ed è su questo che l'ensemble lavora. Tre formati: `equity_curve`, `pnl_daily`, `pnl_sparse`.

Ensemble
La combinazione pesata di più stream. Il peso di ciascuno è deciso da una *modalità di pesatura*.

Modalità di pesatura (`weights/`)
Il criterio con cui l'ensemble distribuisce il capitale fra gli stream. Quattordici installate, dall'equiponderato alla minima varianza con restringimento di Ledoit–Wolf al meta-allocatore addestrato sui regimi. È un plugin: si aggiunge senza toccare il motore.

Overlay (`overlays/`)
Un moltiplicatore di esposizione fra 0 e 1, applicato all'ensemble già composto. `1.0` significa esposizione piena, `0.0` fuori dal mercato. Non produce rendimenti: li scala. Gli overlay attivi si moltiplicano fra loro, e il risultato non dipende dall'ordine — proprietà voluta, non accidente.

Regime (`regime/`)
Uno stato di mercato fra tre: `bull`, `neutral`, `bear`. Si costruisce con due artefatti distinti: una *regola* (dalla serie al regime) e un *mercato* (quali simboli usano quale regola, con quali parametri). Le etichette sono sempre quelle tre, perché le strategie dichiarano i propri `regime_weights` su quei nomi.

Walk-forward
Il modo in cui si valuta una strategia: si addestra su una finestra, si valuta sulla successiva, si avanza. Mai una valutazione sull'intero campione, che misurerebbe quanto bene il modello ricorda invece di quanto bene prevede.

Artefatto
Un plugin impacchettato per essere pubblicato: strategia, approccio, overlay, modalità di pesatura, metrica, indicatore. È l'unità che il marketplace vende e installa.

Porta (`ports/`)
Un contratto Python (`Protocol`) fra il centro del sistema e il mondo: un repository, una sorgente dati, un broker, un orologio. Trentadue, e i casi d'uso conoscono solo queste.

Due parole che tradiscono

Overlay in `overlays/` è un moltiplicatore di esposizione; ma esistono anche approcci con `stream_category = "ml_overlay"`, che *non* sono overlay: sono approcci, con un contratto diverso. Il capitolo 5 tiene i due casi separati con cura, perché confonderli produce un artefatto che si installa e non compare mai.
Project nell'interfaccia non è il progetto software: è una composizione salvata di stream, pesi e overlay. Nella *User guide* è un capitolo a sé.

1.5 Da dove cominciare, secondo cosa si deve fare

Questa guida non va letta tutta. Quattro percorsi, a seconda del motivo per cui si è qui.

Si deve...	Leggere
far partire una copia e guardarsi intorno	capitolo 2. Un'ora, e si ha il sistema che gira.
aggiungere una strategia, un overlay, una pesatura	capitolo 4 per capire che cosa decide che cosa, poi il capitolo 5, che è una procedura da seguire con il codice aperto.
toccare l'interfaccia	capitolo 6. Prima però § 3.2: c'è un motivo se ci sono due applicazioni web e non una.
mettere mano a pagamenti, abbonamenti, marketplace	capitolo 7, e la parte sui permessi nel capitolo 3. È l'area dove un errore costa denaro vero.

Il minimo indispensabile

Chi ha fretta e deve comunque toccare il codice legga due sole cose: la separazione fra decisione ed esecuzione (§ 4.2) e le due forme del sistema a plugin (§ 5.1). Sono i due punti in cui un errore non dà errore: produce un numero sbagliato, o un artefatto che non compare.

1.6 Che cosa questa guida non copre

- **Il contenuto finanziario delle strategie.** Perché il momentum 12-1 funzioni, se il PEAD sia ancora sfruttabile, come si tari un *vol target*: sono domande di merito, e questo è un documento di ingegneria. Dove una scelta numerica è codificata nel sistema, la guida dice dov'è e che cosa cambia, non se sia giusta.
- **L'uso dell'applicazione.** Dove si clicca per lanciare un backtest sta nella *User guide*, il manuale utente in inglese, documento separato nella stessa cartella.
- **Il codice del sito, riga per riga.** Il sito è documentato qui per ciò che espone alla metà locale: endpoint, formato dei token, regole dei permessi. Il resto è un'applicazione

Next.js ordinaria.

2

Mettere in moto una copia

2.1 Che cosa serve avere installato

Per la metà locale: Python, e nient'altro di obbligatorio. Per il sito: Node.js e un Postgres raggiungibile. Le due metà si avviano in modo indipendente, e per lavorare sul motore il sito non serve — senza di esso si perde solo il marketplace.

Componente	Versione	Note
Python	3.10 o più	Il codice usa `match`, `X | None` e le `dataclass` con `slots`.
pandas	< 3	Vincolo reale: il codice precede l'API di pandas 3.
numpy	< 2	Stesso motivo. In caso di errori nello stack numerico, questi due vincoli sono la prima cosa da provare.
Node.js	18 o più	Solo per il sito.
Postgres	14 o più	Solo per il sito. In sviluppo va bene anche un'istanza gestita (Neon).

2.1.1 Installazione delle dipendenze Python

Dalla radice del progetto

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Il file `requirements.txt` è diviso in blocchi commentati e dichiara di che cosa è responsabile ogni dipendenza. Tre di essi vanno guardati:

Stack numerico
: `numpy`, `pandas`, `scipy`, `scikit-learn`, `statsmodels`, `pyarrow`. Senza `pyarrow` non si leggono i `parquet`, cioè non si legge niente.

Machine learning
: `lightgbm`, `hmmlearn`, `shap`. Servono agli approcci ML e alle modalità di pesatura `regime_ml` e `regime_style`. Un sistema senza di esse parte, e quelle voci semplicemente non funzionano.

Sorgenti dati
: `yfinance`, `requests`, `fredapi`, `python-dotenv`.

Il modello locale è opzionale

Lo Studio può generare il codice di un artefatto a partire da una descrizione in italiano. Il modello gira in locale via Ollama, che non è nelle dipendenze Python e va installato a parte. Senza, lo Studio funziona ugualmente: si perde la generazione da testo, restano l'editor e il grafo a nodi.

2.1.2 Con Docker, invece

Nella radice ci sono `Dockerfile` e `docker-compose.yml`. Avviano la sola dashboard, con i dati montati come volume.

Dalla radice del progetto

```
UID=$(id -u) GID=$(id -g) docker compose up --build
# poi: http://localhost:5000
```

Su Linux servono UID e GID

Senza passarli, i file che il contenitore scrive nei volumi — i `parquet` scaricati, i risultati dei backtest — risultano di un altro proprietario e dall'esterno non si riescono più a modificare. Su macOS e Windows il problema non si pone.

2.2 I quattro modi di avviarla

Nella radice del progetto ci sono quattro punti d'ingresso, e fanno cose diverse. Si lanciano tutti *dalla radice*, mai da dentro una sottocartella: gli import sono assoluti rispetto a quella.

Comando	Porta	Che cosa avvia
`pythonscripts/run_dashboard.py`	5 000	L'interfaccia completa. È questo che si usa nel 99 % dei casi.
`pythonrun_api.py`	8 000	Solo l'API FastAPI del motore. Nessuna pagina, solo JSON.
`pythondesktop_app.py`	5 000	La stessa dashboard, in una finestra nativa invece che in una scheda del browser. Richiede `pywebview`.
`npmrundev` (in `sito/`)	3 000	Il sito. Indipendente dai tre precedenti.

2.2.1 La dashboard

Avvio normale

```
python scripts/run_dashboard.py
# http://127.0.0.1:5000
```

Host e porta si cambiano con le variabili d'ambiente `FLASK_HOST` e `FLASK_PORT`, oppure nei campi corrispondenti della configurazione. Il default è `127.0.0.1`, cioè raggiungibile solo dalla macchina stessa, e questo non è un dettaglio di comodità.

La dashboard non va esposta in rete

Quasi nessuna rotta della dashboard richiede un login, ed è una scelta deliberata: l'applicazione gira sul computer di chi la usa, e chiedere un account per entrare in casa propria non avrebbe senso. Il prezzo è che chi raggiunge la porta 5 000 può fare tutto: lanciare backtest, leggere le credenziali del broker, scrivere file. Mettere `FLASK_HOST=0.0.0.0` su una macchina raggiungibile da fuori significa consegnarla. Le uniche aree protette sono quelle che toccano il sito — marketplace, profilo, abbonamento, amministrazione — e il §3.4 spiega esattamente quali e perché.

2.2.2 L'API del motore

Con ricaricamento automatico

```
uvicorn api.app:app --reload
# http://127.0.0.1:8000/docs  -> la documentazione interattiva generata
```

Gira *accanto* alla dashboard, non al suo posto. Per lavorare sull'interfaccia non serve; serve quando si tocca un caso d'uso, perché l'API lo espone senza passare da Flask ed è il modo più rapido per provarlo.

2.2.3 Il sito

Dalla cartella del sito

```
cd sito
npm install
npm run dev
# http://localhost:3000
```

Il sito ha bisogno di un Postgres e delle proprie variabili d'ambiente (`DATABASE_URL`, `NEXTAUTH_SECRET`, le chiavi Stripe e SMTP). In produzione queste non stanno in un file: si gestiscono dal pannello del sito, ed è lo stesso meccanismo che poi le passa alla metà locale attraverso il canale 2 (§1.2).

Il giro minimo per vedere qualcosa

1. `pipinstall-rrequirements.txt`
2. `pythonscripts/download_d1.py` — scarica un po' di dati giornalieri (§2.3).
3. `pythonscripts/run_dashboard.py`
4. Nel browser: **Run** → una strategia → **Run backtest**. Alla fine compare uno stream in **Streams**.

Senza il passo 2 l'applicazione parte e le pagine sono vuote: non è un errore, è che non c'è niente da mostrare.

2.3 Dove stanno i dati

Tutti i percorsi del sistema sono definiti in un file solo, `core/paths.py`, e nessun altro modulo costruisce percorsi a mano. Chi deve aggiungere una cartella la aggiunge lì.

2.3.1 La cartella di lavoro

La radice non è necessariamente quella del repository. `PROJECT_ROOT` si decide a ogni avvio, in questo ordine:

1. `LYRA_HOME` impostata nell'ambiente → si usa quella. È il modo per tenere i dati su un disco diverso, o condividerli fra più installazioni.
2. Applicazione impacchettata (PyInstaller, Electron) → la cartella utente del sistema operativo: `%LOCALAPPDATA%/Lyra` su Windows, `~/Library/ApplicationSupport/Lyra` su macOS, `~/.local/share/Lyra` su Linux.
3. Altrimenti → la radice del repository.

> **Perché i dati escono dal bundle**
>
> In un'applicazione impacchettata il codice vive in una cartella di sola lettura che ogni aggiornamento sovrascrive. Se i `parquet` e i risultati dei backtest stessero lì dentro, un aggiornamento cancellerebbe mesi di lavoro. `CODE_ROOT` — dove sta il codice — e `PROJECT_ROOT` — dove si scrive — sono due cose distinte proprio per questo, e vanno tenute distinte anche quando coincidono, come in sviluppo.

2.3.2 Che cosa c'è sotto `data/`

Cartella	Contenuto
`data/assets/<TF>/`	OHLCV, un `parquet` per simbolo, divisi per timeframe: `D1/`, `H4/`, `H1/`, `M30/`, `M15/`, `M5/`. Lo stesso simbolo può esistere a più timeframe, ciascuno con il proprio intervallo di date.
`data/driver/`	Le serie che guidano i regimi (SPY, DXY, VIX…). Stanno a parte perché possono avere un timeframe diverso dagli asset.
`data/rates/`	Tassi di politica monetaria per valuta, da FRED.
`data/news/`	Archivio delle notizie.
`data/earnings/`	Date e sorprese degli utili, per simbolo.
`data/fundamentals/`	Bilanci, per simbolo.
`data/mms/`	Feature della heatmap di liquidità, per (simbolo, timeframe).
`data/_universe.json`	Che cosa il sistema sa di avere. Non è un indice generato: è il registro, e una cartella piena di `parquet` non censiti qui per il sistema è vuota.

I risultati stanno altrove, sotto `output/`: `output/strategies/` per gli stream (è da qui che la dashboard li legge, seguendo il `output_path_pattern` dichiarato da ciascun artefatto), `output/ml/` per feature e predizioni, `output/reports/` per i report aggregati.

> **I percorsi legacy sono ancora letti**
>
> `tests/outputs/`, `data/feature_store/`, `data/ml_outputs/` sono posizioni storiche. L'helper `discover_parquet()` cerca prima nei percorsi nuovi e poi in questi, così una installazione vecchia continua a funzionare. Scrivere lì non è più previsto.

2.3.3 Portare dentro i primi dati

Gli script di scarico stanno in `scripts/` e sono uno per sorgente:

I quattro che servono per cominciare `scripts/`

```
python scripts/download_d1.py              # azioni ed ETF, giornaliero
python scripts/download_regime_drivers.py  # le serie dichiarate dai mercati di regime
python scripts/download_fred_rates.py      # tassi (serve FRED_API_KEY)
python scripts/download_polygon_m5.py      # intraday (serve POLYGON_API_KEY)
```

Lo stesso si fa dall'interfaccia, in **Data**, che chiama le rotte `/api/data/download/<modalità>` e mostra l'avanzamento. La differenza è che dall'interfaccia il lavoro parte come *job* con un registro consultabile, dalla riga di comando no.

`download_regime_drivers.py` legge i file di mercato

Non ha un elenco di simboli suo: scarica i `driver` dichiarati nei file sotto `regime/markets/`. Se si aggiunge un mercato che guarda una serie nuova, questo script è il modo di procurarsela — e finché non lo si lancia, quel mercato non modula niente e lo dice solo in un avviso nel registro.

2.4 La mappa delle cartelle

Trentuno cartelle nella radice. Raggruppate per mestiere diventano sei famiglie, e la famiglia dice già quasi tutto su che cosa ci si può aspettare di trovarci dentro.

2.4.1 Il guscio a strati

Sono le cartelle introdotte con la migrazione all'architettura esagonale (§ 3.1). Sono piccole: la loro dimensione non misura quanto lavoro fanno, ma quanto poco codice serve a tenere separato il resto.

Cartella	Righe	Che cosa contiene
`domain/`	478	I tipi puri: `dataclass` congelate, nessuna dipendenza, nemmeno da pandas.
`ports/`	239	I trentadue contratti. È il file da leggere per primo di tutto il progetto.
`application/`	579	I casi d'uso. Conoscono solo le porte.
`adapters/`	2 593	Le implementazioni concrete, e i validatori `AST` degli artefatti.
`composition/`	109	Centonove righe, ed è l'unico posto in tutto il sistema dove si sceglie un concreto.
`api/`	548	L'applicazione FastAPI e i suoi tredici router.

2.4.2 Il motore

Cartella	Righe	Che cosa contiene
`core/`	7 587	Tipi del dominio quantitativo (`Signal`, `Portfolio`, `Fill`), percorsi, manutenzione.
`backtest/`	1 567	Due file. Uno è il motore di simulazione, ed è il punto in cui si decide che cosa è lecito guardare.
`execution/`	2 428	I runner: walk-forward, backtest singolo, approcci, gestione dei job.
`data/`	6 251	Caricamento, universo, categorie, qualità, sorgenti esterne.
`reporting/`	2 096	`EnsembleEngine`: la composizione degli stream, i pesi, gli overlay.
`live/`	4 170	Lo stesso calcolo, collegato a un broker.

2.4.3 Gli artefatti

Sono le cinque famiglie di plugin, più tre cartelle che ospitano artefatti minori. Ogni sottocartella è un artefatto, e il nome della cartella è il nome canonico — non quello dichiarato dentro il file.

Cartella	Installati	Forma del contratto
`strategies/`	9 attive, 20 spente	`DESCRIPTOR` in `strategy.py`
`approaches/`	3 attivi, 4 dormienti	`APPROACH_DESCRIPTOR` in `approach.py`
`weights/`	14	`META` + `compute` in `weight_mode.py`
`overlays/`	7	`META` + `compute` in `overlay.py`
`regime/`	3 regole, 3 mercati	`META` + `compute`, in due cartelle distinte
`indicators/`, `metrics/`	—	Artefatti minori, stesso schema

Accendere e spegnere un artefatto

Il prefisso `_` davanti al nome della cartella è l'interruttore: `strategies/_value/` non viene caricata, `strategies/value/` sì. Rinominare la cartella è l'unico gesto necessario — non c'è nessun elenco da aggiornare da nessuna parte. Delle 29 cartelle sotto `strategies/`, 20 cominciano con il trattino basso e restano fuori.

2.4.4 L'interfaccia, e il resto

`dashboard/` è l'applicazione Flask: 28 494 righe di Python in 36 file, più 42 072 righe fra template Jinja, JavaScript e CSS. È l'oggetto più grande del progetto e ha un capitolo suo (capitolo 6).

Restano: `features/`, `mms/` e `volume_profile/` (calcoli specialistici che le strategie possono richiedere dichiarandoli nel descrittore), `genetic/` (la generazione di strategie per program-

mazione genetica), `validation/` (controlli statistici sui risultati), `scripts/` (gli scarichi e le utilità da riga di comando), `tests/`, `config/` (il solo `quantsys.yaml`) e `build/` (impacchettamento).

> **`brokers/` è morta**
>
> Nella radice c'è una cartella `brokers/` di 402 righe che duplica codice vivo in `live/`. Nessun modulo la importa. È un residuo: non va estesa, e ciò che sembra doverci andare va in `live/broker.py`.

Parte II

Come è fatto

3

Come è fatto

3.1 L'architettura esagonale, in concreto

Il guscio a strati occupa 4 546 righe su 119 084: meno del 4 % del Python del progetto. Non ha riscritto niente. Avvolge il codice che c'era — `strategies/`, `reporting/`, `execution/`, `live/`, `data/` — e gli mette davanti un confine.

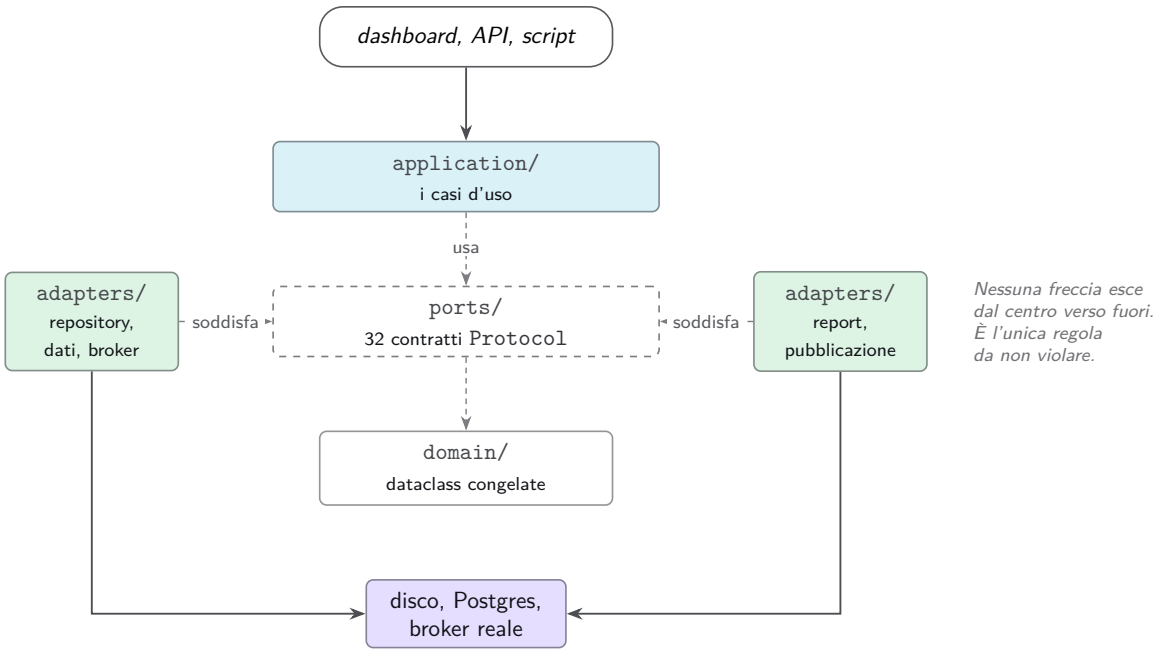


Figura 3.1. I cinque strati. L'unica regola è la direzione delle frecce.

3.1.1 La regola, e perché è una sola

Si dipende solo verso l'interno. `application/` non conosce nessun concreto: conosce solo le porte. I concreti si scelgono in un file solo, `composition/container.py`.

Perché il confine vale la pena

L'ecosistema che Lyra vuole essere — applicazione scaricabile, sito community, marketplace — è fatto di *tre client dello stesso motore*. Finché il motore è una libreria che ciascun client importa a modo suo, ogni client ne vede una parte diversa e ogni cambiamento ne rompe due. Rendendolo un servizio con un contratto stabile, i client tornano a essere consumatori, e l'inversione delle dipendenze non è più una buona pratica da ricordarsi: è imposta dal confine di processo.

3.1.2 `domain/`: i tipi, e basta

Quattrocentosettantotto righe di `dataclass` congelate. Nessuna dipendenza — nemmeno pandas. `StrategyInfo`, `StreamInfo`, `RunReport`, `OrderRequest`, `SecurityReport`, `LiveRunnerConfig` e una trentina d'altri.

Sono il vocabolario che attraversa gli strati. Un caso d'uso restituisce un `StrategyInfo`, non un `DataFrame` e non un `StrategyDescriptor` — che è un oggetto del motore, con dentro una `factory` richiamabile.

3.1.3 `ports/`: trentadue contratti

Il file da leggere per primo in tutto il progetto, e sono 239 righe. Ogni porta è un `Protocol` marcato `@runtime_checkable`.

Due porte intere, non un estratto — `ports/__init__.py`

```python
@runtime_checkable
class StrategyRepository(Protocol):
    """Sorgente delle strategie disponibili (mappate a StrategyInfo)."""
    def list(self) -> list[StrategyInfo]: ...
    def get(self, name: str) -> Optional[StrategyInfo]: ...


@runtime_checkable
class WalkForwardRunner(Protocol):
    """Esegue un walk-forward per una strategia. Ritorna l'exit code."""
    def run(self, strategy_name: str) -> int: ...
```

Sono `Protocol`, non classi base astratte, e la differenza conta:

Tipizzazione strutturale
Un concreto soddisfa la porta *senza ereditarla*. Non deve importare `ports/`, non deve dichiarare niente: le basta avere i metodi giusti. L'accoppiamento è il minimo possibile, e si può avvolgere codice preesistente senza toccarlo.

Porte piccole e separate
`DataSource` ≠ `Broker` ≠ `StrategyRepository` ≠ `Clock`. Un adattatore implementa quello che gli serve. È l'*Interface Segregation* preso sul serio: una porta «`Everything`» avrebbe costretto ogni finto a implementare trenta metodi per provarne uno.

Sostituibilità verificata
Nei test i casi d'uso girano con repository e runner finti. Non è una proprietà sperata: è il modo in cui le prove sono scritte.

> **La porta `Broker` è ri-esportata, non riscritta**
>
> `live.broker.LiveBroker` era già una classe base astratta, cioè già una porta, scritta prima del guscio. `ports/` la ri-esporta invece di duplicarla:

ports/__init__.py

```python
try:
    from live.broker import LiveBroker as Broker  # noqa: F401
except Exception:  # pragma: no cover
    Broker = object  # type: ignore
```

È la mossa corretta: due definizioni della stessa interfaccia divergono, e quando divergono nessuno se ne accorge finché un broker non si comporta in un modo in un posto e in un altro altrove.

3.1.4 `application/`: i casi d'uso

Ogni caso d'uso è una classe con una responsabilità, riceve le porte dal costruttore e non conosce nessun concreto. Sono brevi — spesso tre righe — e devono restarlo: il caso d'uso orchestra, non calcola.

Tre casi d'uso, per intero — application/__init__.py

```python
class ListStrategies:
    def __init__(self, repo: StrategyRepository) -> None:
        self._repo = repo

    def execute(self) -> list[StrategyInfo]:
        return self._repo.list()


class RunWalkForward:
    """Esegue un walk-forward e (se c'e' un sink) ne registra il report."""
    def __init__(self, runner: WalkForwardRunner,
                 report_sink: Optional[ReportSink] = None) -> None:
        self._runner = runner
        self._sink = report_sink
```

Il calcolo non va qui

La tentazione, aggiungendo una funzionalità, è di metterne «un pezzetto» nel caso d'uso: un filtro, una media, una conversione. Il giorno in cui succede, quel pezzetto esiste solo nel percorso che passa per l'API e non in quello che passa per la dashboard, e i due danno numeri diversi. Il calcolo sta nel motore; il caso d'uso lo chiama e basta.

3.1.5 `composition/`: centonove righe, e sono quelle decisive

`composition/container.py` è un `dataclass` con venti campi — un caso d'uso ciascuno — e una funzione che lo costruisce.

L'unico punto di scelta dei concreti — composition/container.py

```python
    # --- unico punto di scelta dei CONCRETI ---
    repo = RegistryStrategyRepository(Path(strategies_dir) if strategies_dir else None)
    weighting = EnsembleWeightingModeProvider()
    runner = InProcessWalkForwardRunner()
```

```
    registry = FileModelRegistryAdapter(model_dir)
    sink = FileReportSink(report_dir)
    ...
    return Container(
        list_strategies=ListStrategies(repo),
        get_strategy=GetStrategy(repo),
        run_walkforward=RunWalkForward(runner, sink),
        ...
    )
```

Per cambiare sorgente dati, broker o registro dei modelli si tocca questo file, e nessun altro. Per far girare i casi d'uso nei test con dei finti si costruisce un `Container` diverso, senza toccare né i casi d'uso né le porte.

Come si aggiunge una capacità, nell'ordine giusto

1. Un tipo in `domain/`, se serve un dato nuovo che attraversa il confine.
2. Una porta in `ports/`: che cosa deve saper fare il mondo.
3. Un caso d'uso in `application/`: tre righe, nessun calcolo.
4. Un adattatore in `adapters/` che avvolge il codice del motore che fa davvero la cosa.
5. Una riga in `composition/container.py` che li collega.
6. Un router in `api/` che lo espone, se serve via HTTP.

Saltare il passo 2 è l'errore tipico: si collega il caso d'uso direttamente all'adattatore, «tanto è uno solo», e da quel momento il caso d'uso non è più verificabile senza l'infrastruttura.

3.2 Due applicazioni web, e il motivo

Dentro la metà locale ci sono due applicazioni web (figura 1.2):

	Flask	**FastAPI**
File principale	`dashboard/app.py`	`api/app.py`
Porta	5 000	8 000
Dimensione	249 rotte	13 router
Restituisce	pagine HTML e JSON	solo JSON
Autenticazione	guardia globale, quasi tutto aperto	nessuna
Parla con	direttamente il motore	i casi d'uso, via `Container`

3.2.1 Che cos'è uno strangler fig

La pianta da cui prende il nome cresce attorno a un albero ospite, lo avvolge e lo sostituisce gradualmente: alla fine resta in piedi da sola, e l'albero di dentro non c'è più. Nessun momento in cui il bosco resta senza albero.

Applicato qui: la dashboard Flask ha dentro il codice che è stato scritto per primo, con il motore chiamato direttamente. L'API FastAPI è il nuovo guscio e parla solo con i casi d'uso. Le due convivono, e il confine si sposta una funzionalità alla volta.

Perché non si riscrive la dashboard e basta

Perché la dashboard funziona, e perché 28 494 righe di Python più 42 072 di template non si riscrivono con la garanzia di non perdere niente. Una riscrittura in blocco ha due esiti noti: o dura abbastanza da essere abbandonata a metà, o arriva in fondo avendo perso silenziosamente i casi limite che nessuno aveva scritto da nessuna parte — e in un sistema che muove denaro, un caso limite perso è un ordine sbagliato.

3.2.2 Come si avvia l'API

Il montaggio dell'applicazione `api/app.py`

```python
def create_app(container: Optional[Container] = None, jobs=None) -> FastAPI:
    container = container or build_container()
    app = FastAPI(title="QuantSys Engine API", version="0.2.0")
    app.state.container = container

    if jobs is None:
        # JobManager reale: max_workers=1 = serializzato (default sicuro del progetto)
        from execution.job_manager import JobManager
        job_manager = JobManager(max_workers=1)
        app.state.job_manager = job_manager
        jobs = JobManagerJobService(container.run_walkforward, job_manager)
    app.state.jobs = jobs

    app.include_router(health.router)
    app.include_router(strategies.router)
    ...
```

Due cose meritano attenzione. La prima è che `create_app` accetta un `Container` dall'esterno: nei test si passa un contenitore di finti e l'API intera gira senza toccare il disco. La seconda è `max_workers=1`: i job sono *serializzati*, ed è il default sicuro del progetto. Due backtest in parallelo si contendono la stessa cache dati e gli stessi file di output.

3.2.3 I tredici router

Router	Che cosa espone
`health`	Stato del servizio.
`strategies`	Elenco e dettaglio delle strategie.
`strategy_admin`	Validazione e installazione di una strategia.
`runs`	Avvio di un walk-forward, stato, log in SSE.
`streams`	Catalogo degli stream prodotti.
`ensemble`	Composizione: pesi, overlay, metriche.
`models`	Registro dei modelli ML e promozione.
`factor_attribution`	Scomposizione del rendimento per fattore.
`stress_test`	Prove sotto scenario.
`live`	Controllo dell'operatività.
`generator`	Generazione di codice da descrizione.
`nodes`	Compilazione del grafo a nodi in Python.
`fidelity`	Confronto fra trade disegnati a mano e strategia.

Le rotte FastAPI non esistono per Flask

Sono due applicazioni separate, in due processi. Nessuna rotta di `api/` è registrata nell'app Flask, e viceversa. Cercare `/api/strategies` sulla porta 5 000 e trovarci una rotta Flask con lo stesso nome ma comportamento diverso è possibile, ed è successo: quando si documenta o si collauda un endpoint, la porta fa parte dell'indirizzo.

3.2.4 Dove passa il confine, oggi

La regola pratica: **ciò che è nuovo nasce nel guscio**. Una funzionalità nuova si scrive come caso d'uso, si espone dall'API, e la dashboard la chiama come chiamerebbe qualunque servizio. Ciò che esiste si sposta quando lo si deve toccare comunque, non prima e non per ordine — spostare codice che funziona e che nessuno sta modificando è rischio senza ritorno.

3.3 Le 249 rotte, e chi le registra

La dashboard Flask ha 270 decoratori `@route` che danno 249 percorsi distinti — la differenza sono le coppie «`GET` in una funzione, `POST` in un'altra». Sono sparsi su undici moduli, montati in un ordine che conta.

Modulo	Rotte	Come si registra
`app.py`	137	`@app.route` dentro `create_app()`
`community.py`	43	`Blueprint("community")`, senza `url_prefix`
`studio.py`	43	`Blueprint("studio")`
`marketplace_routes.py`	12	Agganciate *al blueprint community*, non a uno proprio
`agent.py`	10	`Blueprint("agent")`
`genetic.py`	8	`Blueprint("genetic")`
`auth.py`	8	`@app.route` dentro `init_auth(app)`
`ui_ext_a/b/e/f.py`	9	Caricate automaticamente (§ 3.3.2)

Due moduli senza rotte

`live_runners.py` e `autopubblica.py` non registrano niente: sono moduli di servizio. Il primo è chiamato da `app.py` sotto `/api/runners/*` e `/api/brokers/*`; il secondo parte come thread all'avvio.

3.3.1 L'ordine di montaggio

3.3.2 Il caricamento automatico delle estensioni v2

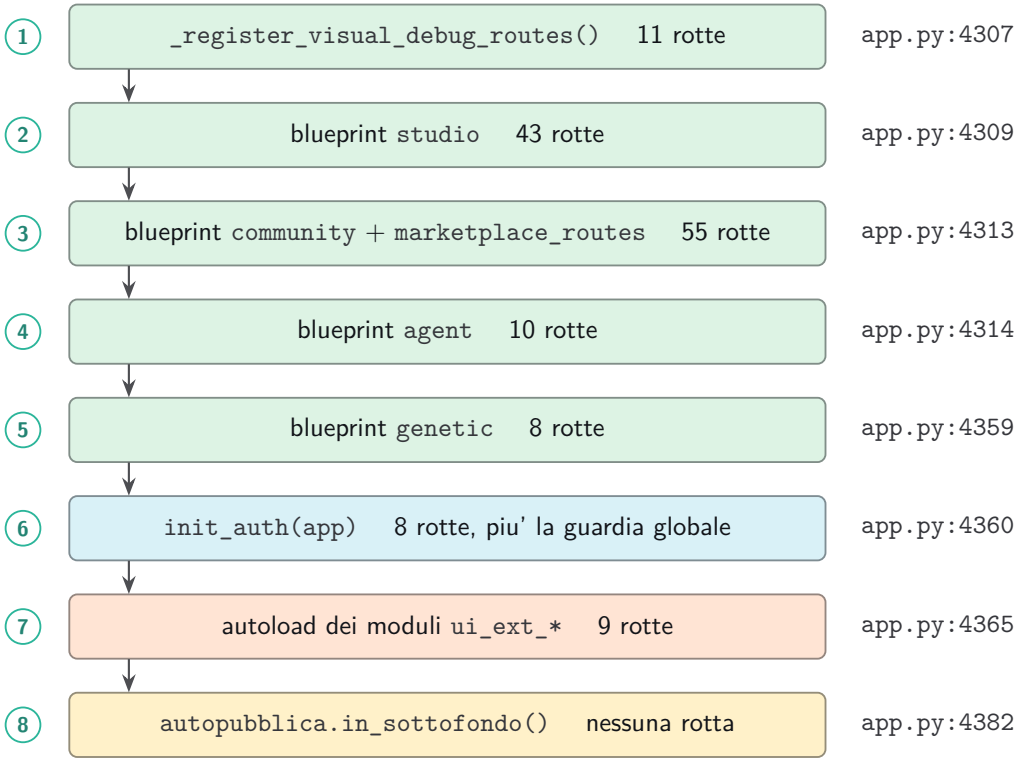
L'ultimo passo di registrazione è diverso dagli altri: non c'è nessun elenco.

Il meccanismo delle estensioni `dashboard/app.py:4362`

```
    # Rotte aggiunte con il nuovo stile (v2): ogni modulo dashboard/ui_ext_*.py
    # espone register(app, data_layer, base_path). Tenerle in moduli propri
    # evita di far crescere ancora questo file.
    try:
        import importlib as _il, pkgutil as _pu, dashboard as _dpkg
        for _m in sorted(m.name for m in _pu.iter_modules(_dpkg.__path__)
                         if m.name.startswith("ui_ext_")):
            _il.import_module(f"dashboard.{_m}").register(app, data_layer, base_path)
    except Exception as _e:  # noqa: BLE001
        logging.getLogger("app").warning("v2 page extensions not loaded: %s", _e)
```

Cinque conseguenze, tutte rilevanti per chi estende:

1. **Nessun registro da aggiornare.** Si crea `dashboard/ui_ext_<lettera>.py` con una funzione `register(app, data_layer, base_path)` e al riavvio c'è.
2. **Ordine alfabetico garantito** dal `sorted`: `a`, `b`, `e`, `f`. Le lettere `c` e `d` non esistono — un buco nella serie, non un errore.
3. **Rotte sull'app, non su un blueprint.** Gli endpoint sono globali (`a_overview`, `b_project_detail`…) e `url_for` li raggiunge senza prefisso.
4. **Caricate dopo tutto il resto**, compreso `init_auth`: gli hook di queste estensioni girano



La guardia di `auth` *vale per* tutte *le rotte, anche per quelle registrate prima di lei: un* `before_request` *non guarda l'ordine. Ciò che l'ordine decide è la sequenza fra gli hook — quello di* `ui_ext_b` *gira dopo* `_require_login`*, e non prima.*

Figura 3.2. Gli otto passi con cui `create_app()` assembla l'applicazione.

dopo la guardia di autenticazione.

5. **Fallimento silenzioso.** Un solo `except` avvolge l'intero ciclo: se `ui_ext_a` solleva all'import, *anche b, e, f non vengono caricati*, e tutto ciò che si vede è una riga di avviso nel registro.

Il punto più fragile della dashboard

Il punto 5 è il modo tipico in cui metà dell'interfaccia sparisce dopo una modifica apparentemente innocua. Il sintomo è una pagina che va in `404` o un pannello vuoto; la causa è un `ImportError` in un altro modulo. Quando qualcosa della v2 non c'è più, la prima cosa da cercare nel registro è la riga `v2 page extensions not loaded`.

3.3.3 Rotte statiche che coprono rotte dinamiche

Werkzeug dà la precedenza ai segmenti statici, quindi `/api/marketplace/items` vince su `/api/marketplace/<artifact_id>`. Funziona, ed è voluto. Ma significa che un artefatto il cui identificativo fosse `items`, `mine`, `acquisti`, `diagnostica`, `contestazioni`, `video-check` o `system-artifacts` sarebbe *irraggiungibile*. Lo stesso vale per `/marketplace/sell` contro `/marketplace/<artifact_id>`, e per `/api/jobs/conclusi` contro `/api/jobs/<job_id>`.

Perché oggi non è un problema

Gli identificativi degli artefatti sono generati, non scelti dall'utente. Se un giorno diventassero scegliibili — uno *slug* leggibile nell'URL, per dire — questa lista diventerebbe un elenco di parole riservate da vietare in fase di pubblicazione.

3.3.4 Tre cose morte, da non seguire

`POST /api/market/publish`
Ritirata. Il docstring dice `"""RITIRATA."""` e la rotta risponde sempre `403`, ma resta registrata e occupa il percorso. La pubblicazione vera è `POST /api/marketplace/publish`, in `community.py`.

L'endpoint `auth_google_token`
Elencato fra quelli pubblici in `auth.py`, ma nessuna rotta ha quel nome. Riferimento pendente, innocuo.

Il prefisso `/claim`
Elencato fra i percorsi che richiedono un account, ma nessuna rotta comincia con `/claim`: la pagina delle contestazioni è `/marketplace/rimborso/<claim_id>`, già coperta dal prefisso `/marketplace`. L'equivoco nasce dal template, che si chiama `claim.html`.

3.3.5 Due template orfani

In `dashboard/templates/` ci sono 35 file. Due non sono raggiunti da nessuna rotta e da nessun `include`:

- `brokers.html` — il suo JavaScript è stato estratto in `static/js/brokers.js` e l'interfaccia dei broker vive ora dentro `live.html`. Residuo di una ristrutturazione.
- `server_runners.html` — i runner non stanno più sul server, ma sul computer dell'utente. Funzionalità rimossa, template rimasto.

Altri quattro non sono orfani ma non sono nemmeno pagine: `base.html` è il genitore di tutte le 32 pagine, `_robustness_panel.html`, `_v2_profile_security.html` e `_v2_studio_bp.html` sono frammenti inclusi. `serve_account.html` è un caso a sé: non lo rende nessuna rotta, lo rende la guardia di autenticazione (§ 3.4).

> **`live.html` non è la pagina di Studio**
>
> `studio.html` è servito da `/studio` e da `/studio/dati-esterni`; `live.html` è servito da `/studio/runner-live`. Il nome del percorso dice Studio, la pagina è Live. È facile sbagliare, e il blueprint che la registra si chiama `studio`.

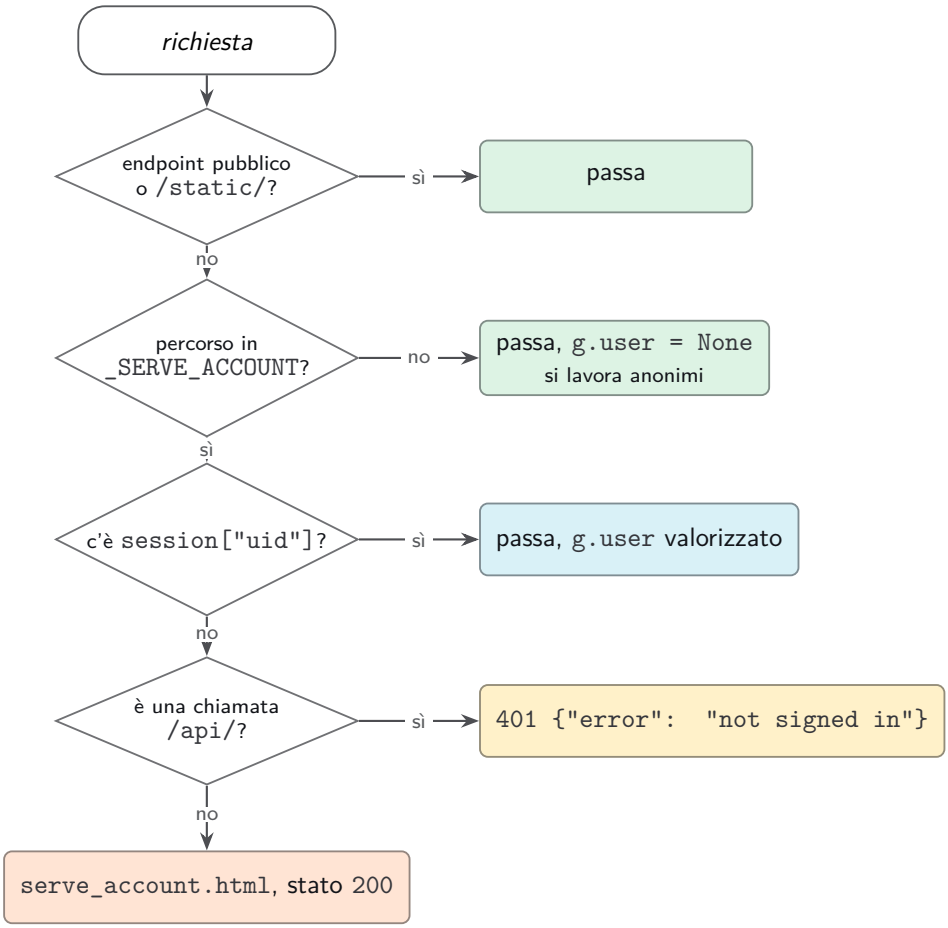
3.4 Chi può fare che cosa

Cercando un decoratore `@login_required` sulle rotte della dashboard non se ne trova quasi nessuno. Non è una dimenticanza: il controllo è uno solo, globale, e sta in `dashboard/auth.py`.

La guardia, ridotta all'osso — `dashboard/auth.py:1474`

```
@app.before_request
def _require_login():
    if request.endpoint in _PUBLIC_ENDPOINTS or request.path.startswith("/static/"):
        return None
    if not session.get("uid") and (not _serve_account(request.path)
                                   or request.path in _ANONIMO_AMMESSO):
        g.user = None
        return None            # <-- si lavora ANONIMI
    if not session.get("uid"):
```

```
        if "/api/" in request.path or ...:
            return jsonify({"error": "not signed in"}), 401
        return render_template("serve_account.html", destinazione=request.path), 200
    g.user = {...}
```



Il ramo marcato si lavora anonimi *è la scelta di fondo: la dashboard gira sul computer di chi la usa, e la gran parte delle rotte non chiede nulla. Solo i percorsi elencati in* `_SERVE_ACCOUNT` *ricadono nel ramo di sinistra.*

Figura 3.3. Il percorso di una richiesta. Il ramo di destra è quello che prende la grande maggioranza delle rotte.

3.4.1 Le tre liste

`_SERVE_ACCOUNT` — i prefissi che richiedono un account

L'elenco è scritto a mano, e il confronto avviene per segmenti: `/marketplace`, `/profile`, `/security`, `/admin`, `/messages`, `/api/market`, `/api/marketplace`, `/api/account`, `/api/admin`, `/api/messages`, `/api/community`, `/claim`.

`_PUBLIC_ENDPOINTS` — gli esenti

`auth_login`, `auth_logout`, `auth_register`, `auth_verify`, `auth_reset`, `static`, più i quattro dell'accesso con Google. Sono nomi di *endpoint*, non percorsi.

`_ANONIMO_AMMESSO` — l'eccezione dentro l'eccezione
Una voce sola: `/api/marketplace/claim/aperte`. Alimenta il contatore nella barra laterale, e senza account deve restituire un elenco vuoto invece di un `401` — altrimenti ogni pagina mostrerebbe un errore a chi non ha fatto l'accesso.

3.4.2 I quattro livelli, sopra la guardia

Livello	Come si verifica	Che cosa succede se manca
anonimo	—	Si lavora: tutto il motore è accessibile.
con account	`session["uid"]`	Pagina `serve_account.html` con stato `200`, oppure `401` in JSON.
amministratore	`auth.is_admin()`	`403`, oppure `404` per le sezioni che non devono rivelare di esistere.
super-amministratore	`auth.is_super_admin()`	`403`. È l'unico che può nominare o togliere un amministratore.

Perché `404` e non `403` su alcune rotte di amministrazione

Un `403` dice «esiste, e non puoi». Un `404` dice «non c'è». Per le sezioni che elencano utenti, feedback e artefatti di tutti, la seconda risposta non concede nemmeno l'informazione che quella sezione esista. La distinzione è usata in modo consapevole: `/admin`, `/api/admin/users`, `/api/admin/feedback`, `/api/admin/artefatti` e `/api/admin/beta` rispondono `404`; le altre rotte di amministrazione, che riguardano impostazioni di cui l'esistenza non è un segreto, rispondono `403`.

3.4.3 La scelta di fondo, e il suo prezzo

L'applicazione gira sul computer di chi la usa. Obbligare all'accesso avrebbe significato chiedere un account per entrare in casa propria — per questo `app.py` (tranne la parte di amministrazione), `studio.py`, `genetic.py` e `agent.py` sono aperti per intero.

Il prezzo è che **la porta 5 000 va considerata una console di amministrazione**: chi la raggiunge può lanciare backtest, leggere le credenziali del broker e scrivere file sul disco. È una decisione, non una falla, ma implica che l'applicazione non vada esposta in rete.

3.4.4 Quattro punti dove la protezione non è dove ci si aspetta

Conseguenze dirette del fatto che `_SERVE_ACCOUNT` è un elenco scritto a mano: ciò che non è elencato nasce aperto.

Percorso	Stato
`GET /subscription`	La pagina «Il mio abbonamento» non è fra i prefissi e non fa controlli: si apre senza account. I dati però arrivano da `/api/abbonamento*` e `/api/f/subscription/invoices`, che rispondono `401`: si vede l'impalcatura vuota.
`GET /feedback`, `GET /account`	Chiamano `_uid()` senza verificarne l'esito. Si aprono, ed elencano niente.
`/api/abbonamento*`, `/api/e/*`, `/api/f/*`, `/api/feedback`	Fuori dai prefissi: *ciascuno si difende da sé* con un controllo scritto nella rotta. Funziona, ma un endpoint nuovo aggiunto a queste famiglie nasce senza protezione.
`POST /api/feedback` e `POST /api/community/feedback`	Due endpoint per la stessa cosa, con autenticazione diversa: il primo si difende da sé, il secondo è coperto dal prefisso. Il secondo prima si chiamava come il primo.

Aggiungere una rotta che tocca l'account

1. Metterla sotto uno dei prefissi già elencati in `_SERVE_ACCOUNT`. È il modo che non si dimentica.
2. Se il percorso non può stare lì, aggiungere il prefisso nuovo a quella lista — non un controllo dentro la rotta.
3. Decidere se la rotta debba rispondere `403` o `404` a chi non è amministratore, e usare `_require_admin()` per il secondo caso.
4. Se la risposta deve esserci anche per gli anonimi (un contatore, un badge), aggiungerla a `_ANONIMO_AMMESSO` e restituire un risultato vuoto, non un errore.

3.4.5 Gli endpoint sull'account sono quattro

`GET /api/account/security` in `app.py`, `GET /api/f/account` in `ui_ext_f.py`, `GET /api/e/account-data` in `ui_ext_e.py` e `POST /api/account` in `community.py`. Tre moduli, quattro percorsi, tre stili di autenticazione diversi. Non è un difetto che rompa qualcosa, ma è il posto dove cercare quando un dato dell'account è aggiornato in una schermata e vecchio in un'altra.

4

Il motore quantitativo

4.1 Che cosa succede a ogni barra

Il ciclo principale sta in `backtest/engine.py`. Per ogni data del calendario, dopo il periodo di riscaldamento, fa sei cose nello stesso ordine.

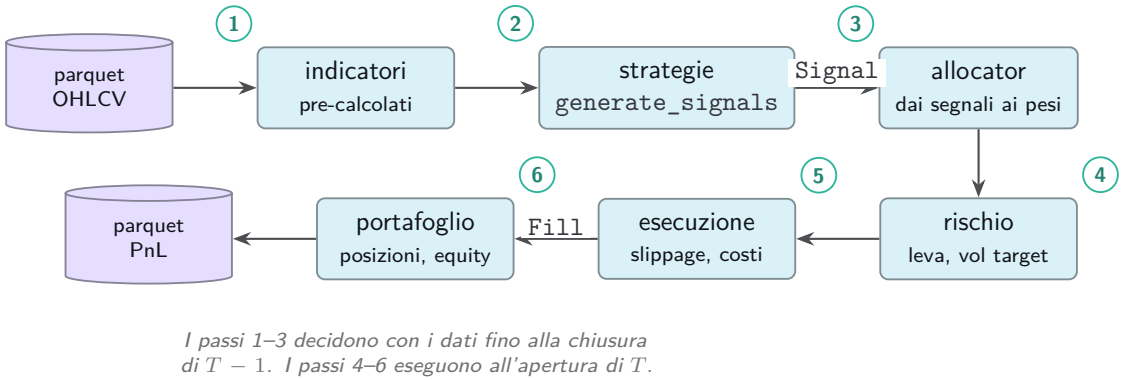


Figura 4.1. I sei passi del ciclo. La divisione fra la riga superiore e quella inferiore non è grafica: è la divisione fra decidere ed eseguire.

1. **Indicatori.** Non si calcolano dentro la strategia: si pre-calcolano una volta per simbolo e si passano già pronti. Che cosa serve lo dichiara ogni strategia nel proprio descrittore, e il registro ne fa l'unione.
2. **Strategie.** Ciascuna riceve gli indicatori e il portafoglio e restituisce una lista di `Signal`: che cosa, in che direzione, con quale convinzione. Non quanto.
3. **Allocatore.** Dai segnali ai pesi di portafoglio. È qui che si decide quanto, e il come è configurabile (§ 4.3).
4. **Rischio.** Tetti su peso per posizione, esposizione lorda per classe di attività, leva complessiva, volatilità obiettivo.
5. **Esecuzione.** Prezzo di apertura, slippage, costi.
6. **Portafoglio.** Posizioni aggiornate, equity segnata a mercato, storia registrata.

4.1.1 Prima di tutto: i trigger intra-barra

C'è un passo che precede la decisione, e non compare nella figura perché non è parte del ciclo decisionale: il controllo degli *stop* e dei *take profit* sulle posizioni già aperte.

Il controllo che viene prima di tutto `backtest/engine.py:745`

```
            # ---- INTRA-BAR TRIGGER CHECK ----
            # PRIMA della decision/execution phase, controlla se i SL/TP delle
```

```
            # posizioni aperte sono stati toccati nella bar t (high/low).
            trigger_hits = exit_tracker.check_bar(t, data)
```

Se un livello è stato toccato, la posizione si chiude subito e il simbolo finisce in una lista nera fino al prossimo ribilanciamento vero. La filosofia dichiarata nel codice è che «ribilanciare è quando rivaluti la tesi»: uno stop toccato non aspetta il prossimo ribilanciamento, ma nemmeno autorizza a riaprire la stessa posizione nella stessa barra.

La lista nera non blocca le chiusure

Un simbolo in lista nera non può essere *aperto*, ma un target a zero passa sempre. Invertire questa asimmetria — cioè bloccare anche le chiusure — significherebbe tenere aperta una posizione che il sistema ha deciso di chiudere, ed è il tipo di errore che si scopre su un conto reale.

4.2 Decisione ed esecuzione sono due istanti

È la regola più importante del motore, e si riduce a due righe di codice.

Le due righe che tengono in piedi tutto `backtest/engine.py:740`

```
            # t_execution = t (open T)
            # t_decision = data point precedente in all_dates (close T-1)
            t_execution = t
            t_decision = all_dates[i - 1]
```

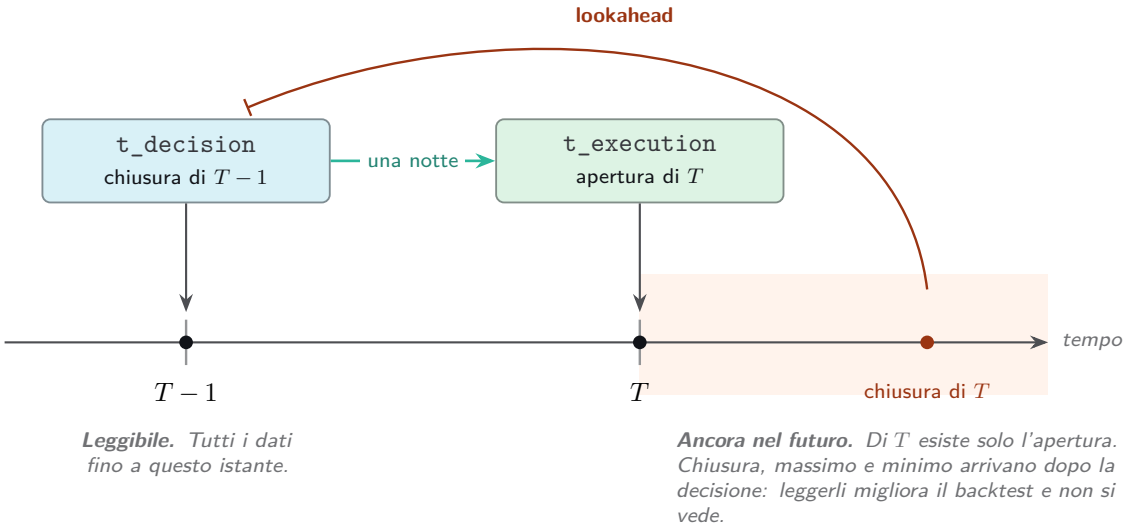


Figura 4.2. I due istanti. Tutto ciò che sta nella fascia a destra non esiste ancora quando si decide.

4.2.1 Perché due istanti e non uno

Una strategia che decide alla chiusura di T e compra alla chiusura di T non è implementabile: nel momento in cui si conosce il prezzo di chiusura, la sessione è finita. Le implementazioni ingenue lo fanno lo stesso, e il backtest che ne esce è eccellente.

Qui la decisione avviene con i dati disponibili fino alla chiusura di $T-1$ e l'ordine viene

eseguito al prezzo di apertura di T:

La fase di esecuzione backtest/engine.py:945

```
            # ---- EXECUTION PHASE: open T+1 = t_execution ----
            # Per ogni target ottengo l'open price a t_execution.
            ...
                df_sym = data.get(sym)
                if df_sym is None or t_execution not in df_sym.index:
                    # Symbol non disponibile, skip silenzioso
                    continue
                open_price = float(df_sym.loc[t_execution, "open"])
```

4.2.2 La firma che non si può cambiare

Le strategie ricevono i due istanti come argomenti *nominati*:

Il contratto backtest/engine.py:834

```
                        t_decision=t_decision,
                        t_execution=t_execution,
```

Il motore li passa per nome, quindi una strategia che chiami i suoi parametri in modo diverso fa fermare il backtest alla prima barra con un `TypeError`. Il messaggio di errore del motore lo dice esplicitamente, ricordando la firma richiesta:

backtest/engine.py:857

```
                            "Firma richiesta: generate_signals(self, t_decision, "
                            "t_execution, indicators_by_symbol, portfolio) -> "
```

Il bug che migliora i risultati

Dentro `generate_signals` si legge solo fino a `t_decision`:

Corretto	Lookahead
`ind.close.loc[:t_decision]`	`ind.close.loc[:t_execution]`
`serie.shift(1)` prima di decidere	`serie` senza shift

La seconda colonna non dà errore, non produce avvisi, e fa salire lo Sharpe. È per questo che la separazione è imposta dalla firma invece che raccomandata in un commento: una regola che si può violare senza accorgersene viene violata.

4.2.3 La stessa regola altrove

La causalità non riguarda solo le strategie. Si ripresenta identica in tre altri punti, e in ciascuno è scritta nel codice come vincolo:

Dove	Come
Regime	Lo snapshot è calcolato con `as_of=True`, che usa solo dati anteriori a `t_decision`.
Overlay	Il moltiplicatore si calcola dal PnL *base* e si applica shiftato: «la decisione di oggi non può usare il rendimento di oggi».
Regole di regime	Niente statistiche sull'intero campione: un quantile calcolato su tutta la storia è lookahead travestito da soglia. Gli helper condivisi (`_expanding_median`, `_hysteresis`) esistono per non doverlo riscrivere ogni volta.
Normalizzazione della volatilità	La vol trailing viene shiftata di un giorno nel backtest (§4.4).

4.3 Dall'intenzione alla posizione

Una strategia dice *che cosa*; l'allocatore decide *quanto*. Sta in `core/allocator.py`, e il suo comportamento si governa con una sola `dataclass` di configurazione.

4.3.1 Le quattro modalità

`strategy_mix_mode`	Come distribuisce
`equal_weight`	Gerarchico classico: il portafoglio si divide fra le strategie, poi dentro ciascuna. Il fattore $\sqrt{N_{\text{strat}}}$ assume indipendenza fra strategie. È il valore predefinito, ed è anche il ripiego quando le altre modalità non hanno i dati che servono.
`stream_level`	Ogni strategia opera come stream indipendente con un proprio budget di volatilità (`stream_target_vol_annual`, 0.12 di default); i contributi si sommano con peso $1/N$.
`explicit`	Si esegue il vettore di pesi dichiarato dalla strategia, punto. Serve a chi ha già calcolato le proprie dimensioni e non vuole che nessuno le tocchi.
`covariance`	Le correlazioni si *stimano* invece che assumerle: la volatilità del libro è $\sqrt{w^{\top}\Sigma w}$, con Σ da `core/covariance.py`.
`bayesian`	Dichiarata e non implementata: solleva `NotImplementedError`. È elencata nei valori ammessi perché il messaggio d'errore è più utile di un «valore non valido».

Perché `explicit_vol_target` è un campo a sé

In modalità `explicit` si può chiedere che il libro venga scalato una volta sola perché la sua volatilità prevista sia un certo valore — le proporzioni fra i titoli restano identiche, cambia solo quanto pesa il libro nel suo insieme. Il parametro non è `portfolio_target_vol_annual` perché quello ha sempre un valore (ha un default), e con un default non si distingue «lo voglio» da «c'era». Qui l'assenza significa qualcosa: `None` vuol dire «esegui ciò che ti ho dato, senza scalare».

4.3.2 I tetti di rischio

Parametro	Default	Che cosa limita
`portfolio_target_vol_annu`	0.10	Volatilità annua obiettivo del portafoglio.
`max_position_weight`	0.10	$\lvert w \rvert$ del singolo strumento.
`max_asset_class_gross`	0.20	$\sum \lvert w \rvert$ dentro una classe di attività.
`max_gross_leverage`	1.5	$\sum \lvert w \rvert$ totale.
`min_vol_floor`	0.05	Pavimento sulla volatilità stimata.

Il ragionamento dietro il tetto per classe

Il commento nel codice lo dice senza giri: quattro obbligazioni tutte lunghe fanno il 40 % lordo senza alcun tetto, e in situazione di stress sono una posizione sola. Il tetto al 20 % limita a «un'esposizione obbligazionaria con leva 2 al massimo», che è realistico. Non è un numero scelto per prudenza generica: è la risposta a un modo preciso in cui una diversificazione apparente sparisce.

4.3.3 La validazione è nel costruttore

`__post_init__` controlla ogni campo e solleva `ValueError` con il valore ricevuto nel messaggio. Due vincoli meritano attenzione perché non sono ovvi:

- `max_gross_leverage >= 1.0`. Un tetto inferiore a 1 significherebbe non poter mai investire tutto il capitale, ed è quasi sempre un errore di unità di misura.
- `max_asset_class_gross` ammesso fino a 4.0. Il limite è stato ampliato per sostenere i portafogli lungo-corto, che arrivano a un lordo di 2 e oltre.

La configurazione è congelata

`AllocatorConfig` è una `dataclass` congelata: non si modifica un campo dopo averla costruita. Per variarne uno in un test si usa `dataclasses.replace(config, max_gross_leverage=2.0)`, che ne restituisce una copia nuova. Assegnare direttamente solleva `FrozenInstanceError`, ed è voluto: una configurazione di rischio modificata a metà di un backtest renderebbe i risultati non riproducibili.

4.4 Comporre più stream

Una strategia da sola è uno stream. `EnsembleEngine`, in `reporting/ensemble.py`, ne prende diversi e ne fa uno solo. Il percorso ha tre stadi, e due di essi sono punti d'innesto per i plugin.

4.4.1 Primo stadio: portare gli stream alla stessa scala

Due strategie con lo stesso Sharpe ma volatilità diversa non vanno pesate allo stesso modo: la più volatile dominerebbe il risultato indipendentemente dal peso dichiarato. Ogni stream viene quindi riportato a una volatilità obiettivo comune prima di essere pesato.

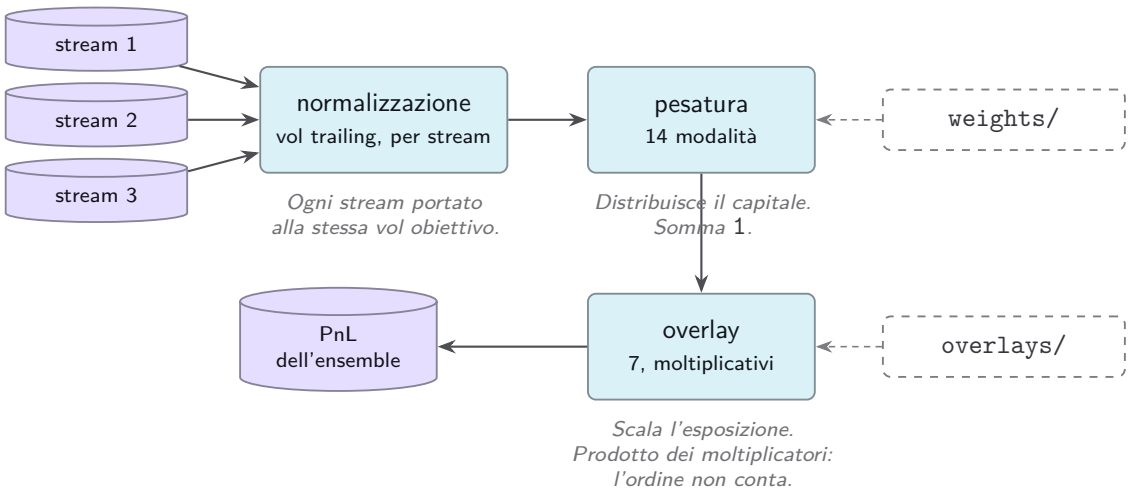


Figura 4.3. La composizione. Pesatura e overlay fanno cose diverse e non vanno confuse: la prima distribuisce, la seconda scala.

Il mattone è uno solo, e serve sia al backtest sia all'operatività dal vivo:

Il mattone comune `reporting/ensemble.py:335`

```python
    def _vol_trailing(self, returns: pd.Series) -> pd.Series:
        """Vol annualizzata TRAILING e NON shiftata: il valore a t usa le
        osservazioni fino a t INCLUSO.

        E' il mattone comune fra backtest e live. Il backtest la shifta di un
        giorno (`_compute_scale_series`) perche' lo scale applicato al ritorno
        di t deve conoscere solo il passato; il live la usa non shiftata
        (`scala_corrente`) perche' li' l'ultima osservazione e' il ritorno
        appena realizzato e la posizione che si sta per aprire rendera' nel
        periodo SUCCESSIVO. Due usi, una sola formula: e' l'unico modo perche'
        il libro che si esegue sia quello che si e' misurato.
        """
```

Perché uno shift di differenza non è un dettaglio

Nel backtest il fattore applicato al rendimento di t deve conoscere solo il passato, quindi la serie si shifta di uno. Dal vivo l'ultima osservazione *è* il rendimento appena realizzato, e la posizione che si sta per aprire renderà nel periodo successivo: shiftare anche lì significherebbe dimensionare con una stima vecchia di un giorno. Stesso numero, due usi, e se le due strade calcolassero la volatilità ciascuna per conto proprio il libro eseguito non sarebbe quello misurato. Il motivo per cui la funzione è una sola è esattamente questo.

Due costanti governano i casi limite:

Costante	Valore	A che serve
`SCALE_MIN_OBS`	20	Sotto questa soglia non c'è stima, e il fattore vale `1.0`: il libro resta quello dell'allocatore, mai amplificato da una volatilità stimata su quattro dati.
`SCALE_MAX_LEVA`	10.0	Con volatilità che tende a zero il fattore esploderebbe. La volatilità ha quindi un pavimento a `target_vol / SCALE_MAX_LEVA`.

4.4.2 Secondo stadio: la pesatura

Quattordici modalità installate, raggruppate per famiglia:

Famiglia	Tipo	Modalità
Ingenua	statica	`equal`
Rischio / covarianza	statica	`inverse_cov`, `min_variance_lw`, `min_variance_rmt`, `erc`
Robusta	statica	`hrp`, `nco`, `cluster_risk_budget`, `max_div`
Coda	statica	`cvar`
Con vista sui rendimenti	statica	`black_litterman`, `kelly`
Adattiva al regime	**rolling**	`regime_ml`, `regime_style`

Il campo `kind` del `META` governa il dispatch, e non è documentazione: è la condizione di un `if`.

Il dispatch, senza una catena di if — `reporting/ensemble.py:491`

```
        spec = next((m for m in self.WEIGHTING_MODE_REGISTRY
                     if m["key"] == self.weighting_mode), None)
        if spec is None:
            raise ValueError(f"Unknown weighting_mode: {self.weighting_mode}")
        weight_fn = _WEIGHTING_COMPUTERS[self.weighting_mode]
        if spec["kind"] == "static":
            weights = weight_fn(self, stream_pnls)
        else:  # rolling: pesi time-varying; per la UI esponi la MEDIA come scalare
            weights_over_time = weight_fn(self, stream_pnls)
            mean_w = weights_over_time.mean(axis=0)
            weights = {name: float(mean_w[name]) for name in stream_pnls}
```

Il modello documenta un valore che non esiste

`weights/WEIGHT_MODE_TEMPLATE.md` dice che `kind` vale `"static"` o `"dynamic"`. Nel codice il confronto è con `"static"`, e *tutto il resto* finisce nel ramo rolling. I quattordici plugin installati usano `static` (dieci) e `rolling` (quattro); `dynamic` non lo usa nessuno. Un plugin scritto seguendo il modello funzionerebbe per caso. Il modello è da correggere, il codice no.

4.4.3 Terzo stadio: gli overlay

Ogni overlay attivo produce un moltiplicatore in $(0, 1]$ calcolato dal PnL *base*, cioè da quello prima di qualunque overlay. I moltiplicatori si moltiplicano fra loro.

L'applicazione degli overlay `reporting/ensemble.py:555`

```
        # Overlay di esposizione CAUSALI (ortogonali al weighting_mode e componibili
        # fra loro). Ogni overlay attivo produce un moltiplicatore in (0,1] calcolato
        # dal PnL/stream BASE (=> ordine-indipendente); l'esposizione finale e' il
        # PRODOTTO dei moltiplicatori. Nessun overlay attivo => ensemble_pnl invariato.
        base_pnl = ensemble_pnl
        overlay_multipliers: dict[str, pd.Series] = {}
        for ov in self.OVERLAY_REGISTRY:
            if ov["key"] in self.enabled_overlays:
                m = _GATE_COMPUTERS[ov["key"]](self, base_pnl, stream_pnls)
                m = m.reindex(base_pnl.index).fillna(1.0)
                overlay_multipliers[ov["key"]] = m
        if overlay_multipliers:
            total_mult = pd.Series(1.0, index=base_pnl.index)
            for m in overlay_multipliers.values():
                total_mult = total_mult * m
            ensemble_pnl = base_pnl * total_mult
```

L'indipendenza dall'ordine è una proprietà costruita

Se ogni overlay calcolasse il proprio moltiplicatore dal PnL già modificato dai precedenti, il risultato dipenderebbe dall'ordine in cui compaiono nel registro — cioè dal campo `order`, che è un dato di presentazione. Calcolandoli tutti dal PnL base, l'ordine smette di contare e attivare due overlay dà lo stesso risultato comunque li si elenchi.

4.4.4 I sette overlay installati

Chiave	Famiglia	Quando taglia
`vol_regime`	Volatilità	Quando la volatilità realizzata sale. Mira al drawdown massimo.
`jump_vol`	Volatilità	Reazione in un giorno a salti oltre N sigma: copre il rischio di apertura che gli overlay lenti perdono.
`corr_gate`	Correlazione	Quando la correlazione media fra gli stream schizza, cioè quando la diversificazione sparisce.
`regime_gmm`	Stress	Miscela gaussiana su feature multivariate; riduzione graduale al crescere della probabilità di stress.
`iso_stress`	Stress	Isolation forest su dispersione, volatilità, ampiezza, correlazione media.
`macro_regime`	Macro	Fattore latente (PCA causale) su curva, spread di credito, VIX, disoccupazione.
`news_pressure`	Flusso informativo	Quando il volume di notizie supera la norma storica. È l'unico che dichiara dati esterni e una scadenza giornaliera.

4.5 Lo stesso calcolo, su un conto vero

`live/` non è un secondo motore: è lo stesso, con un broker al posto dei dati storici. Il valore di tutto il progetto dipende da questa frase, e il modo in cui viene mantenuta vera merita di essere letto per intero, perché è anche l'esempio migliore di che cosa va storto quando non lo è.

4.5.1 Il difetto che ha definito il progetto

Fino a un certo punto, il backtest normalizzava la volatilità di ogni stream e l'operatività dal vivo no. Il commento in testa a `live/ensemble_live.py` dice che cosa significava:

> *Senza il punto 2 un libro dichiarato al 4 % di vol veniva mostrato in backtest moltiplicato per 2.5 ed eseguito dal vivo a dimensione piena: 0.4 volte il rischio e il rendimento misurati. Al contrario, un libro al 25 % veniva tagliato a 0.4 in backtest e girava dal vivo a 2.5 volte.*

Nessun errore, nessuna eccezione, nessun avviso. Il backtest mostrava un numero e il conto ne produceva un altro, e la differenza era un fattore fra 0.4 e 2.5 a seconda della volatilità dello stream — cioè più grande quanto più il libro si discostava dall'obiettivo, che è esattamente il caso in cui si ha bisogno che il backtest sia attendibile.

Perché il rimedio è stato condividere la funzione

Si sarebbe potuto riscrivere la normalizzazione in `live/` copiando la formula. Due implementazioni della stessa formula restano identiche finché qualcuno non ne tocca una, e questo difetto è nato esattamente così. La soluzione è che `_vol_trailing` è *una*, e le due strade la usano in modo diverso in un solo punto documentato: il backtest la shifta di un giorno, il live no (§ 4.4).

4.5.2 I due punti in cui la normalizzazione entra

1. **I pesi si calcolano sui rendimenti scalati**, non su quelli grezzi. Le modalità di pesatura guardano la matrice di covarianza: su serie non normalizzate uno stream al 25 % di volatilità riceve un peso del tutto diverso da quello che riceveva in backtest.
2. `scales()` **restituisce il fattore da applicare adesso** al vettore di posizioni — cioè la leva che il backtest stava mostrando e che il live non applicava.

4.5.3 Il riscaldamento

La volatilità trailing non esiste prima di una ventina di osservazioni, e fra venti e sessanta è ancora rumorosa. In quel tratto il sistema mescola la stima con una volatilità *a priori* — per uno stream di approccio, la sua volatilità di backtest — con peso lineare crescente.

Osservazioni	Che cosa si usa
meno di `SCALE_MIN_OBS`	Solo la volatilità a priori. Senza una, il fattore è `1.0`.
fra le due soglie	Miscela lineare fra priori e stima.
oltre `scale_min_obs`	Solo la stima: qui coincide con il backtest.

Il requisito dichiarato è: *mai un salto di dimensione in un solo giorno.* Una transizione netta fra priori e stima produrrebbe, il giorno in cui la soglia viene superata, un ordine di ribilanciamento che non corrisponde a nessun cambiamento del mercato.

Lo stato sopravvive ai riavvii

I rendimenti per stream sono salvati su JSON, così la finestra non riparte da zero a ogni riavvio del processo. Senza, ogni riavvio rimetterebbe il sistema in riscaldamento, e il riscaldamento usa la volatilità a priori: il conto cambierebbe dimensione per il solo fatto di essere stato riavviato.

4.5.4 I broker

L'interfaccia è `live.broker.LiveBroker`, la stessa classe che `ports/` ri-esporta come porta `Broker`. Le implementazioni attuali coprono Interactive Brokers (`live/brokers/ibkr.py`), MetaTrader 5 (`live/brokers/mt5.py`), l'esecuzione su carta e un broker di prova con il tempo accelerato — quest'ultimo serve alla pagina di simulazione, che fa passare una giornata di mercato in pochi secondi.

Oltre ai metodi obbligatori, cinque *ganci* con un default innocuo:

Metodo	A che serve
`account_label()`	Una riga che dice a quale conto ci si è collegati: numero, demo o reale, terminale, cambio applicato. Va nel registro all'avvio.
`account_identity()`	Chi è il conto, in un dizionario (`broker`, `account`, `where`, `kind`); `None` se il broker non sa dirlo. Lega la conferma degli ordini reali a un conto (§ 4.5.5).
`real_orders_blockers()`	I motivi per cui il conto *non* può ricevere ordini da Lyra. Se la lista non è vuota, un runner con ordini reali non parte; se compare mentre gira, i cicli si saltano.
`open_orders()`	Gli ordini ancora in lavorazione, `{simbolo: quanti}`. Il motore non manda un ordine su un simbolo che ne ha uno in corso.
`cancel_open_orders()`	Cancella gli ordini aperti *di Lyra* (non quelli fatti a mano) e dice quanti. Il runner la chiama all'avvio e all'arresto.

L'esito di ogni ordine è un `Fill` con `status` fra `filled`, `partial`, `pending` (accettato, né eseguito né cancellato), `skipped` (sotto la quantità minima, non inviato), `rejected`, `cancelled` ed `error`, e una `note` con il perché nelle parole del broker. Un adattatore che non riesce a leggere il conto solleva `BrokerUnavailable` (una `ConnectionError`): il runner lo riprova invece di fermarsi. Anche un fill parziale entra nel libro della strategia (`LivePortfolioMirror.record_fill`).

Adattatore	Che cosa garantisce
IBKR	L'ordine va al conto indicato (`order.account`) e porta il riferimento del runner (`orderRef lyra:<id>`); equity e posizioni si leggono di quel conto e solo per i contratti `STK`, così un'opzione su AAPL non viene letta come azioni AAPL. Un ordine non eseguito entro `fill_timeout_s` viene cancellato; se la cancellazione non arriva entro `cancel_timeout_s` l'esito è `pending`, e un ordine `Inactive` viene cancellato perché può riattivarsi. `equity_in_valuta()` converte la `NetLiquidation` della valuta base del conto con il tag `ExchangeRate` (in riserva il cambio dei dati di Lyra): senza, un conto in euro risultava a equity zero. `open_orders()` legge `reqAllOpenOrders()` (solo `STK`, solo il conto); `cancel_open_orders()` cancella solo i riferimenti `lyra`. Il collegamento passa `account` e un `timeout` di 20 s. Posizioni, valori del conto e stati degli ordini di `ib_insync` si aggiornano solo mentre il suo event loop gira, e fra un ciclo e l'altro il runner dorme: prima di leggere il conto `_aggiorna()` fa girare il loop e le posizioni si chiedono al server (`reqPositions()`), altrimenti un ordine eseguito durante la notte risultava ancora da fare; se la richiesta fallisce l'errore sale, senza ripiegare sulla copia locale. Le richieste bloccanti hanno `RequestTimeout` di 20 s (di default aspettano per sempre). Si importa `ib_async`, il seguito mantenuto, oppure `ib_insync`, nel thread che lo usa.
MT5	Il riempimento segue `symbol_info.filling_mode` (IOC, FOK o Return): imporre IOC fa rifiutare ogni ordine sui simboli che non lo accettano. Il retcode 10 010 è un'esecuzione parziale, il 10 008 un ordine accettato non ancora eseguito (`pending`). Un conto *hedging*, il trading disabilitato, `trade_expert` falso, **Algo Trading** spento o `tradeapi_disabled` bloccano gli ordini reali. Un conto in un'altra valuta si converte con il tick della coppia nel terminale (`EURUSD`, anche con il suffisso del broker). Gli ordini in attesa vengono da `orders_get()`; si cancellano con `TRADE_ACTION_REMOVE` solo quelli con il `magic` di Lyra. `None` da `positions_get()`, `orders_get()` o `account_info()` è un errore dell'API, non un conto vuoto: letto come «nessuna posizione» faceva ricomprare il libro. `is_connected()` interroga il terminale, così dopo un suo riavvio ci si ricollega. Il pacchetto parla con un terminale per processo: le chiamate passano da un lucchetto, e `shutdown()` lo chiama solo l'ultimo che si scollega.

Le credenziali restano su file locale

`/api/brokers/credentials` legge e scrive un file sul computer dell'utente. Non passano dal sito, non finiscono nel database, e non c'è nessun percorso che le porti in rete. Questo è il motivo per cui la porta della dashboard non va esposta: chi la raggiunge può leggerle. Per IBKR porta e client ID sono controllati al salvataggio, e `live_runners.cfg_ibkr()` li passa al collegamento come `broker_cfg`.

4.5.5 Simulazione o ordini reali

Un runner su un broker vero parte in simulazione: `dry_run` vero, il motore calcola gli ordini e non li invia. Gli ordini reali richiedono tre cose insieme, e nessuna basta da sola:

Quando un runner invia davvero dashboard/live_runners.py:216

```
def ordini_reali(runner: dict) -> bool:
    r = runner or {}
    return (str(r.get("broker") or "") in BROKER_REALI
            and r.get("dry_run") is False
            and bool(r.get("real_orders_confirmed_at")))
```

`dry_run` deve essere `False` in modo esplicito — non basta che manchi — e la conferma la scrive solo una richiesta che la porta: `confirm_real_orders` quando si salva il runner, `confirm` quando si cambia modalità. L'avvio di un runner con ordini reali chiede a sua volta `confirm_real`. Le rotte `/api/live/*`, rimaste per le prove da script, su un broker vero girano sempre in simulazione: gli ordini reali passano solo dai runner, dove ci sono conferma, registro e kill-switch salvato.

La conferma è legata a un conto — Al primo avvio dopo una conferma, `_prepara_conto` scrive in `data/runners/<id>/account.json` la chiave del conto (`chiave_conto()`: broker, server per MT5, numero) insieme a `real_orders_confirmed_at`. Agli avvii successivi con la stessa conferma un conto diverso ferma l'avvio (`_NonParte`); a ogni ciclo `_controlla_conto` confronta di nuovo l'identità e, se è cambiata, ferma il runner prima del ciclo (`_Fermo`) senza toccare gli ordini del conto nuovo. Una conferma nuova lega il runner al conto del prossimo avvio (`legame_valido()`). Senza identità (`account_identity()` a `None`), o con `account.json` illeggibile (`leggi_legame()`), gli ordini reali non partono.

4.5.6 Il registro del runner

Ogni riga scritta con `REGISTRO.log(rid, testo, kind, data)` va in memoria (le ultime 200, per lo stato) e su disco, in `data/runners/<id>/journal.jsonl`: un oggetto JSON per riga con `ts` (UTC), `session` (una per avvio), `mode`, `broker`, `kind`, `msg` e, quando serve, `data`. Oltre 5 MB il file ruota su `journal.1.jsonl` … `journal.9.jsonl` (`_ruota_registro`), e `registro_leggi` li ripercorre all'indietro finché servono righe. I `kind` sono `mode`, `start`, `stop`, `info`, `cycle`, `order` (simulati), `fill` (inviati, con l'esito), `warning`, `error`, `risk` e `maintenance`.
Nella stessa cartella stanno lo stato del motore (`live_state.json`), la storia della normalizzazione (`ensemble.json`), lo stato del `RiskManager` (`risk.json`) e il legame con il conto (`account.json`). Prima stato e normalizzazione erano un file unico in `data/`: due runner attivi insieme se li sovrascrivevano a vicenda.

4.5.7 Dati vecchi

`LoaderLiveFeed` legge gli stessi file del backtest e non scarica niente. Se nessuno li aggiorna, il motore continua a decidere sugli stessi prezzi — e con ordini veri dimensiona le quantità su prezzi di giorni prima. `live.freshness.motivo_dati_vecchi` conta le *sedute mancanti* fra l'ultima barra e l'ultimo giorno di borsa prima di oggi (UTC), sul calendario del mercato del simbolo (`calendario_per`, dall'asset class). Prima di oggi, perché il feed legge i file con una data di fine esclusiva: la barra di oggi la serve domani, e pretenderla dopo la chiusura americana tratteneva ogni ordine dalle 22 a mezzanotte.

Calendario	Sedute	Tolleranza
`nyse`	Lunedì–venerdì meno le festività NYSE (regole di `pandas`: Venerdì Santo, Juneteenth dal 2022, 4 luglio e Natale con il giorno osservato)	nessuna
`fx`	Lunedì–venerdì	una seduta
`crypto`	Tutti i giorni	una seduta

Mancano le chiusure straordinarie (lutti nazionali, uragani): quel giorno i dati risultano vecchi di una seduta e gli ordini aspettano, cioè l'errore va nella direzione prudente. Per barre infragiornaliere si contano le barre mancanti in tempo di borsa (oltre tre, vecchi); per barre settimanali, oltre due periodi e tre giorni.
Il motore misura ogni simbolo a ogni ciclo, anche in simulazione (`_dati_vecchi`), e riporta il riassunto in `CycleReport.data_stale`. Fuori dal dry-run trattiene gli ordini dei simboli indietro e lo scrive in `orders_held`: un'ultima barra fresca su SPY non rende freschi i prezzi di AAPL. Un simbolo del runner con una posizione aperta e nessun prezzo nel ciclo (file mancante, simbolo saltato dal caricamento) finisce nello stesso riassunto: il motore non può né valutarlo né ridurlo. In entrambi i casi gli ordini che aumentano l'esposizione aspettano (vedi *Prima le riduzioni*, più avanti). Passano solo le chiusure ordinate dal kill-switch del conto, che vendono la quantità che c'è e non dipendono dal prezzo. Il controllo vale per ogni broker tranne quello su carta (`guard_stale_data`), che il simulatore TEST e la prova da terminale guidano con il proprio orologio.

4.5.8 Il kill-switch del conto

Il `RiskManager` vive quanto il processo, e il livello del conto protegge del denaro che sta nel conto. Per questo, con ordini reali, picco di equity e chiusura sono salvati per *conto*, in `data/accounts/<chiave>.json` (`carica_stato_conto`, `salva_stato_conto`); mai nella cartella temporanea, che il sistema svuota. Il resto dello stato — strategie sospese con la data, asset class ridotte con la scadenza, finestre di PnL — va in `risk.json` del runner con il suo contesto (modalità, broker, conto), attraverso `export_state()` e `restore_state()`: uno stato per contesto (`contesti_rischio`), così una corsa in simulazione non cancella le strategie sospese della modalità reale.

- Il drawdown si controlla all'inizio del ciclo, appena letta l'equity (`check_account`), non solo dopo gli ordini: un calo avvenuto fra due cicli faceva partire un altro giro di acquisti prima che il kill-switch scattasse. Con il conto chiuso il ciclo procede anche con l'equity a zero: chiudere non ne ha bisogno.
- Passare a simulazione e tornare agli ordini reali, o usare un altro runner sullo stesso conto, non riapre il conto: le rotte di creazione e di cambio modalità non toccano più lo stato. Lo riapre solo `POST /api/runners/<rid>/kill-switch/reset`, che azzera il file del conto a cui il runner è legato.
- Uno stato del conto illeggibile ferma gli ordini reali: trattarlo da vuoto riaprirebbe un conto chiuso.
- Se all'avvio l'equity è già sotto il picco del conto oltre la soglia, il kill-switch scatta e si scrive nel file del conto, ma il runner non manda ordini: non può sapere se è un prelievo o una perdita. Al prossimo avvio chiude (perdita); il reset lo evita (prelievo). Con l'equity non leggibile all'avvio non parte.

- Un conto chiuso non impedisce l'avvio: il runner parte in sola chiusura (il `RiskManager` riparte chiuso, ogni peso è zero), chiude e a conto piatto (`_posizioni_aperte`) si ferma; in sola chiusura nessun errore lo ferma. L'elenco dei runner segnala la sola chiusura (`close_only`) solo se il conto chiuso è quello della conferma attuale; altrimenti `killed_unbound`.
- Quando il kill scatta durante un ciclo, il ciclo di chiusura parte subito, senza aspettare la cadenza; se resta qualcosa di aperto si riprova dopo `min(passo, 60 s)` per tre volte, poi alla cadenza normale.

Il drawdown del conto si controlla a ogni ciclo (`check_account`); i livelli per strategia e per asset class, che lavorano su finestre di barre, una volta per barra nuova (`_nuova_barra_rischio`), come nel backtest: aggiornarli a ogni ciclo — quattro per barra — accorciava la finestra di 30 giorni a 35 cicli. Il PnL realizzato fra due barre si attribuisce tutto alla barra nuova (`LivePortfolioMirror.consolida_realizzati`). Il livello per asset class riceve dal ciclo il PnL di prezzo per classe una volta per barra (`_registra_pnl_asset_class`). Nel backtest `update` chiama `check_account` con la stessa regola di prima: i risultati non cambiano.

4.5.9 Il ciclo dal vivo

Il runner fa il primo ciclo subito, poi uno ogni quarto della barra più corta che guarda (`cadenza_reale_secondi`, mai sotto i 15 secondi): per strategie giornaliere, ogni sei ore. L'avvio dalla pagina imponeva 60 secondi, e un runner giornaliero rifaceva lo stesso ciclo ogni minuto. Ogni runner gira in un thread suo, con un proprio event loop `asyncio`: `ib_insync` ne ha bisogno per collegarsi, e un thread che non è il principale non ne ha uno. Fermato il runner, gli ordini di Lyra ancora aperti si cancellano (con ordini reali, sul conto verificato), il collegamento al broker si chiude — altrimenti IBKR terrebbe occupato il client ID — e il loop con lui.

- **Un avvio solo.** `start` tiene un lucchetto dai controlli fino alla registrazione del runner: con due richieste ravvicinate potevano partire due motori sullo stesso conto. Un errore dopo la registrazione rimette il runner fermo.
- **Equity assente.** Un conto con equity zero o mancante salta il ciclo (`skipped_reason` `no_account_equity`): prima ogni peso andava a zero e partiva una chiusura per ogni posizione.
- **Ordini in corso.** Un simbolo con un ordine ancora aperto al broker non riceve un ordine nuovo; se gli ordini aperti non si leggono, non parte nulla.
- **Prima le riduzioni.** Gli ordini che riducono una posizione (`_riduce`) partono prima di quelli che la aumentano, e un'inversione diventa chiusura più apertura (`_separa_inversioni`; la gamba di apertura conta come aumento). Se qualcosa è trattenuto (dati vecchi, ordine in corso, posizione senza prezzo) o una riduzione non è `filled` né `skipped`, gli aumenti del ciclo non partono: in una rotazione con la vendita trattenuta, l'acquisto da solo portava l'esposizione oltre quella del libro vecchio e di quello nuovo.
- **Stop fra un ordine e l'altro.** Il runner assegna al motore `stop_requested`: premuto Stop, gli ordini non ancora inviati del ciclo in corso non partono.
- **Cicli falliti.** Un errore di collegamento (`e_transitorio`: `ConnectionError` e quindi `BrokerUnavailable`, `TimeoutError`, `OSError`) si riprova senza limite, con attese che raddoppiano fino a 15 minuti. Un'eccezione di altro tipo si riprova dopo `min(passo, 60 s)`; tre di fila fermano il runner con la causa, tranne in sola chiusura. Un ciclo saltato non tocca né il kill-switch né la manutenzione.

Rotta	Che cosa fa
`GET` /api/runners	I runner salvati, con `mode`, `real_orders`, `killed`, `close_only`, il conto legato, `data_stale`, `account_blocked` e le soglie del kill-switch.
`POST` /api/runners	Crea o aggiorna. Ordini reali solo con `dry_run: false` e `confirm_real_orders: true`.
`POST` /api/runners/<rid>/mode	Simulazione o reale, a runner fermo; il reale chiede `confirm`.
`POST` /api/runners/<rid>/start	Avvia. Con ordini reali chiede `confirm_real`; senza `interval`, la cadenza viene dal timeframe.
`POST` /api/runners/<rid>/stop	Ferma il runner: gli ordini non ancora inviati del ciclo in corso non partono.
`GET` /api/runners/<rid>/journal	Le ultime righe del registro (`limit`, di default 300).
`POST` /api/runners/<rid>/kill-switch/reset	Riapre il conto chiuso dal kill-switch (il file del conto legato al runner), a runner fermo.
`POST` /api/live/run_cycle	Un ciclo solo, per le prove da script. Su un broker vero sempre in simulazione.
`POST` /api/live/start	Ciclo continuo per le prove da script, con la stessa regola.
`GET` /api/live/status	Stato, numero di cicli, configurazione e le ultime 60 righe del ciclo di prova.

Parte III

Come si cambia

5

Aggiungere un artefatto

5.1 Non c'è un contratto solo: ce ne sono due

È la prima cosa da sapere prima di scrivere un plugin, e non è scritta da nessuna parte nel codice: le cinque famiglie di artefatti si dividono in due forme, con contratti, meccanismi di scoperta e modi di fallire diversi.

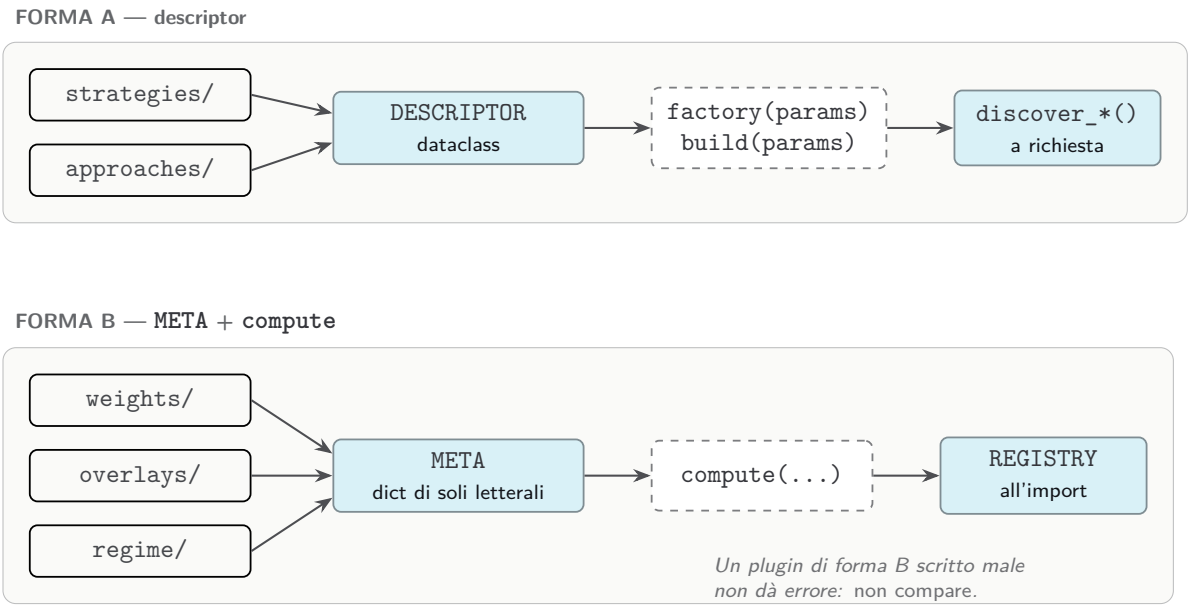


Figura 5.1. Le due forme. Scegliere la famiglia significa scegliere la forma, e con essa il modo in cui l'artefatto può non comparire.

	Famiglie	Oggetto esposto	Punto d'ingresso
Forma A	`strategies/`, `approaches/`	`DESCRIPTOR`, `APPROACH_DESCRIPTOR`	`factory(params)`, `build(params)`
Forma B	`weights/`, `overlays/`, `regime/`	`META` (dict di soli letterali)	`compute(...)`

5.1.1 Come si differenziano i due registri

	Forma A	Forma B
Quando si costruisce	A richiesta: `discover_*()` restituisce un dict nuovo ogni volta	All'import del package, una volta sola
Come trova i file	`importlib`, cartella per cartella	`pkgutil`, più un ripiego sulle cartelle senza `__init__.py`
File atteso	`strategy.py`, `approach.py`	`weight_mode.py`, `overlay.py`, o un modulo `.py`
Se un plugin è rotto	Avviso nel registro, saltato	Avviso, saltato
Se un plugin è *scritto male*	Spesso un errore all'uso	**Invisibile**, senza un rigo da nessuna parte
Per vederne uno nuovo	Basta richiamare `discover_*()`	Serve `reload_registry()`, o un riavvio

Il modo tipico di fallire è l'invisibilità

Vale per tutta la forma B: un `META` che non è fatto di soli letterali, una `key` che non coincide con il nome del file, un `APPROACH_DESCRIPTOR` messo in `overlays/`, un `compute` mancante. Nessuno di questi solleva un'eccezione. Il plugin semplicemente non appare nell'elenco, e non c'è niente da cercare nel registro perché non è successo niente.
Chi scrive un artefatto e non lo vede comparire non ha un bug da trovare: ha un contratto da rileggere. Il § 5.6 elenca i casi uno per uno.

5.1.2 Il nome canonico è il nome della cartella

In entrambe le forme, e vale la pena insistere: il nome con cui il sistema conosce un artefatto è il nome della sua cartella, non quello dichiarato dentro il file. Il registro delle strategie arriva a sovrascriverlo:

Il nome della cartella vince `strategies/registry.py:106`

```
        if descriptor.name != name:
            # il nome CARTELLA vince sempre
            descriptor.name = name
```

Per la forma B la regola è ancora più stretta: la `key` del `META` *deve* coincidere con il nome del file, e per le regole di regime il controllo è esplicito. Negli altri casi il disallineamento non dà errore: l'artefatto compare con un nome e si cerca con un altro.

5.2 Una strategia

5.2.1 Che cosa si crea

La struttura minima strategies/

```
strategies/mia_strategia/
|-- __init__.py       # ri-esporta DESCRIPTOR
|-- strategy.py       # espone DESCRIPTOR
`-- mia_impl.py       # l'implementazione
```

La cartella **non** deve cominciare con `_`: il prefisso è l'interruttore di spegnimento, e il filtro che lo applica è questo.

Il filtro delle cartelle strategies/registry.py:55

```python
def _is_valid_strategy_dir(p: Path) -> bool:
    if not p.is_dir():                                  return False
    if p.name.startswith("_") or p.name.startswith("."): return False
    if p.name in SKIP_DIR_NAMES:                        return False
    if not (p / "strategy.py").exists():                return False
    return True
```

5.2.2 Il livello di esecuzione

Si sottoclassa `BaseAtomicStrategy` da `strategies/base.py`, che offre già `applicable_symbols()`, `get_value()` e `get_last_value_until()`. Tre cose sono obbligatorie:

Il contratto di esecuzione strategies/base.py:55

```python
class AtomicStrategy(ABC):
    @property
    @abstractmethod
    def style(self) -> StrategyStyle: ...

    @property
    @abstractmethod
    def allowed_asset_classes(self) -> set[AssetClass]: ...

    @abstractmethod
    def generate_signals(
        self,
        t_decision: datetime,
        t_execution: datetime,
        indicators_by_symbol: dict[str, PrecomputedIndicators],
        portfolio: Portfolio,
    ) -> list[Signal]:
```

I valori ammessi per `style` vengono da `core.types.StrategyStyle`: `TREND`, `MEAN_REVERSION`, `MOMENTUM`, `BREAKOUT`, `STAT_ARB`, `STAT_ARB_INTRADAY`, `FACTOR`, `VALUE`, `MACRO`, `CARRY`, `VOL`.

I nomi dei parametri fanno parte del contratto

Il motore chiama `generate_signals` per *keyword* (§ 4.2). Rinominare `t_decision` in `t`, o `indicators_by_symbol` in `ind`, fa fermare il backtest alla prima barra con un `TypeError`.

5.2.3 Il livello dei metadati

`StrategyDescriptor` ha sei campi obbligatori e una ventina di opzionali.

I sei obbligatori — strategies/descriptor.py:107

```
    # Required
    name: str
    description: str
    category: StrategyCategory
    timeframe: StrategyTimeframe
    factory: Callable[[dict], Any]
    default_params: dict
```

Un esempio reale, che mostra anche il modo idiomatico di scrivere la `factory`:

Il descrittore di una strategia installata — strategies/momentum/strategy.py:16

```
DESCRIPTOR = StrategyDescriptor(
    name="momentum",
    description="Cross-Sectional Momentum 12-1 (Jegadeesh-Titman). "
                "Top decile, monthly rebalance, long-only.",
    category=StrategyCategory.D1_LONG_ONLY,
    timeframe=StrategyTimeframe.D1,

    excluded_instruments=["EURUSD", "GBPUSD", "USDJPY", "USDCHF", "AUDUSD"],
    required_indicators=["momentum_252_21"],

    factory=lambda params: CrossSectionalMomentumStrategy(**{
        k: v for k, v in params.items() if not k.startswith("exit_")
    }),
    default_params={
        "formation_window": 252,
        "skip_window": 21,
        "top_decile_pct": 0.10,
        "max_positions": 25,
        "rebalance_every_n_days": 21,
        "exit_catastrophic_sl_pct": None,
    },

    min_lookback_days=300,
    backtest_script="strategies/momentum/run_backtest.py",
    output_path_pattern="output/strategies/wf_momentum_oos_equity*.parquet",
    stream_type="equity_curve",
    stream_category="d1",
    exit_policy_type="signal_only",
    regime_weights={"bull": 1.0, "neutral": 1.0, "bear": 0.7},
)
```

Il filtro delle chiavi `exit_`

La `factory` scarta le chiavi che cominciano con `exit_` prima di passarle al costruttore. Quei parametri non appartengono alla strategia: li consuma la politica di uscita, costruita a parte da `exit_policy_type`. Scrivere la `factory` senza quel filtro produce un `TypeError` su un argomento inatteso.

5.2.4 I campi opzionali che contano davvero

Campo	A che serve
`output_path_pattern`, `stream_type`, `stream_category`	**Integrazione con la dashboard, senza toccarla.** Con questi tre campi lo stream compare da solo in **Streams**: non c'è nessun elenco da aggiornare in `dashboard/config.py`. Senza `output_path_pattern` lo stream non compare.
`required_indicators`	Nomi presi dal pool comune. Il registro ne calcola l'unione su tutte le strategie e li pre-calcola una volta sola.
`regime_weights`	Modulatori `bull` / `neutral` / `bear`. Le chiavi devono essere esattamente quelle tre e i valori numerici non negativi: è l'unico controllo del descrittore che solleva `ValueError` invece di un avviso.
`allocator_overrides`	Scavalca per questa sola strategia `max_position_weight`, `max_gross_leverage`, `max_asset_class_gross`, `portfolio_target_vol_annual`.
`extra_timeframes`	Per le strategie multi-timeframe. Il caricamento parte dal più fine.
`required_external_data`	`news`, `fundamentals`, `policy_rates`. Il sistema li *scarica* se mancano.
`required_mms_features`, `required_volume_profile`	Feature specialistiche calcolate su richiesta.
`maintenance`	`retrain_every_days`, `retrain_every_bars`, `retrain_on_drawdown_pct`, `data_refresh_every_days`. È da qui che il sistema sa quando un modello va riaddestrato.
`_module_path`, `_folder_path`	**Li scrive il registro.** Non vanno impostati a mano.

`regime_weights` si dichiara a priori

Il commento nel descrittore è esplicito: modificarli guardando il backtest «sarebbe curve-fitting mascherato da regime adaptation». Si dichiarano sul ragionamento strutturale — questa strategia compra il momentum, quindi in regime ribassista deve pesare meno — e poi si verifica che il modulato non peggiori rispetto al neutro. Non si cercano i valori che massimizzano il risultato.

5.2.5 La procedura

Aggiungere una strategia

1. Creare la cartella **senza** prefisso `_`.
2. `mia_impl.py`: sottoclassare `BaseAtomicStrategy`, implementare `style`, `allowed_asset_classes`, `reset()` e `generate_signals()` *con quei nomi esatti di parametro.*
3. Leggere solo fino a `t_decision`. Mettere `Signal.timestamp = t_execution`.
4. `strategy.py`: `DESCRIPTOR = StrategyDescriptor(...)` con i sei campi obbligatori; `factory` che filtra le chiavi `exit_`.
5. Per vedere lo stream in dashboard: `output_path_pattern` + `stream_type` + `stream_category`.
6. `__init__.py`: ri-esportare `DESCRIPTOR`.
7. Verificare con `tests/diagnostica_strategie.py`, che esiste proprio per rispondere alla domanda «perché `discover_strategies()` torna vuota».

Il riferimento completo è `strategies/GUIDE_STRATEGIE_CUSTOM.md` (2 281 righe), affiancato da sei guide tematiche: il grafo a nodi come alternativa al Python, l'uso dei regimi dentro una strategia, le posizioni multiple sullo stesso simbolo, le feature MMS, il volume profile e un documento di proposta sulle strategie in valuta.

5.3 Un approccio

Un approccio *scavalca il motore*. Non implementa `AtomicStrategy`, non emette `Signal`, non passa per allocatore, rischio ed esecuzione. Costruisce il proprio calcolo e consegna direttamente la serie dei rendimenti.

	Strategia	Approccio
File	`strategy.py`	`approach.py`
Oggetto	`DESCRIPTOR`	`APPROACH_DESCRIPTOR`
Entrata	`factory(params)` → istanza	`build(params)` → `pd.Series`
Chi esegue	il motore di backtest	l'approccio stesso
Produce	segnali, poi posizioni	rendimenti giornalieri
Runner	`execution/run_walkforward.py`	`execution/run_approach.py`

5.3.1 Il contratto

Dichiarato nel docstring del modulo `approaches/descriptor.py:17`

```
CONTRATTO della funzione build:
    build(params: dict) -> pandas.Series
        - index = date (tz-naive); valori = rendimenti DAILY (frazione)
        - usa SOLO dati fino a t per ogni t (niente lookahead)
```

«Tz-naive» non è pignoleria: una serie con fuso orario e una senza non si allineano, e il risultato è un ensemble che perde silenziosamente le osservazioni di uno stream. Per questo gli approcci installati chiudono tutti allo stesso modo:

La chiusura obbligata di build approaches/momentum_ema_approach/approach.py:74

```
    # Assicuriamo che l'indice sia tz-naive come richiesto
    pnl.index = pd.to_datetime(pnl.index).tz_localize(None)
    return pnl
```

5.3.2 Il descrittore

Più corto di quello delle strategie, e con tre campi che si derivano da soli se li si omette:

I default derivati approaches/descriptor.py:79

```
        if not self.label:
            self.label = self.name
        if not self.output_path_pattern:
            self.output_path_pattern = f"output/strategies/{self.name}_pnl_daily.parquet"
        if not self.backtest_script:
            self.backtest_script = f"approaches/{self.name}/run_backtest.py"
```

Due campi hanno invece una lista chiusa di valori ammessi, e un valore fuori lista solleva `ValueError` al caricamento:

Le due allowlist approaches/descriptor.py:31

```
APPROACH_STREAM_TYPES = ("pnl_daily", "pnl_sparse", "equity_curve")
APPROACH_STREAM_CATEGORIES = ("ml", "d1", "intraday", "ml_overlay")
```

Un esempio completo:

Un descrittore di approccio approaches/momentum_ema_approach/approach.py:79

```
APPROACH_DESCRIPTOR = ApproachDescriptor(
    name=NAME,
    label="EMA Momentum Approach",
    description=("Generates weights from the distance between the closing "
                 "price and its exponential moving average (EMA)."),
    build=build,
    default_params={"lookback_ema": 12, "short_lookback": 20},
    stream_type="pnl_daily",
    stream_category="d1",
    tags=["momentum", "ema", "d1"],
)
```

5.3.3 Approcci e overlay: lo stesso registro, due elenchi

Il registro degli approcci serve *due* cartelle, `approaches/` e `overlays/`, e il package si deduce da dove viene la cartella:

Il package dipende dalla provenienza approaches/registry.py:56

```
    # il package dipende dalla cartella di provenienza: approaches/ oppure overlays/
    pkg = approach_dir.parent.name if approach_dir.parent.name in ("approaches", "overlays") else "approaches"
    module_dotted = f"{pkg}.{name}.approach"
```

Di conseguenza ci sono tre funzioni pubbliche con semantiche distinte:

Funzione	Che cosa restituisce, e perché
`discover_approache`	Solo `approaches/`. Sono quelli eseguibili dal pulsante **Run backtest**.
`discover_overlays()`	Solo le cartelle `overlays/*/approach.py`. Non sono eseguibili da sole.
`discover_all()`	L'unione, **solo per risolvere un nome**, mai per costruire un elenco da mostrare.

Perché gli elenchi sono separati

Il razionale è scritto nel registro: mettere gli overlay fra i backtest lanciabili «li faceva comparire fra i backtest lanciabili, dove sarebbero falliti o — peggio — avrebbero prodotto uno stream vuoto senza dirlo». Uno stream vuoto che non si lamenta è peggio di un errore, perché entra nell'ensemble e lo cambia.

Il campo `family` non esiste nella dataclass

`_scansiona` attacca all'oggetto un attributo `family` che non è dichiarato in `ApproachDescriptor` e non compare in `to_dict()`:

approaches/registry.py:113

```
        # Da dove viene: e' l'unico modo affidabile per distinguere un
        # approccio da un overlay, perche' il contratto del file e' identico.
        try:
            desc.family = etichetta
        except Exception:      # dataclass congelata: non e' un problema
            pass
```

Funziona perché la dataclass non è congelata. Chi legge `descriptor.py` non sa che esiste.

Aggiungere un approccio

1. Cartella senza `_`, con `approach.py` e `__init__.py`.
2. `def build(params: dict) -> pd.Series`: indice di date tz-naive, valori di rendimento giornaliero in frazione, causale.
3. `APPROACH_DESCRIPTOR` con almeno `name`, `description`, `build`.
4. Controllare che `stream_type` e `stream_category` stiano nelle due allowlist.
5. Provarlo: `python-mexecution.run_approach<nome>`.

5.4 Una modalità di pesatura, o un overlay

Le due famiglie condividono forma, meccanismo di scoperta e modi di fallire. Cambiano il file atteso e la firma di `compute`.

	File	Firma e significato del ritorno
`weights/`	`weight_mode.py`	`compute(eng, stream_pnls)` `kind="static"` → `dict[str, float]`, somma 1 `kind="rolling"` → `DataFrame` di pesi nel tempo
`overlays/`	`overlay.py`	`compute(eng, ensemble_pnl, stream_pnls)` → `Series` di *moltiplicatori* in $[0, 1]$, allineata a `ensemble_pnl.index`

5.4.1 Due forme di cartella, e perché

Entrambe le famiglie accettano sia un modulo singolo (`weights/<chiave>.py`, forma storica) sia una cartella (`weights/<chiave>/weight_mode.py`, da preferire per i nuovi). La scoperta ha un ripiego che vale la pena conoscere:

Il ripiego sulle cartelle senza `__init__.py` — `weights/__init__.py:26`

```
def _nomi_disponibili():
    """I nomi da provare: moduli, package e CARTELLE SENZA __init__.py.

    `pkgutil.iter_modules` non vede una cartella priva di `__init__.py`. Il
    marketplace pero' esclude dai pacchetti i file che iniziano con "__",
    quindi un artefatto installato arriva proprio cosi': con il solo
    weight_mode.py. Senza questa aggiunta un overlay comprato non si sarebbe
    caricato, e l'unico sintomo sarebbe stato la sua assenza dall'elenco.
    """
```

È il tipo di dettaglio che non si deduce leggendo il meccanismo: il marketplace e il caricatore dei plugin hanno un'interazione, e il caricatore si adatta a come il marketplace impacchetta.

5.4.2 Il registro si costruisce all'import

La costruzione del registro — `weights/__init__.py:73`

```
for _nome in _nomi_disponibili():
    try:
        _m = importlib.import_module(f"{__name__}.{_nome}")
    except Exception as exc:  # un plugin rotto non deve impedire l'avvio
        logger.debug("%s: import diretto fallito (%s)", _nome, exc)
        _m = None
    _m = _risolvi(_m, _nome)
    if _m is not None and hasattr(_m, "META") and hasattr(_m, "compute"):
        _mods.append(_m)

_mods.sort(key=lambda m: (m.META.get("order", 999), m.META["key"]))
```

```
12
13 REGISTRY  = tuple(m.META for m in _mods)
14 COMPUTERS = {m.META["key"]: m.compute for m in _mods}
```

La riga che conta è l'ultima condizione: senza `META` *e* `compute` il modulo viene scartato in silenzio. Non c'è nemmeno un `logger.warning`: l'oggetto semplicemente non entra nel registro.

5.4.3 Il campo `META`

Campo	Obbl.	A che serve
`key`	sì	Identificativo `[a-z0-9_]`, uguale al nome del file o della cartella. È la chiave del dizionario dei `compute`.
`label`	sì	Il nome mostrato nell'interfaccia.
`kind`	solo `weights/`	`"static"` o `"rolling"`. Governa il dispatch.
`group`	no	Raggruppamento nell'interfaccia.
`order`	no	Ordinamento, default 999.
`hint`	no	Una riga di descrizione.
`maintenance`	no	`retrain_every_days`, `data_refresh_every_days`, `notes`.
`required_external_data`	no	`news`, `macro`, `fundamentals`, `rates`.

Un esempio minimo e completo:

La modalità più semplice che esista `weights/equal/weight_mode.py:16`

```
1  META = {
2      "key": 'equal',
3      "label": 'Equal',
4      "kind": 'static',
5      "group": 'Naive',      # nel sorgente: Naive con la dieresi
6      "order": 10,
7      "hint": "Classic 1/N",
8  }
9
10 def compute(eng, stream_pnls: dict[str, pd.Series]) -> dict[str, float]:
11     """Pesi equal-weight 1/N. Output: dict {stream_name: weight}, somma 1.0."""
12     n = len(stream_pnls)
13     if n == 0:
14         return {}
15     return {name: 1.0 / n for name in stream_pnls}
```

E uno che dichiara dati esterni e una scadenza:

Quando l'artefatto ha bisogno di dati freschi overlays/news_pressure/overlay.py:30

```
    # QUALI dati servono e OGNI QUANTO vanno rinfrescati. Il sistema legge di
    # qui per sapere quando chiamare la porta: senza, la scadenza non
    # esisterebbe e in live i dati resterebbero fermi al giorno
    # dell'installazione.
    "required_external_data": ["news"],
    "maintenance": {
        "data_refresh_every_days": 1,
        "notes": "news goes stale in hours: it is refreshed every day",
    },
```

5.4.4 Che cosa si può importare

Il codice di un artefatto di forma B passa da un validatore `AST` quando arriva dall'editor. Le radici ammesse e quelle vietate:

	Moduli
Ammessi ovunque	`numpy`, `pandas`, `math`, `statistics`, `logging`, `typing`
Ammessi in `weights/`	più `scipy`, `sklearn`, `lightgbm`, `from ..metrics import ...`
Ammessi in `overlays/`	più `approaches`, `core`, `reporting`
Vietati	`os`, `sys`, `subprocess`, `socket`, `requests`, `open`, `eval`, `exec`, `__import__`, `importlib`

Perché un validatore e non la fiducia

Un artefatto del marketplace è codice di qualcun altro che gira sulla macchina di chi lo installa, con i suoi dati e le sue credenziali del broker a portata di `open()`. Il validatore non rende sicuro l'ineseguibile: un calcolo malizioso scritto con solo `numpy` resta possibile. Toglie però la via diretta — leggere un file, aprire una connessione, eseguire una stringa — e rende il resto visibile a chi guarda il sorgente, che il marketplace mostra sempre.

5.4.5 Le regole dei pesi

Oltre al contratto di firma, una modalità di pesatura deve rispettare quattro condizioni, che il motore *non* verifica:

- pesi **solo lunghi**;
- limitati all'intervallo `[eng.min_weight, eng.max_weight]`;
- normalizzati a somma 1;
- `{}` se non calcolabili — l'ensemble ha un ripiego, ma solo se glielo si dice restituendo il vuoto. Restituire pesi sbagliati non attiva nessun ripiego.

5.4.6 Le regole degli overlay

- il ritorno è una serie di *moltiplicatori*, non di rendimenti;
- causale: `shift(1)` sulla decisione — «la decisione di oggi non può usare il rendimento di oggi»;

- **non** esporre `APPROACH_DESCRIPTOR`. Il registro cerca `META` e `compute`; non trovandoli, l'artefatto resta invisibile senza alcun errore.

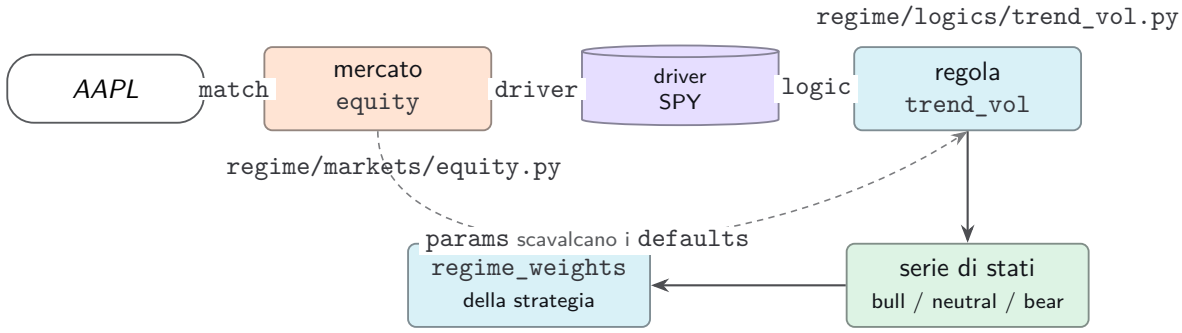
Aggiungere una modalità di pesatura o un overlay

1. `weights/mia_chiave/weight_mode.py` (o `overlays/mia_chiave/overlay.py`), con `__init__.py` che può restare vuoto.
2. `META` con `key` uguale al nome della cartella.
3. `compute` con la firma giusta per la famiglia.
4. Se il processo è già avviato: `from weights import reload_registry; reload_registry()`.

I modelli di riferimento sono `weights/WEIGHT_MODE_TEMPLATE.md` e `overlays/OVERLAY_TEMPLATE.md`; quest'ultimo si apre con una tabella che separa overlay e approccio su sei assi, e la frase è «*An overlay and an approach are two different constructs. This is not a nuance.*»

5.5 Un regime: due file, in quest'ordine

`regime/` è l'unica famiglia divisa in due artefatti distinti, ed è la divisione giusta: *una regola serve più mercati.* Azioni e cripto usano entrambe `trend_vol`, con parametri diversi.



Una regola serve più mercati: azioni e cripto usano entrambe `trend_vol`*, con parametri diversi. Si scrive prima la regola: un mercato che ne nomina una inesistente viene rifiutato.*

Figura 5.2. Dal simbolo allo stato. I due riquadri con il file sotto sono le due cose che si scrivono.

5.5.1 La regola

Il contratto della regola `regime/logics/__init__.py:7`

```
META = {
    "key": "mia_regola",
    "label": "La mia regola",
    "description": "a cosa serve",
    "defaults": {"finestra": 60},
}

def compute(close: pd.Series, p: dict) -> pd.Series:
    ...   # Series di 'bull' / 'neutral' / 'bear'

```

```
Due vincoli non negoziabili:

1. **Causalita'**: il valore al tempo t usa solo dati fino a t. Niente
   statistiche sull'intero campione (quantili full-sample = look-ahead).
2. **Etichette**: sempre ``bull``/``neutral``/``bear``, cosi' i
   ``regime_weights`` gia' dichiarati dalle strategie continuano a valere.
```

Gli helper causali condivisi stanno in `regime/logics/_base.py`: `_realized_vol`, `_expanding_median`, `_hysteresis`. Vanno usati: riscrivere un quantile a mano è il modo più comune di introdurre una statistica full-sample senza volerlo.

Le tre regole installate:

Chiave	Che cosa misura	Che cosa significano le etichette
`trend_vol`	Posizione rispetto alla media mobile lunga, corretta per volatilità realizzata	bull = sopra il trend e volatilità accettabile; bear = sotto il trend, o volatilità alta
`vol_state`	Volatilità realizzata contro i quantili espansivi della propria storia	bull = calmo; bear = sotto stress. Per attività senza deriva
`trend_range`	Rapporto di varianza sui rendimenti: la persistenza, non la direzione	bull = in tendenza; bear = laterale. Indipendente dalla deriva

5.5.2 Il mercato

Il contratto del mercato — `regime/markets/__init__.py:12`

```
    META = {
        "key": "equity",
        "label": "Azioni",
        "logic": "trend_vol",          # regola ereditata da regime/logics.py
        "driver": ["SPY"],             # serie che GUIDA il regime (in ordine)
        "match": [r"^[A-Z]{1,5}$"],    # simboli assegnati a questo mercato
        "priority": 50,                # match piu' bassa = valutata prima
        "enabled": True,
        "params": {"trend_window": 200},   # override sui default della logica
    }
```

Un esempio reale, che mostra come si usa `priority` e che cosa significa `"self"` fra i driver:

Il mercato valutato per primo — `regime/markets/forex.py:17`

```
META = {
    "key": "forex",
    "label": "Forex",
    "logic": "vol_state",
    "driver": ["DXY", "self"],
    "match": [r"^(EUR|USD|GBP|JPY|CHF|AUD|NZD|CAD|SEK|NOK)"
              r"(EUR|USD|GBP|JPY|CHF|AUD|NZD|CAD|SEK|NOK)$",
              r"^[A-Z]{3}/[A-Z]{3}$",
```

```
            r"^DXY$"],
    "priority": 10,          # valutato per primo: pattern molto specifico
    "enabled": True,
    "params": {
        "vol_window": 21,
        "min_obs": 252,
        # banda ampia + isteresi lunga: un regime deve essere uno STATO, non
        # una reazione. Valori strutturali, non tarati sui dati.
        "calm_quantile": 0.30,      # sotto questo quantile: risk-on
        "stress_quantile": 0.80,    # sopra questo quantile: risk-off
        "hysteresis_days": 10,
    },
}
```

`driver` è una lista di ripieghi

Si valutano in ordine: il primo disponibile vince. `"self"` significa «il simbolo stesso». Se nessuno dei driver è disponibile il mercato non modula — cioè si comporta come neutro — e lo dichiara in un avviso nel registro, non in un errore. Un mercato che non modula non si distingue da un mercato che modula con pesi 1, se non leggendo il registro.

5.5.3 `META` di soli letterali: qui è un vincolo vero

Per le regole di regime il sistema legge il `META` con `ast.literal_eval`, cioè **senza eseguire il file**. La conseguenza è netta: un valore come `"x": np.pi` non è un letterale, la lettura fallisce, e la regola diventa invisibile.

La `key` deve coincidere con il nome del file

Per le regole di regime il controllo è esplicito ed è l'unico caso in cui il disallineamento viene rifiutato. Negli altri casi non lo è, e il plugin compare con un nome e si cerca con un altro.

5.5.4 Il merge dei parametri

I `params` dichiarati dal mercato scavalcano i `defaults` dichiarati dalla regola, e il punto in cui accade è uno solo:

Dove i due artefatti si incontrano `regime/logics/__init__.py:89`

```
def apply(logic_key: str, close, params: dict | None = None):
    if logic_key not in REGISTRY:
        raise KeyError(f"unknown regime rule: '{logic_key}' "
                       f"(available: {sorted(REGISTRY)})")
    entry = REGISTRY[logic_key]
    p = dict(entry["defaults"])
    p.update(params or {})
    return entry["fn"](close, p)
```

Aggiungere un regime

1. `regime/logics/mia_regola.py`: `META` di soli letterali con `key` uguale al nome del file, e `compute(close, p)` che restituisce `'bull'` / `'neutral'` / `'bear'` allineati a `close`, causali, con isteresi.
2. `regime/markets/mio_mercato.py`: `META` con almeno `key` e `logic`; la regola nominata deve *già* esistere.
3. Scaricare le serie dei driver: `pythonscripts/download_regime_drivers.py`.
4. Scorciatoia: `regime.logics.scaffold(key)` genera lo scheletro corretto, ed è quello che usa l'editor dell'interfaccia.

Le guide sono `regime/GUIDE_REGIMI.md` («come costruirne uno») e `strategies/GUIDE_REGIME.md` («come usarlo dentro una strategia»): sono due documenti distinti e si rimandano a vicenda.

5.6 «L'ho scritto e non compare»

Questa sezione è un elenco di cause, ordinate per frequenza. Serve quando un artefatto è stato scritto, il file è al suo posto, e nell'interfaccia non c'è.

Causa	Come si riconosce
Il registro è vecchio	Il processo era già avviato. Il registro di forma B si costruisce all'import: chiamare `reload_registry()`, o riavviare.
Il nome della cartella comincia con `_`	È l'interruttore di spegnimento. Si vede guardando, ma è anche la cosa che non si guarda.
Manca il file atteso	`strategy.py`, `approach.py`, `weight_mode.py`, `overlay.py`. Il filtro delle cartelle lo richiede per nome.
Manca `META` o `compute`	La condizione di ammissione al registro li richiede entrambi. Senza, il modulo viene scartato senza una riga di registro.
`META` non è fatto di soli letterali	Vale per i regimi, letti con `ast.literal_eval`: un `np.pi` dentro i `defaults` rende la regola invisibile.
La `key` non coincide col nome del file	Rifiutata per i regimi; altrove l'artefatto compare con un nome diverso da quello che si sta cercando.
In `overlays/` si è esposto `APPROACH_DESCRIPTOR`	Il registro degli overlay cerca `META` + `compute`. Quel file è un *approccio*, e come overlay non esiste.
`stream_type` fuori allowlist	Per gli approcci è l'unico caso che dà un `ValueError` chiaro al caricamento.
`output_path_pattern` vuoto	L'artefatto c'è, ma lo *stream* non compare in **Streams**: la dashboard salta i descrittori senza pattern.
Un altro modulo `ui_ext_*` è rotto	Se l'artefatto si vede ma manca il pannello che lo mostra, è questo (§ 3.3.2).

5.6.1 Il nodo della ricarica

Il registro di forma B si costruisce una volta, all'import del package. Un artefatto scritto a runtime — dallo Studio, dall'agente, dal marketplace — esiste su disco e non compare. `reload_registry()` risolve, ma deve fare due cose che non sono ovvie:

Le due accortezze `overlays/__init__.py:144`

```
    Due accortezze necessarie perche' la ricarica sia davvero visibile:
    - COMPUTERS viene aggiornato SUL POSTO: `reporting.ensemble` ne tiene un
      riferimento preso all'import, e rilegarlo qui non lo aggiornerebbe;
    - EnsembleEngine.OVERLAY_REGISTRY e' un attributo di classe legato alla
      creazione della classe: va riassegnato esplicitamente.
```

Per le modalità di pesatura c'è un terzo punto, ed è quello che produce il sintomo più sconcertante:

Il terzo oggetto da aggiornare `weights/__init__.py:117`

```
    try:                        # l'attributo di classe e' legato alla creazione
        from reporting.ensemble import EnsembleEngine
        EnsembleEngine.WEIGHTING_MODE_REGISTRY = REGISTRY
        # WEIGHTING_MODES e' una tupla DERIVATA dal registro alla creazione
        # della classe ed e' quella usata per validare le richieste: senza
        # aggiornarla, un weight mode nuovo compare nell'elenco ma il compute
        # risponde 400 "Invalid weighting_mode".
        EnsembleEngine.WEIGHTING_MODES = tuple(m["key"] for m in REGISTRY)
```

> **Compare nell'elenco e risponde `400`**
>
> È il sintomo di una ricarica parziale: l'elenco che l'interfaccia mostra viene da `WEIGHTING_MODE_REGISTRY`, la validazione della richiesta da `WEIGHTING_MODES`. Aggiornare il primo e non il secondo produce una voce cliccabile che risponde «modalità non valida». Chi aggiunge un registro nuovo di forma B deve chiedersi quali altri oggetti ne derivino una copia al momento della creazione della classe.

5.6.2 Due package che sono solo rinvii

`reporting/weighting/` e `reporting/gates/` non contengono logica: ri-esportano `weights/` e `overlays/`. Sono il ricordo di dove quelle due famiglie stavano prima. Non vanno estesi, e il codice nuovo importa dalle cartelle vere.

5.6.3 Un modulo morto da non seguire

Accanto al package `regime/logics/` esiste un file `regime/logics.py`, 237 righe, con una bella tabella della semantica delle etichette nel docstring. **Non è raggiungibile**: a parità di nome il package vince sempre, e `regime.logics` risolve a `regime/logics/__init__.py`. Chi lo modifica non cambia niente e non se ne accorge.

5.6.4 Due incoerenze nelle mappe dell'interfaccia

- `dashboard/studio_browser.py` dichiara `overlay` e `weightmode` come artefatti «a file», mentre tutti e ventuno quelli installati sono a cartella.

- `dashboard/agent.py` ha la stessa incoerenza al suo interno, e i suoi commenti la annotano già: «*Qui indicava un file singolo, e altrove* `approach.py`*: tre posti, tre risposte diverse alla stessa domanda*».

Non rompono niente oggi, perché la scoperta accetta entrambe le forme. Vanno saputе quando si aggiunge una famiglia: la mappa canonica da seguire è `_FILE_ZONA` in `dashboard/agent.py`.

Il file principale atteso per famiglia `dashboard/agent.py:66`

```
_FILE_ZONA = {"strategies": "strategy.py", "approaches": "approach.py",
              "overlays": "overlay.py", "weights": "weight_mode.py",
              "metrics": "metric.py", "indicators": "indicator.py"}
```

5.6.5 Un elenco obsoleto

`strategies/__init__.py` elenca nel docstring «momentum, quality_lv, value, custom_example». Delle quattro, due sono oggi spente (`_value` , `_custom_example`) e mancano le altre sette attive. È documentazione nel codice che ha smesso di essere vera, e il modo in cui ci si accorge è cercarvi una strategia che esiste e non trovarla.

5.7 Una metrica: i dati del sistema e la pagina dello stream

Una metrica è il più piccolo degli artefatti: un file `metrics/<nome>.py` con un `META` di soli letterali e, sotto, codice vettoriale sull'equity della pagina. Produce un numero (`result(...)`) o uno o più grafici (`plot` , `bars` , `heatmap` , `chart`). La si scrive in Studio, che il `META` lo compone da sé con i campi **Output** e **Chart type**; il contratto completo, con gli esempi, è in `metrics/METRIC_TEMPLATE.md`.

Gira in `dashboard/metric_runtime.py`, in una sandbox: builtins ridotti, niente `import` , niente attributi `__dunder__` né privati. Fino a questa versione vedeva soltanto l'equity della pagina e, quando la pagina lo passava, il benchmark. Ora legge anche i dati del sistema.

5.7.1 L'oggetto `data`

Nel codice di una metrica c'è `data` , istanza di `Data` (`dashboard/metric_data.py`). Ogni cartella di `data/` è una categoria e ogni categoria è una classe, tutte con gli stessi metodi:

Metodo	Che cosa restituisce
`names()`	i file della categoria: simboli per le categorie 1:1, file condivisi per le 1:N (`settore_tech` , `USD`)
`timeframes(nome)`	le sottocartelle di timeframe presenti, dalla più fine
`columns(nome)`	i campi veri di un file
`get(nome)`	il file com'è sul disco (con `start` ed `end` se ne ritaglia un periodo)
`series(nome, campo)`	un campo, allineato alle date della metrica (`align=False` lo dà grezzo)

Tranne `timeframes` , accettano tutti `timeframe=` : senza, vale quello predefinito (sotto). Le categorie del sistema hanno una classe propria: `Assets` (con `close` , `returns` , `ohlcv`), `Earnings`

(con `events`), `Fundamentals`, `Rates`, `Macro` (anche i campi del driver macro), `News` (titoli al giorno), `MMS` (file `<SIMBOLO>_<TF>`). Quelle create dalla pagina Dati sono `Category`.

Una metrica che legge un asset — metrics/beta_spy.py

```
META = {"label": "Beta vs SPY", "type": "number", "chart_kind": None, "active": True}

r_m = data.assets.returns("SPY", timeframe="D1")   # aligned to the equity dates
both = returns.notna() & r_m.notna()
result(returns[both].cov(r_m[both]) / r_m[both].var(), "Beta vs SPY", fmt="ratio")
```

Tre regole fanno il comportamento.

Il timeframe predefinito
È quello della strategia dietro la pagina, letto dal `DESCRIPTOR` per albero sintattico, senza importare la strategia (`strategy_timeframes`). Per una strategia su più timeframe vale il più basso, cioè il più fine; per approcci e pagine senza strategia D1; in un ensemble il più basso fra le strategie dei suoi stream. Il codice ne sceglie un altro con `timeframe="H1"`.

Il timeframe che manca
Se su disco non c'è, lo si costruisce da uno più fine che lo divide esattamente (H4 da H1 prima che da M5, D1 da M5), con le regole OHLCV del motore. Se non si può, `DataError` elenca i timeframe che ci sono e propone `timeframe=...`.

L'allineamento
`series`, `close` e `returns` sono allineate alle date della metrica con la regola del motore: alla data t l'ultimo valore noto a t, mai uno successivo. I dati intraday prima si chiudono al giorno, così la data t riceve la chiusura di t.

Un dato che manca non è un errore del codice

`DataError` (sottoclasse di `LookupError`) arriva alla pagina come «Data not available», non come errore. Studio salva comunque la metrica, con un avviso: funzionerà quando il dato sarà caricato. La prova che precede il salvataggio gira sullo stream con più dati, non su un'equity finta: una metrica che legge `data` si prova davvero.

5.7.2 Grafici a più serie

Con `chart_kind: "figure"` un grafico ha più serie, punti con etichetta, dispersioni, barre, aree e linee di riferimento, su un asse x di date oppure di numeri — quello che serve per la Security Market Line del CAPM, dove sull'asse x c'è il beta. Il runtime produce una specifica JSON (`kind: "figure"`, `traces`); `plotCustomFigure` di `static/js/charts.js` la disegna con Plotly.

Un grafico con l'asse x numerico

```
fig = chart("CAPM - Security Market Line", x="Beta", y="Annual return", y_format="pct")
fig.line([0.0, 2.0], [rf, rf + 2.0 * premium], name="SML", dash="dash")
fig.point(1.0, market, name="S&P 500")
fig.point(beta, holding, name="Holding", label="beta %.2f" % beta)
```

La label del valore è quella dei grafici predefiniti: una sola, unificata sulla x (`hovermode: "x unified"`), con il valore di ogni serie.

Perché le linee radi si riempiono di punti

Con la label unificata Plotly cerca, per ogni serie, il punto più vicino sulla x entro 20 pixel. La SML qui sopra ha due vertici: fra l'uno e l'altro la label non troverebbe nulla e sparirebbe. Prima le figure su un asse di numeri usavano `closest`, ed era peggio: la label saltava dal punto alla retta e ritorno, e a vederla lampeggiava. Ora `_densify` di `charts.js` aggiunge punti intermedi *sul tracciato* delle linee (interpolazione fra vertici consecutivi, i buchi restano buchi) finché nessun tratto copre più di 1/500 dell'asse x: la linea disegnata è la stessa, e la label trova un valore in ogni suo punto. Le figure di soli punti tengono `closest`: lì la label giusta è quella del punto sotto il puntatore.

5.7.3 La pagina di uno stream

Ogni metrica della pagina, predefinita o personale, numerica o grafica, si nasconde e si rimette. Le predefinite sono gli elementi con `data-sv` in `templates/stream.html`; le personali le dispone `static/js/stream_metrics.js`: i numeri dopo quelli predefiniti, i grafici in fondo alla pagina, dopo grafici e tabelle predefiniti, con **New chart metric** per ultima.

Pezzo	Che cosa fa
`/api/stream-view/prefs`	la scelta, una per tutti gli stream: `shown` (le chiavi accese) e `known` (quelle esistenti quando la scelta è stata fatta). `shown: null` = predefinite più personali attive
`sv_off(chiave)`	nel template: le predefinite nascoste partono nascoste, senza comparire per un istante
`/api/metrics/compute`	con `stream`, `ids` e `include_inactive`: calcola solo le personali accese, sui risultati dello stream e con il suo benchmark
`/studio?tab=metric`	con `output=number` o `output=chart` (**New number metric**, **New chart metric**): l'uscita scelta, un nome libero e, con `from=`, il ritorno allo stream; con `metric=<id>` riapre una metrica. **Save** valida e salva in un colpo solo

Perché `known` oltre a `shown`

Una metrica creata dopo la scelta — per esempio con **New chart metric** dalla pagina stessa — non è stata scartata da nessuno. Senza `known` resterebbe nascosta finché qualcuno non va a cercarla nell'elenco; con `known` compare da sola, se è attiva.

I riquadri come finestre — Grafici e tabelle, predefiniti e personali, stanno in `#sv-grid`: righe di al massimo tre riquadri visibili, che l'utente sposta dalla testata, allunga dal bordo basso, allarga col divisore fra due riquadri e massimizza col doppio clic sulla testata. La disposizione sta nello stesso file della scelta, sotto `layout`, una per tutti gli stream:

La disposizione salvata

```
"layout": [ {"h": 420, "items": ["chart.equity", "metric:capm#0"],
             "w": {"chart.equity": 1.4, "metric:capm#0": 0.6}},
            {"h": null, "items": ["table.stress"]} ]
```

Campo	Che cosa vuol dire
`items`	le chiavi dei riquadri nell'ordine della riga: `chart.*` e `table.*` sono le `data-sv` predefinite; `metric:<id>#<n>` è l'n-esimo grafico di una metrica personale (una metrica che ne disegna quattro fa quattro riquadri)
`h`	altezza dell'area del grafico o della tabella, in px (160–1600), uguale per tutta la riga; `null` = normale
`w`	larghezze relative dei riquadri affiancati (0,2–5); assente = uguali

`POST /api/stream-view/prefs` aggiorna solo ciò che riceve: `{shown, known}` la scelta, `{layout}` la disposizione (`null` = **Reset layout**). Il server (`_clean_layout` in `app.py`) controlla solo la forma; il resto lo fa la pagina, che sa quali riquadri si vedono.

Perché una riga può avere più di tre chiavi

Un riquadro nascosto, o il grafico di una metrica che non è ancora stato calcolato, tiene il suo posto nella riga e lo ritrova quando compare. Il limite è di tre riquadri *visibili*: se ricompare un quarto, la riga va a capo. Un riquadro che la disposizione non conosce (una metrica nuova, un grafico predefinito arrivato con un aggiornamento) va in fondo, in una riga sua.

Durante lo spostamento il grafico non si muove

Al suo posto resta il contorno; sotto il puntatore va una copia ferma (`cloneNode`), e il grafico vero si sposta una volta sola, al rilascio, con `insertBefore`: Plotly non si ridisegna. Il contenuto del riquadro lasciato si nasconde con l'opacità, non con `visibility`: i `clipPath` del grafico, a cui punta anche la copia, smetterebbero di ritagliare. I grafici seguono la nuova misura con `Plotly.relayout(gd, {autosize: true})`, solo se è cambiata: niente `ResizeObserver`, che faceva lampeggiare la label del valore. Mentre si trascina, i risultati delle metriche personali che arrivano aspettano la fine (`Grid.whenIdle`).

5.7.4 La guardia: numpy e pandas filtrati

Il codice di una metrica o di un indicatore può arrivare dalla Community. numpy e pandas gli arrivavano così com'erano, e da un loro sottomodulo si raggiungeva il sistema operativo senza scrivere un import (`pd.core.config_init.os`); `pd.read_pickle` di un indirizzo eseguiva codice arrivato dalla rete. `dashboard/sandbox_guard.py` li passa filtrati: funzioni e classi restano quelle vere, i sottomoduli solo quelli di calcolo (`np.linalg`, `np.random`, `pd.offsets`, ...); lettura e scrittura di file, `eval`, `query` e `plot` sono rifiutati già nel sorgente, anche chiamati per nome (`agg("to_parquet")`). I dati del sistema si leggono con `data`.

Uno strato, non un isolamento

Come il validatore AST delle strategie (`adapters/ast_validator.py`), la guardia lavora nel processo dell'applicazione: chiude gli abusi diretti, non un attaccante che costruisce i nomi a pezzi. La barriera vera resta l'isolamento di processo. Il validatore delle strategie ha la stessa apertura su numpy e pandas: una strategia può arrivare a `os` per la stessa strada.

6

L'interfaccia

6.1 Anatomia di `dashboard/`

Trentasei file Python, 35 template Jinja, un centinaio di file JavaScript e CSS. È l'oggetto più grande del progetto, e il modo più rapido per orientarsi è guardare le dimensioni: dicono dove sta il peso.

File	KB	Di che cosa si occupa
`app.py`	226	Il montaggio e 137 rotte.
`agent.py`	132	L'assistente che scrive artefatti.
`community.py`	132	Marketplace, contestazioni, account, messaggi, abbonamento.
`live_runners.py`	90	I runner live: avvio, conto legato, kill-switch del conto, registro.
`web_db.py`	90	L'accesso al Postgres del sito.
`agent_tools_sistema.py`	80	Gli strumenti con cui l'agente agisce sul sistema vivo.
`auth.py`	71	Accesso, registrazione, guardia globale, pagine HTML in linea.
`autopubblica.py`	49	Pubblicazione automatica degli artefatti, in un thread.
`genetic.py`	49	Strategie per programmazione genetica.
`visual_debug.py`	45	Il debug visivo sui grafici.
`studio.py`	42	Editor, grafo a nodi, generazione.

6.1.1 Le quattro generazioni di codice, tutte vive

Guardando come una rotta è registrata si capisce quando è stata scritta, e quale stile seguire per estenderla.

Generazione	Dove	Come si riconosce
1 — tutto in `app.py`	`@app.route` dentro `create_app`	137 rotte in un file di 226 KB.
2 — blueprint	`studio`, `community`, `agent`, `genetic`	`Blueprint(...)`, nessun `url_prefix`.
3 — estensioni v2	`ui_ext_*.py`	`register(app, data_layer, base_path)`, caricate da sole.
4 — il guscio	`api/` + `application/`	Non è nella dashboard: è l'altra applicazione (§ 3.2).

Quale stile usare oggi

Per una rotta nuova che serve a una pagina: un modulo `ui_ext_*.py`. Per una capacità nuova del sistema: un caso d'uso nel guscio, esposto dall'API, chiamato dalla dashboard. In nessun caso si aggiunge una rotta a `app.py`: è il file che tutte e tre le generazioni successive sono servite a non far crescere.

6.1.2 I template

Trentadue pagine, tutte figlie di `base.html`, più tre frammenti inclusi e `serve_account.html`, che non è servito da nessuna rotta ma dalla guardia di autenticazione.

Le pagine di accesso sono l'eccezione: `auth.py` non usa i template, ma HTML in linea renderizzato con `render_template_string` (`_LOGIN_HTML`, `_REGISTER_HTML`, `_VERIFY_HTML`, `_RESET_HTML`, `_ATTESA_HTML`). Chi cerca la pagina di login in `templates/` non la trova.

Dove sta l'interfaccia del Live

`live.html` non è servito da `/live`: è servito da `/studio/runner-live`, cioè dal blueprint `studio`. È il nome di percorso più fuorviante del progetto (§ 3.3).

6.2 Come i dati arrivano alle pagine

La dashboard non legge i `parquet` a ogni richiesta. All'avvio li carica tutti in memoria, in un oggetto che viene poi passato a tutto il resto:

L'interfaccia pubblica — `dashboard/data_layer.py`

```
DataLayer()
    .startup()                  - carica streams + computa ensembles
    .get_ensemble(name)         - 'core' | 'full'
    .get_stream(name)           - stream singolo
    .get_stream_names()         - lista
    .get_spy_benchmark()        - SPY returns
```

È il secondo argomento che ogni modulo `ui_ext_*` riceve in `register(app, data_layer, base_path)`, e il motivo per cui una pagina nuova non deve aprire file.

6.2.1 Che cosa è pigro e che cosa no

Gli stream si caricano all'avvio; due serie costose no.

Due eccezioni dichiarate dashboard/data_layer.py:86

```
        # Stress di mercato (iso_stress) e risk-off macro (macro_regime): caricati o
        # CALCOLATI ON-DEMAND e cacheati (lazy), solo quando l'overlay corrispondente e'
        # attivo -> vedi _get_market_stress / _get_macro_riskoff. Nessun calcolo a startup
    .
```

Il criterio: una serie che serve solo a un overlay si calcola quando quell'overlay è attivo. Caricarla all'avvio allungherebbe la partenza per tutti, a beneficio di chi usa una funzione fra le sette.

6.2.2 Gli ensemble preconfigurati non esistono più

La decisione, scritta nel codice dashboard/data_layer.py:100

```
        # Ensemble pre-configurati DISABILITATI: il sistema usa solo l'ensemble
        # custom (pagina /ensemble_custom), costruito a runtime dagli stream
        # selezionati. CORE/FULL rimossi su richiesta. Set vuoto => nessuna
        # scheda ENSEMBLE fissa in nav/home (restano stream singoli + custom).
        self.configured_ensemble_names: set[str] = set()
```

Il metodo `get_ensemble(name)` esiste ancora e il suo docstring parla ancora di `'core'` e `'full'` : sono nomi che non si possono più ottenere. Chi cerca di capire perché la pagina `/ensemble/core` dà `404` lo trova qui.

6.2.3 Come uno stream entra nell'elenco

`dashboard/config.py` costruisce l'elenco degli stream in tre passi, e solo il primo è un elenco scritto a mano:

1. Gli stream dichiarati esplicitamente nella mappa interna.
2. **Le strategie scoperte**, tramite `discover_strategies()`: entra chi ha un `output_path_pattern`; chi non ce l'ha viene saltato con un `continue` e un commento che dice «strategia non self-contained per dashboard».
3. **Gli approcci scoperti**, con lo stesso meccanismo.

Perché il passo 3 usa `discover_all()` e non `discover_approaches()`

Il commento lo spiega: «anche gli overlay producono uno stream, e uno stream va elencato indipendentemente da chi lo ha generato. Escluderli qui li farebbe sparire dalle composizioni, che è l'unico posto in cui servono davvero». È il complemento della scelta fatta nel registro (§ 5.3): gli overlay restano *fuori* dall'elenco dei backtest lanciabili e *dentro* l'elenco degli stream componibili. Due elenchi, due domande diverse.

Il fallimento della scoperta è un avviso

Entrambi i passi di scoperta sono dentro un `try/except` che registra un avviso e prosegue. Un errore di import in una strategia non impedisce alla dashboard di partire: fa sparire

quella strategia dall'elenco, e lascia una riga `Self-contained stream discovery failed` nel registro. È lo stesso schema delle estensioni v2, con la stessa conseguenza: si nota guardando l'interfaccia, non guardando un errore.

6.3 Il lavoro lungo: job e registro in diretta

Un backtest dura minuti o ore. Una richiesta HTTP non può aspettarlo, quindi tutto ciò che è lungo passa dal `JobManager` di `execution/job_manager.py`: la rotta restituisce subito un identificativo, il lavoro continua in un thread, e il registro arriva al browser man mano che viene prodotto.

Rotta	Che cosa fa
`GET /api/jobs`	Elenco.
`GET /api/job/<id>`	Stato di uno.
`GET /api/job/<id>/stream`	**SSE**: le righe di registro, in diretta.
`POST /api/job/<id>/cancel`	Annulla.
`DELETE /api/job/<id>`	Toglie dall'elenco un job concluso.
`DELETE /api/jobs/conclusi`	Svuota tutti i conclusi.

6.3.1 Gli stati

`PENDING` → `RUNNING` → `SUCCESS`, `FAILED` o `CANCELLED`. Sono un `str, Enum`, quindi il valore serializzato in JSON è direttamente la stringa minuscola.

6.3.2 Il registro in diretta

Lo stream SSE — `dashboard/app.py:3720`

```
    def api_job_stream(job_id):
        """Server-Sent Events stream of log lines."""
        job = job_manager.get(job_id)
        if job is None:
            return jsonify({"error": "Job not found"}), 404

        def event_stream():
            # Manda subito le log lines gia' esistenti
            existing = list(job.log_lines)
            for line in existing:
                yield f"data: {line}\n\n"

            # Subscribe via queue per nuove lines
            q = Queue(maxsize=10000)
            def listener(line):
```

```
                try:
                    q.put_nowait(line)
                except Exception:
                    pass
            unsub = job.subscribe(listener)
```

Due dettagli meritano di essere notati, perché sono la ragione per cui questo codice regge:

Prima lo storico, poi la sottoscrizione
Chi apre la pagina a metà di un backtest vede subito tutto ciò che è già successo, e poi il resto. Sottoscrivere senza mandare lo storico mostrerebbe una finestra vuota fino alla riga successiva.

La coda è limitata e il `put` non blocca
`Queue(maxsize=10000)` con `put_nowait` dentro un `try/except` che ignora il fallimento. Se un browser smette di leggere, la sua coda si riempie e le righe nuove vengono scartate — ma il backtest *non* rallenta. L'alternativa (bloccare sulla coda piena) farebbe sì che una scheda lasciata aperta fermi un calcolo.

Il registro è limitato a 5 000 righe

`deque(maxlen=5000)`: un backtest lungo ne produce molte di più, e le prime scorrono via. Per conservarle tutte si guarda il file di registro, non l'interfaccia.

6.3.3 Un lavoro per volta

Il `JobManager` nasce con `max_workers=1`. È il default dichiarato del progetto, e vale sia per l'API FastAPI sia per la dashboard. Due backtest in parallelo si contenderebbero la stessa cache dei dati e gli stessi file di uscita.

Alzare `max_workers` non è una ottimizzazione

La serializzazione non è un limite di prestazioni da togliere: è ciò che rende i risultati riproducibili. Due job concorrenti che scrivono sotto `output/strategies/` possono produrre uno stream mezzo di un calcolo e mezzo dell'altro, e il risultato non dà errore — è un `parquet` valido con dentro numeri di due esecuzioni diverse.

6.3.4 Le notifiche

`GET /api/notifications` alimenta la campanella, e mette insieme due sorgenti: i job reali del `JobManager` e i messaggi che arrivano dal sito. È il motivo per cui la campanella mostra sia «backtest finito» sia «hai una risposta su una contestazione»: una rotta, due origini.

6.4 Aggiungere una pagina o una rotta

La procedura

1. Creare `dashboard/ui_ext_<lettera>.py`. La lettera decide l'ordine di caricamento; `c` e `d` sono libere.
2. Esporre `register(app, data_layer, base_path)` — *quella firma esatta*: è il contratto che il caricatore si aspetta.

3. Dentro, registrare le rotte con `@app.route`.
4. Se serve una pagina, aggiungere il template in `dashboard/templates/` con `{% extends "base.html" %}`. Il suo foglio di stile va nel blocco `page_css`, che finisce nel `<head>`, non nel contenuto. Il contenuto si mostra quando i suoi dati sono arrivati (§ 6.4.1).
5. Se la rotta tocca l'account, metterla sotto uno dei prefissi di `_SERVE_ACCOUNT` (§ 3.4).
6. Riavviare. Non c'è nessun elenco da aggiornare.

6.4.1 Il contenuto si mostra quando è pronto

Ogni pagina è un documento nuovo, e prima il browser ne mostrava ogni passo: il guscio senza i suoi fogli di stile, poi il contenuto vuoto, poi i dati, tre immagini in pochi centesimi di secondo. Ora il cambio di pagina segue due regole, scritte in `static/js/pagina_pronta.js` (il primo script del `<head>` di `base.html`) e nelle regole CSS accanto a lui.

Il guscio, cioè barra di sistema e barra laterale, non si nasconde mai: si disegna dal primo fotogramma già completo, in qualunque momento arrivi, perché tutti i fogli di stile stanno nel `<head>` e lo stato del guscio ricordato fra le pagine lo rimettono `base.js` e `claim_badge.js`, che girano subito dopo la barra laterale, prima di ogni script della pagina. Il contenuto invece compare solo quando è pronto, e fino ad allora resta quello della pagina lasciata: una transizione fra documenti ne tiene la foto nel riquadro del contenuto e la dissolve sul nuovo quando arriva.

Pronta vuol dire: documento letto, testi tradotti, richieste partite durante il caricamento concluse *e lette* (`fetch` e corpo della risposta, poi 40 ms senza richieste nuove: le catene di richieste sono coperte), grafici Plotly disegnati, caratteri caricati. Se la pagina tarda, dopo 400 ms l'icona della voce premuta pulsa e dopo 450 ms compare un filo di caricamento in alto; oltre 1,8 s il contenuto si mostra comunque, con i suoi segnaposto.

Cosa fare in una pagina nuova

- Il foglio di stile della pagina nel blocco `page_css`, mai nel contenuto (§ 6.4.2).
- Per il resto, nel caso normale niente: le richieste fatte all'avvio vengono contate da sole.
- Una richiesta *di fondo*, che la pagina non deve aspettare (un giro periodico, un conteggio per un'icona): `fetch(url, { qsSfondo: true })`.
- Per far aspettare alla pagina anche un lavoro che non passa da `fetch` né da Plotly, lo si consegna a `qsPagina.attendi(promessa)`.
- Per eseguire del codice a contenuto visibile: `qsPagina.quandoPronta(f)`.
- Se la pagina disegna con Plotly lo include lei, prima di `charts.js`, come fa `stream.html`: `base.html` non lo carica più.

Una richiesta lenta all'avvio tiene il contenuto in attesa

Fino al tetto di 1,8 s, con il contenuto della pagina di prima ancora sullo schermo. Se la pagina può mostrarsi senza quel dato, la richiesta è di fondo e va marcata: altrimenti chi apre la pagina aspetta un dato che non gli serve per cominciare.

Il guscio si vede dal primo fotogramma

Ciò che si aggiunge alla barra laterale o alla barra di sistema deve essere già nel suo stato al primo disegno: il suo aspetto lo decide un foglio nel `<head>`, il suo stato lo sistema uno script sincrono che gira subito dopo la barra laterale (`base.js`), non uno in fondo al body. Se dipende dal server, riparte dall'ultima risposta salvata nella sessione, come fanno la voce «Refund» e il pallino della campanella. Un elemento del guscio che compare all'arrivo di una richiesta lampeggia a ogni cambio di pagina.

Il perché delle due regole, i tempi e le reti di sicurezza sono in `ARCHITETTURA.md`, § 5.1.

6.4.2 I fogli di stile stanno tutti nel `<head>`

`base.html` carica prima i fogli comuni, poi quelli della pagina (il blocco `page_css`), poi `ui-system.css` e `ui-v2.css`. È lo stesso ordine di quando i fogli della pagina stavano nel contenuto e questi due in fondo al body, quindi vincono le stesse regole: le regole comuni continuano a valere sopra a quelle della pagina.

Il foglio di una pagina — `dashboard/templates/live.html`

```
{% block page_css %}
<link rel="stylesheet" href="{{ url_for('static', filename='css/live.css') }}">
{% endblock %}
```

Un foglio di stile nel body fa lampeggiare il guscio

Il primo disegno di solito aspetta che il documento sia letto fino a `#qs-fine`, l'ultimo elemento del body (`<link rel="expect">` nel `<head>`). Ma Chromium 126 smette di aspettare appena uno script interpreta un pezzo di HTML con il suo lettore lento: `innerHTML` con `<svg>` o `<table>`, `insertAdjacentHTML`, `createContextualFragment`. Gli script delle pagine lo fanno spesso, mentre il documento è ancora in lettura. Da quel momento il browser disegna ciò che ha: un foglio nel body non ancora letto vuol dire un guscio senza le sue regole per qualche fotogramma. Era il lampeggio del marchio «Lyra Capital» e dei titoletti della barra laterale, finché `ui-v2.css` stava in fondo al body.

6.4.3 Il modello da copiare

`ui_ext_e.py` è l'esempio migliore, per due ragioni che valgono per qualunque rotta nuova.

Il docstring dichiara il contratto —

`dashboard/ui_ext_e.py`

Rotte del gruppo E (Marketplace: vetrina, Sell yours, messaggi, feedback, account) per la UI v2.
GET /api/e/sell-status chi puo' pubblicare: piano (livello e nome), amministratore, lucchetto del marketplace e, se non si puo', il motivo nelle parole del sito. GET /api/e/account-data per la pagina My account, in una richiesta sola: prodotti pubblicati, acquisti, richieste di rimborso proprie e messaggi non letti.
Solo letture, e mai un errore verso la pagina: se il sito non risponde la rotta dice «non so» (`unknown: true`) e la pagina lo mostra come tale, invece di far credere che il piano manchi.

«Non so» non è «no» — È la regola di progetto più utile di tutto il capitolo, e vale ben oltre questo file. Quando la dashboard interroga il sito e il sito non risponde, la risposta sbagliata è `can_publish: false`: l'utente vede «non puoi pubblicare», che è un'affermazione, e va a cercare un problema nel proprio abbonamento. La risposta giusta è `unknown: true`, che la pagina mostra come incertezza.

Lo stesso schema ritorna in `/api/e/account-data`, che riporta un campo `errors`: «se una lettura fallisce lo dice, così la pagina non scambia *non riuscito* per *nessun acquisto*».

Un elenco vuoto è un'affermazione

Restituire `[]` quando non si è riusciti a leggere significa dire all'utente «non hai acquisti». Per chi ne ha, è un errore che preoccupa, e che porta ad aprire una contestazione. La distinzione fra «zero» e «non so» va mantenuta fino alla pagina.

6.4.4 Una funzione di servizio che vale la pena copiare

Le risposte JSON della dashboard contengono spesso date e `Decimal` che arrivano dal Postgres del sito, e che `jsonify` non sa serializzare:

Serializzazione difensiva dashboard/ui_ext_e.py:30

```
def _plain(v):
    """Valori serializzabili in JSON (date in ISO, Decimal in float)."""
    if isinstance(v, (str, int, float, bool, type(None))):
        return v
    if isinstance(v, dict):
        return {str(k): _plain(x) for k, x in v.items()}
    if isinstance(v, (list, tuple, set)):
        return [_plain(x) for x in v]
    iso = getattr(v, "isoformat", None)
    if callable(iso):
        try:
            return iso()
        except Exception:  # noqa: BLE001
            pass
    try:
        return float(v)
    except Exception:  # noqa: BLE001
        return str(v)
```

L'ultimo ripiego è `str(v)`: qualunque cosa arrivi, la pagina riceve qualcosa invece di un `500`. In una rotta di sola lettura è la scelta giusta.

6.4.5 Le due pagine speciali

`ui_ext_b.py` fa un redirect in un `before_request`
Una richiesta a `/ensemble_custom?project=<id>` viene rediretta a `/projects/<id>`. È il modo in cui i collegamenti vecchi continuano a funzionare dopo che una pagina ha cambiato indirizzo. Chi cerca perché un URL si trasforma in un altro lo trova lì, non nelle rotte.

`ui_ext_a.py` intercetta le installazioni in un `after_request`
Annota le installazioni fatte dallo Studio per i tutorial. Un `after_request` che osserva le risposte altrui è potente e invisibile: va dichiarato nel docstring del modulo, come lì è fatto.

7

Account, abbonamenti e marketplace

7.1 Il ponte verso il sito

La dashboard non sa chi è l'utente: lo chiede al sito. Non conserva password né segreti: li tiene il sito. Tutto ciò che riguarda identità, denaro e catalogo passa da `dashboard/app_api.py`, e il meccanismo è un token personale.

7.1.1 Dove sta il token

La ricerca del token, in ordine — `dashboard/app_api.py:67`

```
def token() -> str | None:
    global _TOKEN, _TOKEN_LOADED
    if _TOKEN_LOADED:
        return _TOKEN
    _TOKEN_LOADED = True
    env = (os.environ.get("LYRA_APP_TOKEN") or "").strip()
    if env:
        _TOKEN = env
        return _TOKEN
    try:
        p = _token_path()
        if p.exists():
            data = json.loads(p.read_text(encoding="utf-8"))
            _TOKEN = (data.get("token") or "").strip() or None
    except Exception:
        _TOKEN = None
    return _TOKEN
```

Variabile d'ambiente prima, file dopo, e il risultato è messo in cache per tutta la vita del processo. La cache è la ragione per cui, dopo un accesso nuovo, può servire un riavvio: `_TOKEN_LOADED` resta vero.

7.1.2 La chiamata che non solleva

`dashboard/app_api.py:137` · `chiama_sito()`

Chiamata al sito che NON solleva: ritorna (ok, dati).
Serve alle rotte che fanno solo da ponte — sicurezza dell'account, per esempio. L'app non conserva password ne' segreti: li tiene il sito, e qui si inoltra soltanto aggiungendo il token di chi ha fatto l'accesso.
Gli errori del sito arrivano con il proprio messaggio e il proprio codice di stato: tradurli qui significherebbe perdere spiegazioni che il sito ha scritto apposta.

Perché gli errori del sito si inoltrano senza tradurli

Il sito scrive messaggi pensati per essere letti dall'utente: «il tuo abbonamento è scaduto il . . . », «questa email è già registrata». Sostituirli con un generico «errore del server» cancella l'informazione. La dashboard fa da ponte, e un ponte non riscrive ciò che trasporta.

7.1.3 La regola del token giusto

C'è un secondo segreto, oltre al token dell'utente: quello della dashboard. Servono a cose diverse, e scambiarli produce il più insidioso dei difetti di questo capitolo.

`dashboard/community.py:76` · `SI()`

SI CHIAMA CON IL TOKEN DELL'UTENTE, non col segreto della dashboard. Prima no, e per questo non ha mai funzionato: `/api/app/abbonamento` riconosce la persona dal token, quindi rispondeva 401, l'eccezione veniva inghiottita e il risultato era «non abbonato» per tutti. Il sintomo era invisibile perche' in beta gli unici a pubblicare erano gli amministratori, che passano da un'altra strada.

Tre cose insieme rendono un difetto invisibile

Una risposta `401`, un'eccezione inghiottita, e un ripiego prudente che risponde «non abbonato». Presi uno per uno sono ragionevoli; insieme producono un sistema che funziona male e non lo dice. Chi aggiunge una chiamata al sito controlli quale dei due segreti serve, e verifichi il percorso d'errore con l'utente giusto — non con un amministratore, che spesso passa da un'altra strada e non lo attraversa mai.

7.1.4 «Non sapere vale come no»

Per la pubblicazione il ripiego prudente è invece la scelta corretta, ed è dichiarata:

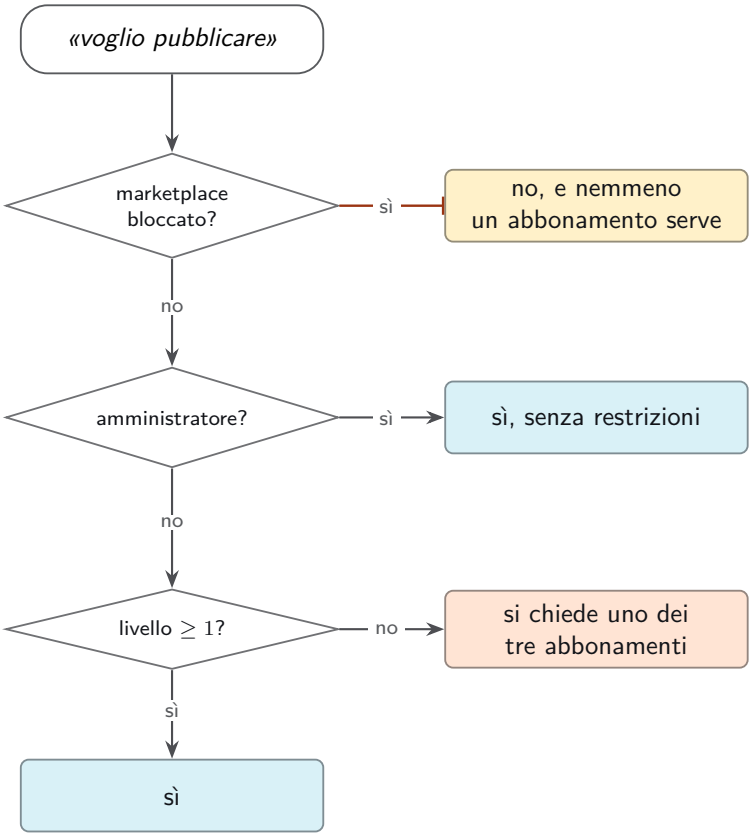
`dashboard/community.py:72` · `In()`

In caso di errore risponde «livello 0, non puo' pubblicare»: non sapere vale come no. Pubblicare per errore mette in vetrina roba di cui nessuno risponde e si toglie dopo; non pubblicare si risolve riprovando.

Il criterio è l'asimmetria del danno: una pubblicazione sbagliata si corregge con fatica e intanto è pubblica; una pubblicazione mancata si corregge riprovando. Si noti che è il contrario della regola vista per le *letture* (§ 6.4): lì «non so» non va mostrato come «no», perché nessuna azione irreversibile ne dipende. La differenza fra i due casi è se c'è un'azione, non se c'è un'incertezza.

7.2 I due strati del permesso di pubblicare

Chi può caricare un artefatto sul marketplace è deciso da due condizioni in cascata, e confonderle è il modo tipico di sbagliare una modifica in quest'area.



Il lucchetto è lo strato esterno: chiuso, non esiste nessun abbonamento che permetta di caricare. In fase di beta è chiuso, e i prodotti già in vetrina restano visibili.

Figura 7.1. I due strati, nell'ordine in cui si valutano.

7.2.1 Primo strato: il lucchetto

`dashboard/community.py:30` · `marketplace_bloccato()`

Il marketplace e' chiuso?
E' il PRIMO dei due strati che regolano la pubblicazione, e durante la beta e' quello che conta: chiuso significa che nessuno carica niente — ne' gratis ne' a pagamento — e nessuno compra. Prima al suo posto c'era un interruttore che, spento, lasciava pubblicare gratis: non bastava, perche' cio' che entra in vetrina ci resta e il filtro che decide cosa merita starci (l'abbonamento) non era ancora in piedi.
Lo decide il sito, non l'applicazione: e' l'unico punto che chi usa il programma non puo' modificare. In caso di dubbio risponde CHIUSO.

Tre proprietà, tutte deliberate:

- **Lo decide il sito.** È l'unico permesso che chi usa il programma non può modificare agendo sulla propria macchina — tutto il resto della dashboard è aperto (§ 3.4), questo no.
- **In caso di dubbio: chiuso.** Il `except` finale restituisce `True`.
- **Gli acquisti ne derivano.** `vendite_attive()` non è più un interruttore a sé: è `not marketplace_bloccato()`. Il commento dice perché resta come funzione: «due nomi per la stessa domanda sono due occasioni di rispondere diversamente».

7.2.2 Secondo strato: l'abbonamento

A marketplace aperto, per pubblicare serve un piano a pagamento, dal Pro in su. I piani sono quelli della pagina Prezzi del sito, che è il riferimento: cinque livelli contando il Gratuito.

I cinque livelli `dashboard/community.py:106`

```
# I CINQUE LIVELLI, con i nomi che l'interfaccia mostra: gli stessi piani
# della pagina Prezzi del sito. Il livello 0 non e' un ripiego: e' l'app
# intera, con cui si sfoglia, scarica, compra e usa tutto. I nomi veri (e i
# prezzi, e le voci) arrivano dal sito con il catalogo; questi servono solo
# quando il catalogo non c'e'.
NOMI_LIVELLO = {0: "Free", 1: "Pro", 2: "Professional", 3: "Quant", 4: "Desk"}
LIVELLO_MAX = max(NOMI_LIVELLO)
```

Liv.	Piano	Prezzo	Che cosa aggiunge
0	Free	0 €	L'app intera: costruire, provare, far girare; sul marketplace sfoglia, scarica, compra. *Non* pubblica. Quota sul generato 0 %: la strategia allocata entra nel track record.
1	Pro	49 € al mese, 490 € l'anno	Privacy totale, vendita sul marketplace e acquisti al −10 %, sandbox sui dati istituzionali (2030), 10 % del generato.
2	Professional	149 € al mese, 1490 € l'anno	Storico grezzo in locale (2030) e −10 % sui vendor di dati, marketplace −20 %, sandbox su più mercati, 30 % del generato.
3	Quant	da 200 € al mese, 2000 € l'anno, più i dati	Real-time con licenza di borsa (2032) a pacchetti per mercato, sandbox esteso.
4	Desk	8000 € l'anno	Cinque postazioni, installazione sui server dello studio, export per la compliance, white-label e SLA. Si vende a contratto, dopo una demo.

L'ordine dei livelli è l'ordine dei piani: l'interfaccia lo usa per dire «Upgrade» o «Downgrade». Pro, Professional e Quant si provano per due mesi, una volta per account; Desk non ha checkout, e la sua scheda apre una richiesta di demo che arriva al team come una richiesta di assistenza.

Il catalogo sta nel database, non nel codice

Nomi, prezzi, frasi e voci delle schede stanno nella tabella `SubscriptionPlan` del sito; il catalogo lo scrive `marketplace_abbonamenti.sql`, una volta sola (un segno in `AppSetting` impedisce che una seconda esecuzione riscriva i prezzi cambiati poi dal pannello). La colonna `features` tiene due specie di chiavi: i *fatti* del piano — privacy, sconto, sandbox, storico, real-time, quota, postazioni — da cui l'app costruisce la tabella di confronto, e le chiavi di *presentazione* — voci della scheda, etichetta, mesi di prova, solo annuale, solo a contratto, prezzo «da» — che il sito toglie da `features` e passa come campi del piano. Il motivo è l'app già installata: non conosce quelle chiavi, e trovandole dentro `features` le scriverebbe come funzionalità.

7.2.3 Gli amministratori

`dashboard/community.py:115` · `livello_utente()`

La categoria dell'utente: 0 se non abbonato, 1..4 secondo il piano.
Gli amministratori risultano al livello massimo, perche' possono fare tutto cio' che fa il livello massimo e anche di piu'. Non sono un livello vero — un livello scade, loro no — ma all'interfaccia serve un numero.

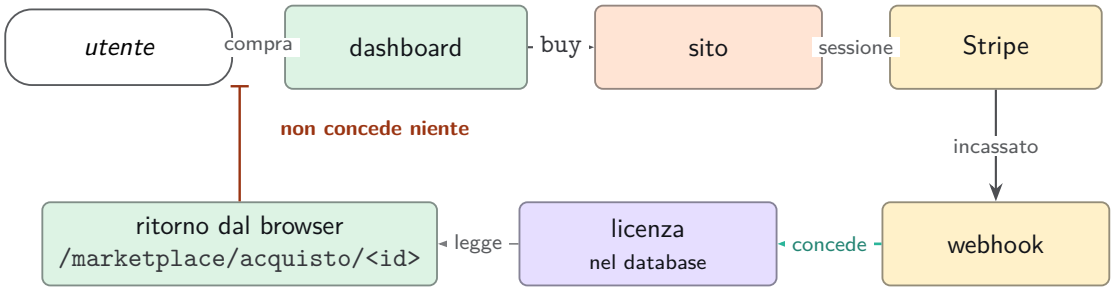
Un amministratore può inoltre installare un artefatto a pagamento senza comprarlo (**POST** `/api/marketplace/<id>/installa-admin`) e, simmetricamente, può scegliere di pagarlo comunque — il che per un amministratore equivale a una donazione all'autore, visto che non ne avrebbe bisogno.

Gli amministratori non attraversano i percorsi d'errore

È il motivo per cui il difetto del token (§ 7.1) è rimasto invisibile per mesi: in beta gli unici a pubblicare erano gli amministratori, che passano da un'altra strada. Chi collauda una modifica in quest'area deve farlo con un utente *non* amministratore, altrimenti collauda il ramo sbagliato.

7.3 I pagamenti

Questa è l'area in cui un errore costa denaro vero, e la regola che la governa è una sola: **l'accesso lo concede il webhook, e nessun altro**.



Prima il ritorno dal browser creava l'acquisto fidandosi della sessione indicata nell'indirizzo: chiunque lo aprisse otteneva gratis qualunque artefatto. Oggi l'unica autorità è il webhook; il ritorno si limita a leggere, e se la licenza non c'è ancora dice di attendere.

Figura 7.2. Il percorso di un acquisto. La freccia sbarrata è quella che esisteva e non esiste più.

7.3.1 La regola, e perché c'è

`dashboard/community.py:1982` · `marketplace_purchase_return()`

Ritorno dal browser dopo il pagamento.
NON concede nulla. Prima questa rotta creava l'acquisto come «paid» fidandosi della sessione indicata nell'URL: chiunque poteva aprirlo e ottenere gratis qualunque artefatto. Ora l'accesso lo apre soltanto il webhook del sito, quando Stripe conferma di aver incassato; qui si verifica solo se la licenza risulta gia' attiva, per dare all'utente un messaggio sensato.
Il pagamento e' appena avvenuto e il webhook puo' arrivare un istante dopo: se la licenza non c'e' ancora non e' un errore, si dice di attendere.

Un identificativo nell'URL non è una prova di pagamento

È il difetto classico dell'integrazione con un fornitore di pagamenti: l'utente torna su un indirizzo che contiene l'identificativo della sessione, e il codice lo tratta come conferma. Ma quell'indirizzo è visibile, indovinabile e condivisibile. L'unica conferma è quella che arriva *dal* fornitore, firmata, sul canale del webhook. Qualunque modifica in quest'area che aggiunga una concessione fuori dal webhook va considerata un difetto di sicurezza, non una comodità.

7.3.2 Quando il webhook non arriva

I webhook si perdono. Esiste quindi una rotta di riconciliazione, costruita con lo stesso principio:

Riconciliare senza concedere dashboard/community.py:2072

```
    def marketplace_riconcilia(artifact_id):
        """Completa un pagamento che il webhook non ha portato a termine.

        Non concede nulla di suo: chiede a Stripe se il pagamento risulta
        incassato, e solo in quel caso fa cio' che avrebbe fatto il webhook.
        """
```

«Non concede nulla di suo» è la frase che la rende legittima: non è una scorciatoia, è una seconda interrogazione alla stessa autorità.

7.3.3 Distinguere «non comprato» da «non ho potuto chiedere»

dashboard/community.py:2094 · `marketplace_perche_non_mio()`

Perche' l'artefatto non risulta acquistato.
Il possesso dipende da due letture: la licenza e l'acquisto. Se entrambe tornano vuote non si distingue «non l'hai comprato» da «non sono riuscito a chiedere», e sono cose opposte. Qui si guardano una per una, dicendo anche se la lettura e' stata rifiutata.

È la stessa distinzione fra zero e «non so» del § 6.4, applicata al caso in cui la differenza è fra «devi pagare» e «c'è un problema nostro». Senza questa rotta, un utente che ha pagato e non vede l'artefatto non ha modo di sapere di chi sia il problema, e l'unica mossa che gli resta è aprire una contestazione.

7.3.4 Gli abbonamenti: una rotta sola per due cose

dashboard/community.py:2680 · `abbonamento_checkout()`

Apre il pagamento di un piano, o cambia quello in corso.
Una rotta sola per entrambe le cose perche' la domanda dell'utente e' una sola — «voglio il livello X con cadenza Y» — e il sito sa se quella richiesta e' un abbonamento nuovo o un cambio con conguaglio. Due rotte avrebbero costretto l'interfaccia a indovinarlo.

Il corpo della richiesta porta `tier` e `periodo`; le cadenze sono mensile e annuale, con l'annuale a dieci mensilità (Desk solo annuale). La disdetta, il cambio di carta e le fatture passano invece dal portale di fatturazione (`POST /api/abbonamento/portale`), che è una pagina ospitata dal fornitore: non c'è codice da mantenere per quelle funzioni.

Tre regole stanno sul sito, nella rotta che crea la sessione di pagamento, e non nell'app:

- **La prova di due mesi.** Se il piano la prevede e l'account non ne ha mai avuta una né ha mai avuto un abbonamento su Stripe, la sessione porta la fine della prova: mesi di calendario, calcolati dal minuto della chiave di idempotenza, così due clic nello stesso minuto mandano a Stripe la stessa richiesta. La carta si chiede comunque, e il primo addebito parte da solo alla fine della prova. Il webhook scrive `subscriptionTrialEnd` finché la prova è in corso e `trialUsedAt` la prima volta, che non si svuota più.
- **Desk non si compra dal checkout.** Il piano porta `sales_only` e la rotta risponde 409 con il codice `sales_only`, anche a un amministratore.
- **Il piano si riconosce dal codice.** Nei metadati dell'abbonamento il checkout scrive il codice del piano accanto al livello; il webhook e la rotta delle fatture cercano il piano

prima dal codice, poi dal prodotto Stripe, poi dal numero. Desk era il livello 3 prima che arrivasse Quant, e un abbonamento nato allora porta ancora quel numero.

7.3.5 Le contestazioni

Un acquirente che ritiene che un artefatto non renda come dichiarato può aprire una pratica. Due rotte la rendono una verifica e non una discussione:

Rotta	Che cosa confronta
`.../verifica-integrita`	I file installati contro i file venduti, uno per uno.
`.../verifica-performance`	Le metriche dichiarate dal venditore contro quelle ottenute rieseguendo qui.

> **Perché la verifica è automatica**
>
> Una contestazione fra due persone su che cosa «renda» una strategia non si risolve discutendo. Le due rotte producono un confronto riproducibile: o i file sono quelli, o non lo sono; o le metriche si riproducono, o non si riproducono. L'amministratore che chiude la pratica decide su un risultato, non su due racconti.

7.3.6 Le variabili d'ambiente

Le chiavi Stripe, SMTP e simili *non* stanno in un `.env` locale: si gestiscono dal sito e arrivano alla metà locale attraverso il canale 2 (§ 1.2). È il motivo per cui un ambiente di sviluppo può avere il circuito di pagamento funzionante senza che nessuna chiave stia sul disco dello sviluppatore.

7.4 Pubblicare e installare

7.4.1 Nessuna moderazione, un filtro all'ingresso

La scelta, scritta dove si applica `dashboard/community.py:1541`

```
        # CHI PUO' PUBBLICARE
        # Solo amministratori e abbonati, anche per gli artefatti
        # gratuiti. Il motivo non e' il denaro: e' che chi pubblica
        # mette il proprio nome accanto a una strategia che altri
        # faranno girare sui propri soldi. Un filtro all'ingresso vale
        # piu' di qualunque moderazione a valle.
        #
        # Chi non e' abbonato resta un utente pieno: sfoglia, scarica,
        # compra e usa. Non puo' soltanto mettere roba in vetrina.
```

Un artefatto pubblicato va online subito: nessuna coda di approvazione. La difesa è a monte — serve un abbonamento — e a valle le contestazioni (§ 7.3), non in mezzo.

7.4.2 I conflitti si dicono prima

`GET /api/marketplace/<id>/conflicts` risponde a «che cosa esiste già con lo stesso nome». Si chiama *prima* di installare, e serve a non scoprire dopo che un'installazione ha scavalcato una strategia su cui si stava lavorando. Il nome canonico di un artefatto è il nome della sua cartella (§ 5.1): due artefatti con lo stesso nome sono la stessa cartella.

7.4.3 La pubblicazione automatica

`dashboard/autopubblica.py` gira in un thread dall'avvio e pubblica da sé gli artefatti che superano una soglia.

Che cosa fa `dashboard/autopubblica.py`

```
COSA FA
Ogni volta che una strategy viene scritta, e ogni volta che un project viene
creato, si guarda lo Sharpe out-of-sample del walk-forward. Sopra 2.0 e non
ancora in vetrina, l'artefatto va online GRATIS da solo. Per un project la
regola e' piu' articolata (vedi `valuta_progetto`): dipende da quante strategie
lo compongono e da cosa e' gia' pubblicato.
```

Il file di tracciamento — Senza, ogni avvio rifarebbe gli stessi controlli e — peggio — riproverebbe le stesse pubblicazioni. Il file registra, per ogni artefatto, se è stato visionato e con quale conclusione. È un JSON indentato, leggibile a mano: non un formato interno.

Quando una voce viene riesaminata — Tre casi, e solo questi:

1. non c'è (artefatto nuovo, o file di tracciamento appena nato);
2. **la conclusione non era definitiva** — mancava il backtest, mancava la rete, nessuno era connesso;
3. l'impronta del codice o quella dei risultati è cambiata.

La distinzione fra un ostacolo e un giudizio

Il commento del modulo la enuncia meglio di qualunque parafrasi: gli ostacoli «non sono giudizi sull'artefatto: sono ostacoli, e un ostacolo che passa deve poter cambiare l'esito. Una soglia non superata, invece, è un giudizio: resta tale finché il codice o i risultati non cambiano».
È il criterio per decidere se un esito negativo vada riprovato o meno, e si applica ben oltre questo modulo: ogni volta che un sistema registra un «no», la domanda è se quel no dipendeva dall'oggetto o dalle circostanze.

Un artefatto può finire in vetrina da solo

È un comportamento attivo che agisce sul mondo esterno senza che nessuno lo chieda in quel momento. Chi lavora su `autopubblica.py` tenga presente che un errore qui non produce un messaggio: produce una pubblicazione. Il file di tracciamento, essendo leggibile a mano, è lo strumento per verificare che cosa il modulo abbia deciso e perché.

7.4.4 L'identità di chi pubblica

Il marketplace ammette due identità diverse per lo stesso artefatto: il conto dell'utente e la macchina. «Già caricata» ha senso solo rispetto a una delle due, ed è la ragione per cui il pannello di amministrazione conta gli artefatti *per conto e per macchina* (`GET /api/admin/artefatti`), non solo in totale.

Parte IV

Tenerlo in piedi

8

Prove, rilascio e diagnosi

8.1 Come sono scritte le prove

Non c'è un `pytest` con mille asserzioni minute. Ci sono cinque suite, ciascuna dedicata a un *contratto* del sistema, che si lanciano come script e stampano un resoconto leggibile. La scelta ha una conseguenza utile: ogni suite si apre con un docstring che dice quale problema sorveglia, e quello è il posto giusto dove cercare prima di modificare l'area che copre.

Suite	Righe	Che contratto sorveglia
`prova_normalizzazione_stream.py`	691	Backtest e live devono produrre lo *stesso libro* (§ 4.5).
`prova_beta_accesso.py`	614	L'accesso riservato in beta, **contro il sito vero**.
`prova_covarianza.py`	489	Lo shrinkage di Ledoit–Wolf e la modalità `covariance`.
`prova_pesi_dichiarati.py`	484	Il contratto di `Signal.target_weight` e la modalità `explicit`.
`prova_permessi_marketplace.py`	429	I due strati del permesso di pubblicare, **contro il sito vero**.

8.1.1 Due suite girano contro il sito vero

Non usano finti. Avviano un Next.js e un Postgres reali e interrogano gli endpoint veri. È più lento e più fastidioso da far partire, e la ragione è dichiarata nel docstring: i permessi e l'accesso non vivono nel codice Python, vivono nelle regole del sito. Una prova con dei finti verificherebbe che la dashboard sa leggere una risposta — cosa che non è mai stata il problema — e non che la risposta è quella giusta.

8.1.2 Le proprietà che una prova deve avere qui

Tre, e tutte e tre sono state imparate rompendole.

Ripetibile
: Una suite che modifica lo stato (prezzi dei piani, elenchi di ammessi) deve azzerarlo all'inizio, non alla fine: se una esecuzione si interrompe a metà, la pulizia finale non viene eseguita e la successiva parte da uno stato sporco. `prova_permessi_marketplace.py` comincia con una reimpostazione in SQL; `prova_beta_accesso.py` ha una funzione `azzera_prova()` e un elenco fisso di indirizzi.

Senza soglie indovinate
: Un'asserzione come `assert scala < 1.3` è una soglia inventata: passa o fal-

lisce per ragioni che nessuno ha verificato. La forma corretta è relativa — `assert scala_degenere < 0.75 * scala_corretta` — perché confronta due cose che il sistema produce, invece di confrontarne una con un numero scritto a mano.

Che isola ciò che misura
Misurare la proporzionalità confrontando due chiamate complete non funziona se ciascuna chiamata registra un'osservazione in più e cambia lo stato. Il confronto va fatto sulla funzione che si vuole misurare, da sola.

Una prova che non è ripetibile è peggio di nessuna prova

Passa la prima volta, fallisce la seconda, e chi la vede fallire conclude che la prova è inaffidabile — non che il codice lo sia. Da quel momento il risultato di quella suite viene ignorato, cioè la suite ha smesso di esistere pur continuando a girare.

8.1.3 Il resto di `tests/`

Oltre alle cinque suite ci sono: prove end-to-end (`e2e_hedging.py`, `e2e_regime_modulation.py`), confronti A/B fra strategie, diagnosi specifiche per singole strategie (`diagnose_ising.py`, `diagnose_regime_signals.py`), un server SMTP finto (`posta_finta.py`) usato dalla prova dell'accesso, e verifiche numeriche di migrazione (`verify_migration_numbers.py`).

8.2 Quando qualcosa non va

Un elenco dei sintomi che capitano davvero, con la causa più probabile accanto. Ciascuno rimanda al punto della guida che lo spiega.

Sintomo	Dove guardare
Metà dell'interfaccia è sparita dopo una modifica	Un modulo `ui_ext_*` solleva all'import, e il `try` unico impedisce anche agli altri di caricarsi. Nel registro: `v2 page extensions not loaded`. §3.3.2
Una strategia non compare nell'elenco	`pythontests/diagnostica_strategie.py`, che stampa l'errore di import *visibile* — nella dashboard finisce in un avviso che nessuno legge.
Un artefatto nuovo non compare	Dieci cause possibili, in ordine di frequenza: §5.6.
Una modalità di pesatura compare ma risponde `400`	Ricarica parziale del registro: `EnsembleEngine.WEIGHTING_MODES` non è stato aggiornato. §5.6
Il backtest si ferma alla prima barra con `TypeError`	La firma di `generate_signals` usa nomi di parametro diversi da quelli attesi. §4.2
Lo Sharpe è straordinario	Prima di festeggiare: lookahead. Si legge oltre `t_decision`, oppure una soglia è un quantile calcolato sull'intero campione. §4.2
Dal vivo il conto si muove molto più (o molto meno) del backtest	La normalizzazione per stream. È il difetto storico del progetto, e la suite che lo sorveglia è `prova_normalizzazione_stream.py`. §4.5

Sintomo	Dove guardare
Il marketplace è vuoto in locale e pieno sul sito	Canale 1: token mancante o scaduto. Il token è in cache per tutta la vita del processo. §7.1
«Non sei abbonato» per tutti	Si sta chiamando il sito con il segreto della dashboard invece che con il token dell'utente. §7.1
Un download dati fallisce solo su una macchina	Canale 2: la chiave non è arrivata. Le variabili d'ambiente le gestisce il sito. §1.2
Una pagina mostra «nessun acquisto» a chi ne ha	Una lettura fallita trattata come elenco vuoto. §6.4
Un mercato di regime non modula	Nessuno dei suoi `driver` è disponibile: lo dice un avviso nel registro, non un errore. Serve `scripts/download_regime_drivers.py`. §5.5
Le pagine sono tutte vuote all'avvio	Non ci sono dati. Non è un errore: non c'è niente da mostrare. §2.3
`/ensemble/core` risponde `404`	Gli ensemble preconfigurati sono stati disattivati: esiste solo l'ensemble costruito a runtime. §6.2

8.2.1 Gli strumenti di diagnosi

Strumento	A che serve
`tests/diagnostica_strategie.py`	Per ogni cartella di strategia: importa o dà errore, con il traceback intero. Si lancia dalla radice, con lo *stesso* interprete con cui si lancia la dashboard — la differenza fra due ambienti virtuali è una causa frequente.
`GET /api/marketplace/diagnostica`	Verifica la catena della vetrina e dice in quale punto si rompe.
`GET /api/marketplace/<id>/perche-non-mio`	Perché un artefatto non risulta acquistato, distinguendo «non comprato» da «non sono riuscito a chiedere». §7.3
`GET /api/data/cache_status`	Che cosa c'è su disco: numero di file, dimensione, intervallo di date.
`GET /api/universo`	Che cosa il sistema *sa* di avere, contro che cosa ha davvero. La differenza fra le due è il registro `data/_universe.json`.
`GET /api/streams/orfani`	Risultati senza una strategia che li abbia prodotti: restano dopo una cancellazione, e sporcano gli elenchi.

Il primo posto dove guardare, sempre

Il registro. Questo sistema è pieno di `try/except` che registrano un avviso e proseguono, ed è una scelta giusta — un plugin rotto non deve impedire l'avvio — che però sposta l'informazione dal comportamento al registro. Un sintomo che non ha un errore corrispondente nell'interfaccia ha quasi sempre una riga nel registro, e quasi sempre quella riga contiene il nome del modulo colpevole.

8.3 Impacchettare e rilasciare

8.3.1 Un comando solo

Dalla radice del progetto build/build_app.py

```
python build/build_app.py                # backend + app per il sistema corrente
python build/build_app.py --backend-only # solo l'eseguibile del backend
python build/build_app.py --app-only     # solo Electron (backend gia' pronto)
```

Tre passi, in ordine: PyInstaller produce un backend autonomo che non richiede Python sulla macchina di chi riceve l'app; il risultato viene copiato dove electron-builder lo cerca; electron-builder produce l'installatore per il sistema corrente (`.exe`, `.dmg`, `AppImage`).

8.3.2 Dove finiscono i dati dell'utente

Non dentro l'applicazione. Il bundle è di sola lettura e ogni aggiornamento lo sovrascrive: al primo avvio il backend crea uno spazio di lavoro nella cartella utente e vi copia le librerie iniziali — strategie, approcci, modalità di pesatura. È lo stesso meccanismo di `PROJECT_ROOT` descritto nel § 2.3, visto dall'altro capo.

> **Le librerie iniziali si copiano una volta**
>
> Al primo avvio, non a ogni aggiornamento. Un artefatto nuovo aggiunto a una versione nuova dell'applicazione *non* compare automaticamente nello spazio di lavoro di chi ha già installato: quello spazio è suo, e sovrascriverlo cancellerebbe le sue modifiche. È una conseguenza voluta del punto precedente, e va tenuta presente quando si aggiunge un artefatto di serie.

8.3.3 Gli aggiornamenti automatici

Chi ha già installato riceve le versioni nuove da solo: all'avvio `electron-updater` interroga le release di GitHub, scarica in silenzio e mostra la notifica «aggiornamento pronto» nella campanella. L'utente riavvia quando gli fa comodo. Il repository si ricava dal remote git, o si dichiara con `GH_REPO`.

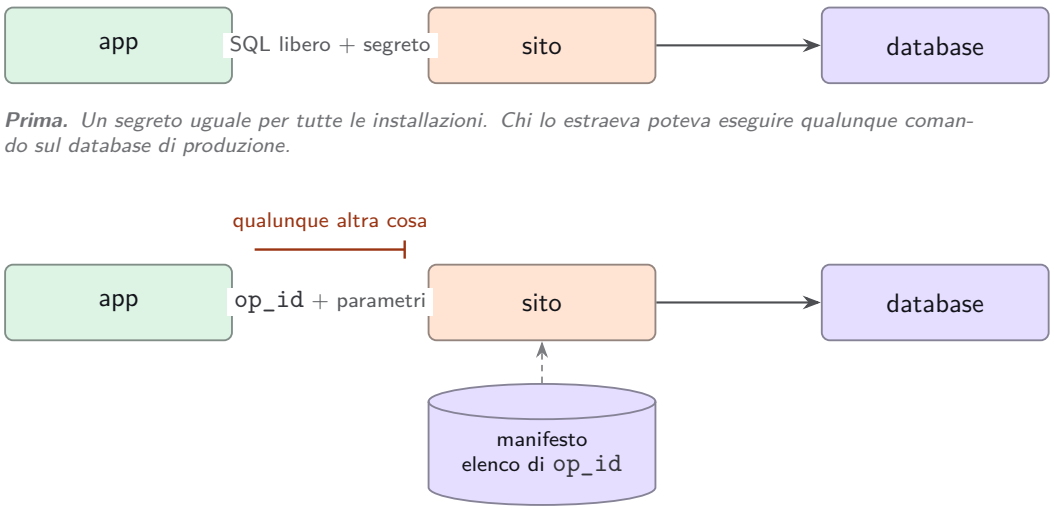
8.4 Il manifesto delle query

È il meccanismo di sicurezza più importante fra la metà locale e il sito, e merita di essere capito prima di toccare qualunque cosa interroghi il database.

Il motivo, dal generatore build/gen_query_manifest.py

```
Perche' esiste
--------------
Finora l'app inviava SQL libero a ``/api/dashboard/query``, protetto da un
segreto uguale per tutte le installazioni. Chi estraeva quel segreto poteva
eseguire qualsiasi comando sul database di produzione.

Con questo file il rapporto si rovescia: il sito conosce in anticipo l'elenco
esatto delle query ammesse e l'app puo' solo chiederne una per NOME (un hash
del testo SQL). Qualunque altra cosa viene rifiutata, segreto o no.
```



Prima. *Un segreto uguale per tutte le installazioni. Chi lo estraeva poteva eseguire qualunque comando sul database di produzione.*

Dopo. *Il sito conosce in anticipo l'elenco esatto delle query ammesse; l'app può solo chiederne una per nome. E il parametro che identifica l'utente viene riscritto dal sito con l'identità del token: anche mentendo, il client ottiene solo i propri dati.*

Figura 8.1. Il rapporto di fiducia, prima e dopo.

8.4.1 L'identificativo di una query

`op_id` è l'impronta SHA-256 del testo SQL normalizzato negli spazi, troncata a 32 cifre esadecimali. Conseguenza diretta: **cambiare una query cambia il suo identificativo**. Una query modificata e non rigenerata nel manifesto viene rifiutata, segreto o no.

8.4.2 La legatura all'utente

Limitare le query non basta: chi è autenticato potrebbe chiedere i dati di un altro passando un identificativo diverso. Per ogni query il generatore individua il parametro che rappresenta l'utente, in due forme:

Le due forme riconosciute `build/gen_query_manifest.py`

```
    WHERE "userId" = %s              -> conta i %s che lo precedono
    INSERT INTO t(id, "userId", ...) -> posizione nella lista delle colonne
```

Il sito *sovrascrive* quel parametro con l'identificativo ricavato dal token. Anche mentendo, il client ottiene solo i propri dati.

Quando si tocca una query

1. Modificare l'SQL.
2. `pythonbuild/gen_query_manifest.py` — riscrive il JSON nel sito.
3. `pythonbuild/gen_query_manifest.py--check` — verifica che sia aggiornato. È il comando da mettere in una verifica automatica.
4. Rilasciare *prima* il sito, poi l'app: un'app che chiede un `op_id` che il sito non conosce ancora viene rifiutata.

L'ordine di rilascio non è simmetrico

Il sito accetta sia gli identificativi vecchi sia quelli nuovi finché il manifesto li contiene entrambi; l'app no. Aggiornare l'app prima del sito rompe le installazioni appena aggiornate, e il sintomo è un rifiuto senza spiegazione — perché dal punto di vista del sito è esattamente ciò che deve succedere a una query sconosciuta.

9
Riferimento

Questo capitolo non si legge: si consulta. Le due tabelle lunghe — le porte e le rotte — sono *generate dal sorgente*, non trascritte, e si rigenerano con gli script in `strumenti/`. Una tabella di riferimento sbagliata è peggio di una tabella assente, perché la si consulta invece di leggere il codice.

9.1 Le porte

Trentuno `Protocol` in `ports/__init__.py`, più la porta `Broker` ri-esportata da `live/broker.py`: trentadue contratti in tutto. Sono ciò che i casi d'uso conoscono del mondo (§ 3.1).

Porta	Che cosa promette
Strategie e risultati	
`StrategyRepository`	Le strategie disponibili, come `StrategyInfo`.
`WalkForwardRunner`	Esegue un walk-forward. Restituisce il codice di uscita.
`StreamCatalog`	Gli stream fuori campione disponibili.
`ReportSink`	Dove finiscono i report di una esecuzione.
`StrategyStore`	Persistenza su filesystem, sotto `strategies/<nome>/`.
`StrategyValidator`	Validazione di sicurezza statica del sorgente (`AST`).
Composizione e analisi	
`WeightingModeProvider`	L'elenco delle modalità di pesatura.
`EnsembleService`	Calcola l'ensemble di N stream con una pesatura scelta.
`FactorAttributionService`	Scompone i rendimenti in alfa più beta ai fattori standard.
`StressTestService`	Prova uno stream sulle crisi storiche (2 018, 2 020, 2 022…).
Dati, tempo, mercato	
`DataSource`	Accesso agli OHLCV; avvolge i loader esistenti.
`Clock`	Orologio iniettabile. Esiste perché `datetime.now()` sparso nel codice rende i test non deterministici.

Porta	Che cosa promette
`BrokerGateway`	Verso un broker, parlando in tipi di dominio: l'adattatore traduce, e i casi d'uso non toccano i tipi dell'infrastruttura.
`Broker`	*Ri-esportata* da `live.broker.LiveBroker`: era già una classe base astratta, cioè già una porta.
`ModelRegistryPort`	Registro versionato dei modelli ML, con promozione.
Generazione e compilazione degli artefatti	
`CodeGenerator`	Generazione di codice. Il backend è Ollama in locale o un modello proprietario; `list_models()` alimenta il menu.
`NodeCompiler`	Compila un grafo a nodi in sorgente Python.
`FidelityScorer`	Confronta i trade disegnati a mano con quelli della strategia.
`WeightModeValidator`	Sicurezza più struttura (`META` e `compute`).
`WeightModeStore`	`exists(key)`, `save(key, sorgente, overwrite)`.
`WeightModeNodeCompiler`	Grafo a nodi → modalità di pesatura.
`IndicatorValidator`	Come sopra, per un indicatore.
`IndicatorNodeCompiler`	Grafo a nodi → indicatore.
`OverlayValidatorPort`	Come sopra, per un overlay.
`OverlayStore`	`exists(nome)`, `save(nome, sorgente, overwrite)`.
`OverlayNodeCompiler`	Grafo a nodi → overlay.
Operatività dal vivo	
`LiveRunnerValidatorPort`	`validate(cfg)` → `LiveRunnerValidation`.
`LiveRunnerStore`	`exists`, `save`, `load`, `list`.
`LiveRunnerUploader`	Carica una configurazione di runner.
Marketplace	
`ArtifactPublisherPort`	Carica un artefatto sul proprio conto sul sito.
`AccountArtifactsPort`	Legge gli artefatti già presenti sul conto — serve a non duplicarli.
`ArtifactCodeSourcePort`	Legge il codice locale di un artefatto, per tipo e nome.

Dieci porte senza docstring

Le porte della famiglia `WeightMode*`, `Indicator*`, `Overlay*` e `LiveRunner*` non hanno un docstring: si capiscono dalla firma, che è di due o tre metodi. Non è grave — la firma *è* il contratto — ma le altre ventuno ce l'hanno, e l'asimmetria si nota aprendo il file.

9.2 Le costanti e la configurazione

9.2.1 `config/quantsys.yaml`

È il file centrale. Dopo una modifica serve riavviare la dashboard: non viene riletto a caldo.

Chiave	Valore	Che cosa governa
`data:`		
`start_date`	2 010-01-01	Inizio del campione.
`train_years`	5	Ampiezza della finestra di addestramento.
`test_years`	1	Ampiezza della finestra di prova.
`slide_years`	1	Di quanto avanza la finestra.
`warmup_days`	300	Barre saltate all'inizio.
`min_folds`	3	Minimo di finestre perché il walk-forward abbia senso.
`benchmark_symbol`	SPY	Il riferimento dei confronti.
`portfolio:`		
`initial_capital`	1 000 000	Capitale iniziale.
`portfolio_target_vol_annual`	0.12	Volatilità annua obiettivo.
`max_position_weight`	0.05	Tetto per strumento.
`max_asset_class_gross`	1.00	Tetto lordo per classe.
`max_gross_leverage`	1.0	Tetto lordo totale.
`ensemble:`		
`target_vol`	0.10	Volatilità a cui ogni stream viene portato *prima* di essere pesato.
`scale_lookback`	252	Barre della volatilità trailing.
`regime_window`	90	Finestra del regime.
`regime_sh_low` / `_high`	0.0 / 1.5	Soglie di Sharpe per la modulazione.

`ensemble.target_vol` non è un parametro di reporting

Il commento nel file lo dice: «non è un dettaglio di reporting: lo stesso fattore moltiplica il libro eseguito dal vivo». Uno stream a volatilità molto bassa chiede molta leva per arrivare all'obiettivo, e il tetto resta `max_gross_leverage`. Cambiare questo valore cambia le dimensioni delle posizioni su un conto reale (§ 4.5).

I valori della tabella sono quelli del file come si trova nel progetto; i default del codice, usati quando una chiave manca, sono diversi e stanno in `AllocatorConfig` (§ 4.3).

9.2.2 Le costanti del motore

Costante	Valore	Dove, e a che serve
`SCALE_MIN_OBS`	20	`reporting/ensemble.py`. Sotto questa soglia non c'è stima di volatilità e il fattore vale `1.0`.
`SCALE_MAX_LEVA`	10.0	Idem. Rapporto massimo fra obiettivo e volatilità stimata: è il tetto alla leva quando la volatilità tende a zero.
`max_workers`	1	`execution/job_manager.py`. I job sono serializzati (§ 6.3).
`max_log_lines`	5 000	Idem. Righe di registro conservate per ogni job.
`min_lookback_days`	252	Default di `StrategyDescriptor`.

9.2.3 Le variabili d'ambiente

Variabile	Effetto
`LYRA_HOME`	Sposta la cartella di lavoro (§ 2.3).
`LYRA_APP_TOKEN`	Scavalca il token personale letto da file (§ 7.1).
`FLASK_HOST`	Default `127.0.0.1`. Cambiarlo espone la dashboard (§ 2.2).
`FLASK_PORT`	Default 5 000.
`GH_REPO`	Repository per gli aggiornamenti automatici (§ 8.3).

Le chiavi dei servizi esterni — Polygon, FRED, Stripe, SMTP, Google, Postgres — *non* stanno qui: le gestisce il sito e arrivano alla metà locale attraverso il canale 2 (§ 1.2).

9.3 Le guide dentro il progetto

Il progetto porta con sé una trentina di file Markdown. Non sono ridondanti rispetto a questa guida: sono *riferimenti di dettaglio* per chi sta già scrivendo un artefatto, mentre questo documento spiega come il sistema sta in piedi. Dove i due divergono, vince il codice — e quando è stato possibile, questa guida lo dice (§ 5.4).

File	Righe	A che serve
Architettura		
`ARCHITETTURA.md`	1 220	Il documento di architettura originale.
`README.md`	122	Il ragionamento dietro il guscio a strati.
`MODIFICHE_QUANTSYS.md`	358	Storia delle modifiche al motore.
`dashboard/MODIFICHE.md`	778	Storia delle modifiche all'interfaccia.
`dashboard/MODIFICHE-UI.md`	494	Idem, lato grafico.
`MODIFICHE_DASHBOARD_V2.md`	118	Che cosa ha portato la v2: pagine nuove, finestra dei pagamenti, marchio.
Strategie		
`GUIDE_STRATEGIE_CUSTOM.md`	2 281	**Il riferimento principale.** Contratto completo, avvio rapido, descrittore campo per campo.
`GUIDE_STRATEGIE_BLUEPRINT.md`	288	La strategia come grafo di nodi invece che in Python. Alternativo, non complementare.
`GUIDE_REGIME.md`	487	Come *usare* i regimi dentro una strategia: `regime_weights`, anti-pattern, validazione.
`GUIDE_HEDGING.md`	248	Posizioni multiple sullo stesso simbolo (il default è il netting).
`GUIDE_STRATEGIE_MMS.md`	625	Le feature della heatmap di liquidità, via `required_mms_features`.
`GUIDE_VOLUME_PROFILE.md`	526	Le zone di supporto e resistenza, via `required_volume_profile`.
`STRATEGIE_FX_CANDIDATE.md`	231	Un documento di *proposta*: strategie in valuta scrivibili nel framework attuale. Non è implementazione.
Approcci		
`approaches/README.md`	303	Che cos'è un approccio e perché scavalca il motore.
`GUIDE_APPROCCI_CUSTOM.md`	359	Il riferimento. Si apre con la tabella di confronto strategia/approccio.
`GUIDE_APPROCCI_BLUEPRINT.md`	76	L'approccio a nodi; copre solo le differenze dalla guida delle strategie.
Artefatti di forma B		
`weights/WEIGHT_MODE_TEMPLATE.md`	197	Contratto, import ammessi e vietati, esempio funzionante. **Attenzione al valore di `kind`** (§ 4.4).
`overlays/OVERLAY_TEMPLATE.md`	212	Si apre con una tabella che separa overlay e approccio su sei assi.

File	Righe	A che serve
`indicators/INDICATOR_TEMPLATE.md`	127	Modello di un indicatore.
`metrics/METRIC_TEMPLATE.md`	320	Modello di una metrica: i dati del sistema (`data`), i grafici a più serie, il CAPM come esempio.
Regimi		
`regime/GUIDE_REGIMI.md`	190	Come *costruire* un regime: i due artefatti, la catena, l'isteresi.
`logics/LOGIC_TEMPLATE.md`	84	Modello della regola.
`markets/MARKET_TEMPLATE.md`	80	Modello del mercato.
Rilascio		
`build/README.md`	289	L'impacchettamento.
`build/RILASCIO.md`	186	Pubblicare un aggiornamento.
`DOCKER.md`	153	Avviare con Docker.

Due guide che sembrano la stessa e non lo sono

`regime/GUIDE_REGIMI.md` è «come costruirne uno»; `strategies/GUIDE_REGIME.md` è «come usarlo dentro una strategia». Il nome quasi identico è infelice, la distinzione è reale, e i due documenti si rimandano a vicenda.

9.4 Le rotte della dashboard

9.4.1 app.py — il nucleo

Metodi	Percorso	Funzione
GET	`/`	`index`
GET	`/admin`	`admin_page`
GET POST	`/api/account/security`	`api_account_security`
GET POST	`/api/admin/amministratori`	`api_admin_elenco`, `api_admin_modifica`
GET	`/api/admin/artefatti`	`api_admin_artefatti`
GET	`/api/admin/artefatti/zip`	`api_admin_artefatti_zip`
GET	`/api/admin/beta`	`api_admin_beta`
GET POST	`/api/admin/beta-access`	`api_admin_beta_access`, `api_admin_beta_access_modifica`
GET	`/api/admin/feedback`	`api_admin_feedback`
GET	`/api/admin/feedback/export`	`api_admin_feedback_export`
POST	`/api/admin/feedback/group`	`api_admin_feedback_group`

Metodi	Percorso	Funzione
GET POST	/api/admin/piani	api_admin_piani, api_admin_piani_modifica
GET	/api/admin/users	api_admin_users
GET POST	/api/admin/vendite	api_admin_vendite, api_admin_vendite_modifica
GET	/api/brokers/<broker>/check	api_broker_check
GET POST	/api/brokers/credentials	api_broker_credentials
GET POST	/api/brokers/test/config	api_test_config
GET POST	/api/categorie	api_categorie, api_categorie_crea
POST	/api/categorie/<cat>	api_categorie_modifica
POST	/api/categorie/<cat>/elimina	api_categorie_elimina
GET POST	/api/config/settings	api_config_settings_get, api_config_settings_save
GET	/api/config/yaml	api_config_yaml
GET	/api/data/cache_status	api_data_cache_status
POST	/api/data/capabilities	api_data_capabilities
GET POST	/api/data/categories	api_data_categories, api_data_category_create
POST	/api/data/categories/<name>/delete	api_data_category_delete
POST	/api/data/categories/<name>/empty	api_data_category_empty
GET	/api/data/categories/<name>/files	api_data_category_files
POST	/api/data/categories/<name>/kind	api_data_category_kind
POST	/api/data/download/<mode>	api_data_download
POST	/api/data/mms/generate	api_data_mms_generate
POST	/api/data/upload	api_data_upload
GET	/api/ensemble/<name>	api_ensemble
POST	/api/ensemble_custom	api_ensemble_custom
GET	/api/health	api_health
GET	/api/job/<job_id>	api_job
POST	/api/job/<job_id>/cancel	api_job_cancel
GET	/api/job/<job_id>/stream	api_job_stream
GET	/api/jobs	api_jobs_list
DELETE	/api/jobs/<job_id>	api_job_rimuovi
DELETE	/api/jobs/conclusi	api_jobs_svuota
GET	/api/live/approaches	api_live_approaches
GET	/api/live/ensemble_modes	api_live_ensemble_modes
POST	/api/live/run_cycle	api_live_run_cycle
POST	/api/live/start	api_live_start
GET	/api/live/state	api_live_state
GET	/api/live/status	api_live_status
POST	/api/live/stop	api_live_stop
GET	/api/live/strategies	api_live_strategies
GET POST	/api/logics	api_logic_create, api_logics
GET POST DELETE	/api/logics/<key>/file	api_logic_file

Metodi	Percorso	Funzione
GET	/api/maintenance/plan	api_maintenance_plan
GET	/api/markets	api_markets
POST	/api/markets/<key>/config	api_market_config
GET POST	/api/markets/<key>/file	api_market_file
GET POST	/api/metrics	api_metrics
DELETE		
POST	/api/metrics/compute	api_metrics_compute
GET	/api/notifications	api_notifications
GET POST	/api/overview/prefs	api_overview_prefs
GET	/api/plugins	api_plugins
GET POST	/api/projects	api_projects
GET POST	/api/projects/<pid>	api_project
DELETE		
GET	/api/regime	api_regime
POST	/api/reload	api_reload
POST	/api/retrain	api_retrain
POST	/api/run/<strategy>	api_run_strategy
POST	/api/run_all	api_run_all
GET POST	/api/runners	api_runners_create, api_runners_list
DELETE	/api/runners/<rid>	api_runners_delete
GET	/api/runners/<rid>/journal	api_runners_journal
POST	/api/runners/<rid>/kill-switch/reset	api_runners_kill_reset
POST	/api/runners/<rid>/mode	api_runners_mode
GET	/api/runners/<rid>/simulation	api_runner_simulation
GET	/api/runners/<rid>/simulation/series/<symbol>	api_runner_simulation_series
POST	/api/runners/<rid>/start	api_runners_start
GET	/api/runners/<rid>/status	api_runners_status
POST	/api/runners/<rid>/stop	api_runners_stop
GET	/api/strategies	api_strategies
GET POST	/api/stream-view/prefs	api_stream_view_prefs
GET	/api/stream/<name>	api_stream
POST	/api/stream/<name>/delete	api_stream_delete
GET	/api/stream/<name>/files	api_stream_files
GET	/api/streams	api_streams_list
GET	/api/streams/orfani	api_streams_orphans
POST	/api/streams/orfani/pulisci	api_streams_orphans_clean
GET	/api/universo	api_universo
POST	/api/universo/<symbol>/categoria	api_universo_categoria
GET POST	/api/user_approaches	api_user_approach_create, api_user_approaches_list
GET DELETE	/api/user_approaches/<name>	api_user_approach_delete, api_user_approach_files
GET POST DELETE	/api/user_approaches/<name>/<filename>	api_user_approach_file_delete, api_user_approach_file_get, api_user_approach_file_save

Metodi	Percorso	Funzione
GET POST	/api/user_strategies	api_user_strategies_list, api_user_strategy_create
GET DELETE	/api/user_strategies/<name>	api_user_strategy_delete, api_user_strategy_files
GET POST DELETE	/api/user_strategies/<name>/<filename>	api_user_file_delete, api_user_file_get, api_user_file_save
GET	/api/visual/<strategy>/chart/<symbol>	api_visual_chart
POST	/api/visual/<strategy>/custom_indicator/<symbol>	api_visual_custom_indicator
GET	/api/visual/<strategy>/indicators/<symbol>	api_visual_indicators
GET	/api/visual/<strategy>/liquidity_heatmap/<symbol>	api_visual_liquidity_heatmap
GET	/api/visual/<strategy>/mms/<symbol>	api_visual_mms
GET	/api/visual/<strategy>/symbols	api_visual_symbols
GET	/api/visual/<strategy>/zones/<symbol>	api_visual_zones
GET	/api/visual/indicator_template	api_visual_indicator_template
GET POST	/api/visual/indicators	api_visual_indicators_list, api_visual_indicators_save
POST	/api/visual/indicators/delete	api_visual_indicators_delete
GET	/config	config_page
GET	/data	data_page
GET	/editor	editor_page
GET	/ensemble/<name>	ensemble_page
GET	/ensemble_custom	ensemble_custom_page
GET	/favicon.ico	favicon
GET	/live/test/<rid>	live_test_page
GET	/profile	profile_page
GET	/projects	projects_page
GET	/regime	regime_page
GET	/run	run_page
GET	/security	security_page
GET	/stream/<name>	stream_page
GET	/streams	streams_page
GET	/subscription	subscription_page
GET	/visual	visual_page
GET	/visual/<strategy>	visual_page

9.4.2 studio.py — blueprint `studio`

Metodi	Percorso	Funzione
GET	/studio	page
POST	/studio/api/approach/install	api_approach_install
GET	/studio/api/available/approaches	api_avail_approaches
GET	/studio/api/available/metrics	api_avail_metrics
GET	/studio/api/available/overlays	api_avail_overlays
GET	/studio/api/available/strategies	api_avail_strategies

Metodi	Percorso	Funzione
GET	/studio/api/available/weightmodes	api_avail_weightmodes
POST	/studio/api/compile	api_compile
POST	/studio/api/data/adapter/apply	api_data_apply
POST	/studio/api/data/adapter/generate	api_data_generate
POST	/studio/api/data/analyze	api_data_analyze
POST	/studio/api/fidelity/codify	api_fid_codify
POST	/studio/api/fidelity/score	api_fid_score
POST	/studio/api/generate	api_generate
POST	/studio/api/indicator/compile	api_ind_compile
POST	/studio/api/indicator/generate	api_ind_generate
POST	/studio/api/install	api_install
GET	/studio/api/installati/<categoria>	api_installati
GET	/studio/api/installati/<categoria>/<nome>	api_installato_apri
POST	/studio/api/installati/<categoria>/<nome>/elimina	api_installato_elimina
GET POST	/studio/api/manual-trades	api_manual_trades
POST	/studio/api/metric/compile	api_metric_compile
GET	/studio/api/metric/doc	api_metric_doc
POST	/studio/api/metric/generate	api_metric_generate
GET	/studio/api/metric/template	api_metric_template
POST	/studio/api/metric/validate	api_metric_validate
GET	/studio/api/models	api_models
POST	/studio/api/overlay/compile	api_overlay_compile
POST	/studio/api/overlay/generate	api_overlay_generate
POST	/studio/api/overlay/install	api_overlay_install
GET POST DELETE	/studio/api/overlay/port	api_overlay_port
GET	/studio/api/overlay/template	api_overlay_template
GET	/studio/api/runner/get/<name>	api_runner_get
GET	/studio/api/runner/list	api_runner_list
POST	/studio/api/runner/save	api_runner_save
POST	/studio/api/runner/validate	api_runner_validate
POST	/studio/api/weightmode/compile	api_wm_compile
POST	/studio/api/weightmode/generate	api_wm_generate
POST	/studio/api/weightmode/install	api_wm_install
GET POST DELETE	/studio/api/weightmode/port	api_wm_port
GET	/studio/api/weightmode/template	api_wm_template
GET	/studio/dati-esterni	data_esterni_page
GET	/studio/runner-live	runner_live_page

9.4.3 community.py — blueprint `community`

Metodi	Percorso	Funzione
GET	/account	account
GET	/api/abbonamento	abbonamento_stato

Metodi	Percorso	Funzione
POST	/api/abbonamento/checkout	abbonamento_checkout
POST	/api/abbonamento/portale	abbonamento_portale
POST	/api/account	account_update
POST	/api/community/feedback	feedback_create
POST	/api/feedback	api_submit_feedback
GET	/api/marketplace/<artifact_id>	marketplace_detail
POST	/api/marketplace/<artifact_id>/buy	marketplace_buy
POST	/api/marketplace/<artifact_id>/comments	marketplace_comment
GET	/api/marketplace/<artifact_id>/conflicts	marketplace_conflicts
POST	/api/marketplace/<artifact_id>/contesta	marketplace_contesta
POST	/api/marketplace/<artifact_id>/install	marketplace_install
POST	/api/marketplace/<artifact_id>/installa-admin	marketplace_installa_admin
GET	/api/marketplace/<artifact_id>/issue-stato	marketplace_issue_stato
GET	/api/marketplace/<artifact_id>/perche-non-mio	marketplace_perche_non_mio
POST	/api/marketplace/<artifact_id>/remove	marketplace_remove
POST	/api/marketplace/<artifact_id>/riconcilia	marketplace_riconcilia
POST	/api/marketplace/<artifact_id>/scheda	marketplace_modifica_scheda
GET	/api/marketplace/acquisti	marketplace_acquisti
GET	/api/marketplace/claim/<claim_id>	marketplace_claim
POST	/api/marketplace/claim/<claim_id>/chiudi	marketplace_claim_chiudi
POST	/api/marketplace/claim/<claim_id>/messaggio	marketplace_claim_messaggio
POST	/api/marketplace/claim/<claim_id>/verifica-integrita	marketplace_claim_integrita
POST	/api/marketplace/claim/<claim_id>/verifica-performance	marketplace_claim_performance
GET	/api/marketplace/claim/aperte	marketplace_claim_aperte
GET	/api/marketplace/contestazioni	marketplace_contestazioni
GET	/api/marketplace/diagnostica	marketplace_diagnostica
GET	/api/marketplace/items	marketplace_items_api
GET	/api/marketplace/mine	marketplace_mine
POST	/api/marketplace/publish	marketplace_publish
GET	/api/marketplace/system-artifacts	marketplace_system_artifacts
GET	/api/marketplace/system-artifacts/<kind>/<path:name>	marketplace_system_artifact
GET	/api/marketplace/video-check	marketplace_video_check
GET POST	/api/messages/<conversation_id>	messages_send, messages_thread
GET	/feedback	feedback
GET	/marketplace	marketplace
GET	/marketplace/<artifact_id>	marketplace_item
GET	/marketplace/acquisto/<artifact_id>	marketplace_purchase_return
GET	/marketplace/rimborso/<claim_id>	claim_page
GET	/marketplace/sell	marketplace_sell
GET	/messages	messages

9.4.4 marketplace_routes.py — sul blueprint `community`

Metodi	Percorso	Funzione
GET	/api/market/admin/payments	market_admin_payments
POST	/api/market/admin/refund	market_admin_refund
POST	/api/market/buy/<artifact_id>	market_buy
GET	/api/market/catalog	market_catalog
GET	/api/market/earnings	market_earnings
POST	/api/market/install/<artifact_id>	market_install
GET	/api/market/item/<artifact_id>	market_item
GET	/api/market/license/<artifact_id>	market_license
POST	/api/market/publish	market_publish
GET POST	/api/market/seller	market_seller, market_seller_azione
POST	/api/market/seller/connect	market_seller_connect

9.4.5 agent.py — blueprint `agent`

Metodi	Percorso	Funzione
GET	/agent	page
GET POST	/agent/api/chats	api_chat_create, api_chats
POST	/agent/api/chats/<chat_id>/delete	api_chat_delete
POST	/agent/api/chats/<chat_id>/message	api_chat_message
GET	/agent/api/chats/<chat_id>/messages	api_chat_messages
POST	/agent/api/chats/<chat_id>/prepare	api_chat_prepare
GET	/agent/api/chats/<chat_id>/progress	api_chat_progress
POST	/agent/api/chats/<chat_id>/rename	api_chat_rename
GET	/agent/api/contexts	api_contexts

9.4.6 genetic.py — blueprint `genetic`

Metodi	Percorso	Funzione
GET	/genetic	page
GET	/genetic/api/config	api_config
POST	/genetic/api/job/<job_id>/installa	api_installa_respinta
GET	/genetic/api/jobs	api_jobs
GET	/genetic/api/jobs/<job_id>	api_job
POST	/genetic/api/jobs/<job_id>/cancel	api_job_cancel
POST	/genetic/api/run	api_run
GET	/genetic/api/strategies	api_strategies

9.4.7 auth.py — dentro `init_auth(app)`

Metodi	Percorso	Funzione
GET	/auth/google/callback	auth_google_callback
GET POST	/login	auth_login

Metodi	Percorso	Funzione
GET	/login/google	auth_google_start
GET	/login/google/pronto	auth_google_pronto
GET	/logout	auth_logout
GET POST	/register	auth_register
GET POST	/reset	auth_reset
GET POST	/verify	auth_verify

9.4.8 ui_ext_a.py — estensione v2

Metodi	Percorso	Funzione
POST	/api/a/metric_preview	a_metric_preview
GET	/api/a/overview	a_overview
GET	/api/a/stream/<name>/meta	a_stream_meta

9.4.9 ui_ext_b.py — estensione v2

Metodi	Percorso	Funzione
GET	/ensemble-custom	b_ensemble_custom_alias
GET	/projects/<path:pid>	b_project_detail

9.4.10 ui_ext_e.py — estensione v2

Metodi	Percorso	Funzione
GET	/api/e/account-data	e_account_data
GET	/api/e/sell-status	e_sell_status

9.4.11 ui_ext_f.py — estensione v2

Metodi	Percorso	Funzione
GET	/api/f/account	f_account
GET	/api/f/subscription/invoices	f_subscription_invoices

Quanti sono

Questa tabella elenca 250 percorsi distinti, da 271 decoratori `@route` su 11 moduli. Se i numeri non coincidono con quelli del progetto che si ha davanti, la tabella è vecchia: si rigenera con `strumenti/genera-rotte.py`.

Indice analitico

Approccio, 5
Artefatto, 6

backtest, 2

Ensemble, 5
ensemble, 2

job, 12

live, 2

Modalità di pesatura, 5
modalità di pesatura, 5

Overlay, 5
overlay, 2

Porta, 6

Regime, 5

segnali, 4
strangler fig, 4
Strategia, 4
Stream, 5
stream, 2

token, 4

Walk-forward, 5
walk-forward, 2